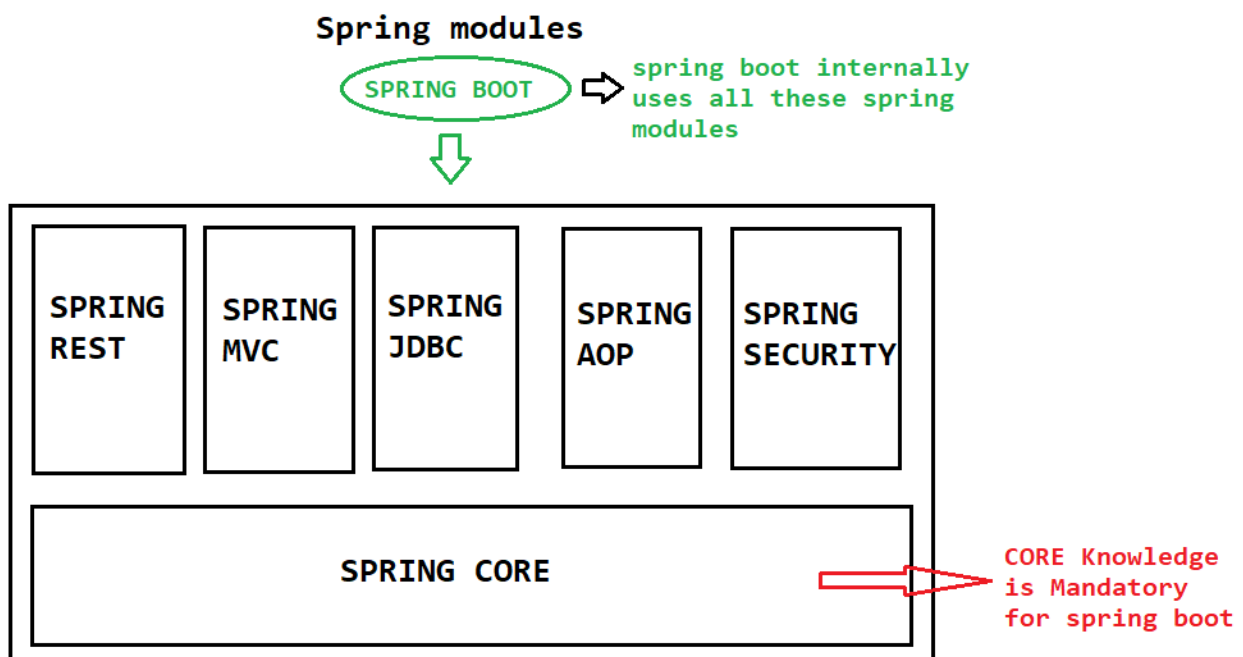


SPRING CORE & SPRING BOOT with MICROSERVICES

SPRING CORE

->To learn spring boot the minimum requirement is to know about spring core . if you know spring core very well and foundation is clear then you are eligible to learn spring boot and go a head.

->spring comes with various modules . and spring boot is top of all that means if we talk about spring boot spring boot internally uses all of these modules. But to go ahead with spring boot we should know atleast about spring core.



Note:- spring boot is not a new technology or framework or alternate for spring actually spring boot internally uses all these modules only.

->before spring there is EJB but it was very complex to write so,**ROD JOHNSON** think how to develop enterprise application without using EJB ,so to reduce complexity and increase productivity he developed a framework called SPRING in **2003** so from that time to till date spring and springboot is ruling the market.

Before go ahead with spring core ,let see the software requirement for spring

-> java JDK (minimum version jdk8)

->eclipse or STS(spring tool suit) or any other..

->maven:- it comes with eclipse /sts so no need to download it externally .

2 major component of spring core.[Spring core = IOC + DI]

->inversion of control (IOC)

->dependency injection (DI)

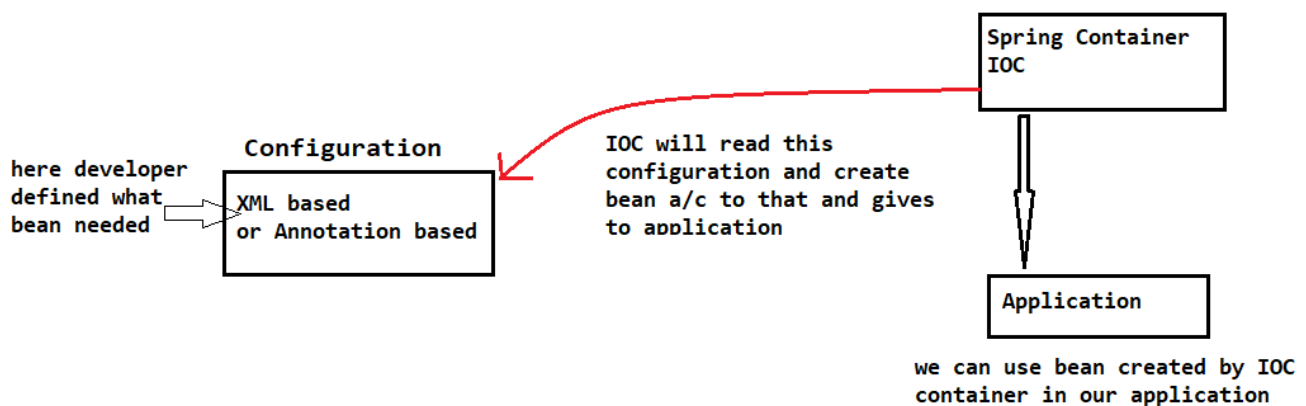
Inversion of Control:

->The design pattern responsible for handling the creation of objects defined in the configuration and providing that to application. In simple words giving the control of creating and instantiating the object to IOC container. our spring IOC container takes the responsibility to create and manage the objects only developer need to tell IOC container in form of configuration that he requires that object ,

->in spring for configuration we can use Xml based configuration or annotation based configuration. (annotation based recommended).

->spring IOC container is also called application context

->Note:- as we know IOC container create and instantiating the object but we not use word "Object" in spring because this object is managed by our IOC so we called it "**Bean**" because its life cycle is managed by IOC.



Steps to create spring first program.

->open eclipse/STS or any IDE

->create maven project

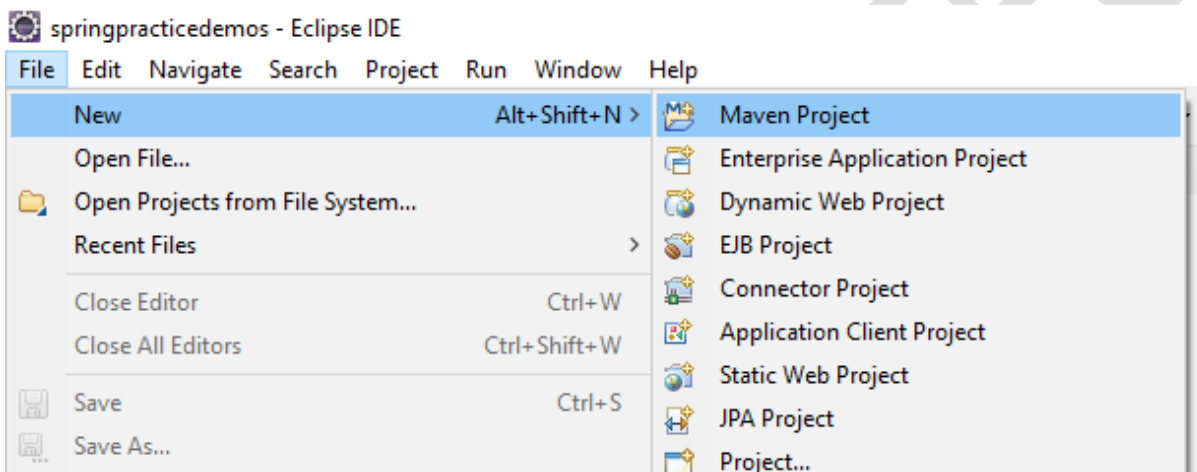
Why Maven Project ?

->see Maven is build tool ,we will learn about maven later it is separate topic but as off now just for knowledge you remember that maven is build tool which help developer to import 3rd party libraries from external source (MVN repository).

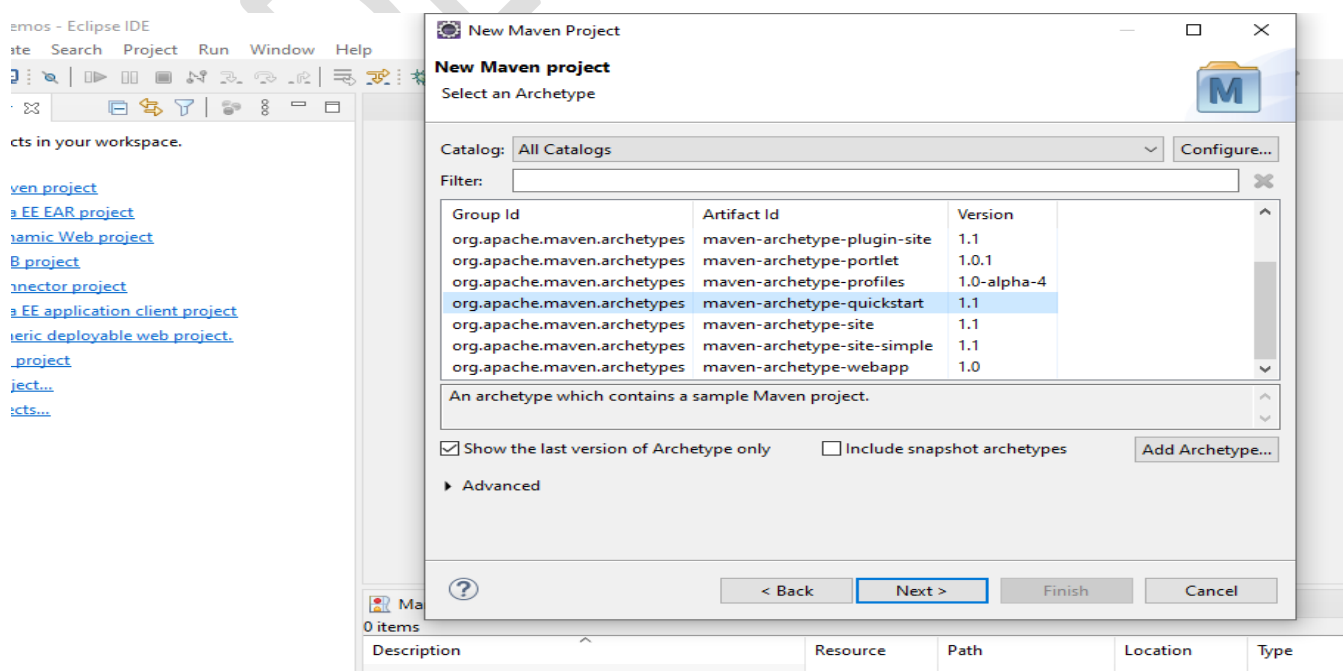
->see when we develop any project let take example of spring project then we requires lot of libraries which present in external source like MVN repository then if we manually download each and every library then it take lot of time as well as version mismatch issue can also occur and lot of issue may come so maven build tools is came into picture

->maven tool automatically downloads all the dependent libraries from mvn repo and import it to our application without any version mismatch issue.

->this is not complete picture of maven but for as off now its enough.



->after that select maven-archetype-quickstart.(just to create simple maven standalone project and it give basic structure as well as boiler code)



->now put project details.

New Maven Project

Specify Archetype parameters

Group Id: finance

Artifact Id: loandept

Version: 0.0.1-SNAPSHOT

Package: finance.loandept

Properties available from archetype:

Name	Value

Advanced

< Back Next > Finish Cancel

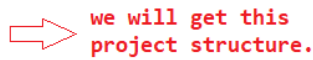
Group Id:- you need to tell your project comes under which group ex:- suppose in your company lots of project are developed related to finance department and this project also related to finance only so you can say that this project comes under finance group that is nothing but group id.

artifact id:-artifact id means here you need to tell purpose like your project belongs to finance group but under finance there are so many things our project related to which thing we need to specify here.ex:- under finance our project related to loan.

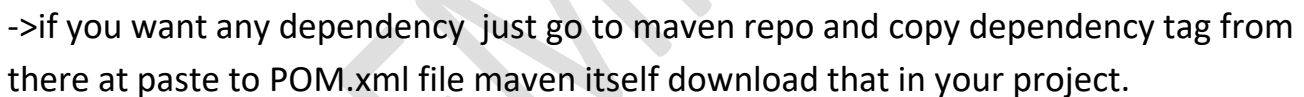
Version:- here you need to specify project version.

Package:-just specify project package in which package you need to put your project.

->click on finish.



Note:- maven internally download all the dependency from **maven central repository**.



License	Apache 2.0
Categories	Dependency Injection
Organization	Spring IO
HomePage	https://github.com/spring-projects/spring-framework
Date	(Apr 13, 2022)
Files	pom (2 KB) jar (1.2 MB) View All
Repositories	Central
Used By	11,754 artifacts

Maven
Gradle
Gradle (Short)
Gradle (Kotlin)
SBT
Ivy
Grape
Leiningen
Buildr

```
<!-- from https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.19</version>
</dependency>
```

copy
in POM

☒ Include comment with link to definition

copy this and paste
in POM.xml

NOTE:- here i am using spring version 5.3.19 it is not mandatory to use same version because in spring frequently new version is realised . if this version is not worked or compatiabile then try another version. Take any stable version by your own.

->we have created project but still it is maven project only to make it spring project and to use spring core functionality we need to add **spring-context** dependency .to add this add spring-context dependency in POM.xml.

->go to maven repository search spring-context and add in pom file.

License	Apache 2.0
Categories	Dependency Injection
Organization	Spring IO
HomePage	https://github.com/spring-projects/spring-framework
Date	(Apr 13, 2022)
Files	pom (2 KB) jar (1.2 MB) View All
Repositories	Central
Used By	11,754 artifacts

Maven Gradle Gradle (Short) Gradle (Kotlin) SBT Ivy Grape Leiningen Buildr

```
<!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.19</version>
</dependency>
```

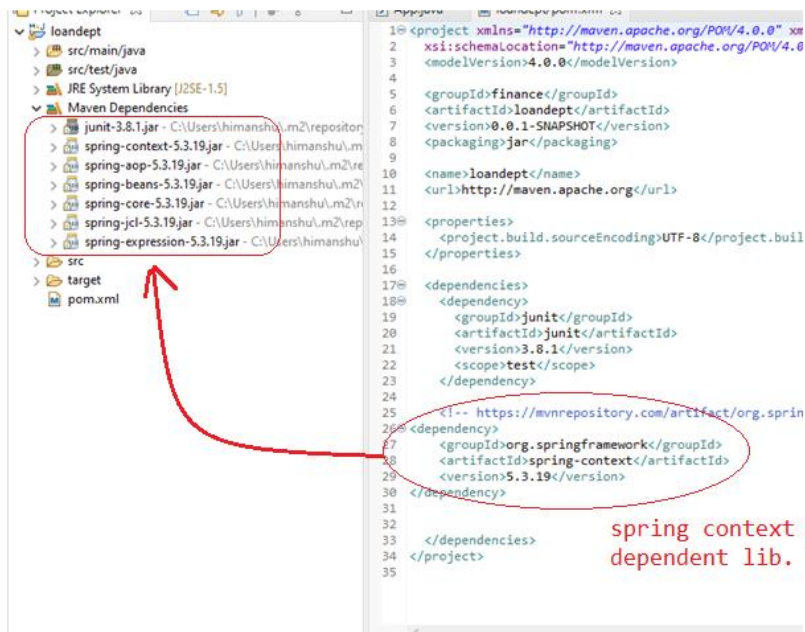
☒ Include comment with link to declaration

```
5  <groupId>finance</groupId>
6  <artifactId>loandep</artifactId>
7  <version>0.0.1-SNAPSHOT</version>
8  <packaging>jar</packaging>
9
10 <name>loandep</name>
11 <url>http://maven.apache.org</url>
12
13 <properties>
14   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15 </properties>
16
17 <dependencies>
18   <dependency>
19     <groupId>junit</groupId>
20     <artifactId>junit</artifactId>
21     <version>3.8.1</version>
22     <scope>test</scope>
23   </dependency>
24
25   <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
26   <dependency>
27     <groupId>org.springframework</groupId>
28     <artifactId>spring-context</artifactId>
29     <version>5.3.19</version>
30   </dependency>
31
32 </dependencies>
33 </project>
```

Note:- maven first see in local repository in your system if it available there then it use it from there otherwise it download from internet.

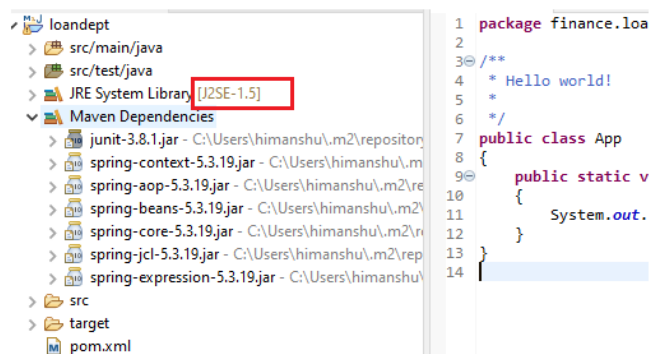
->maven also see version if dependency already present in local but it is not compatible with version then it download again from central repo.

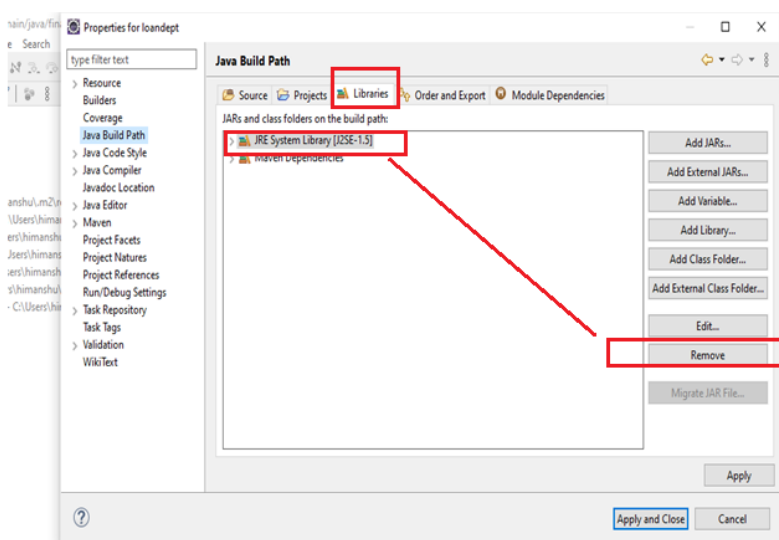
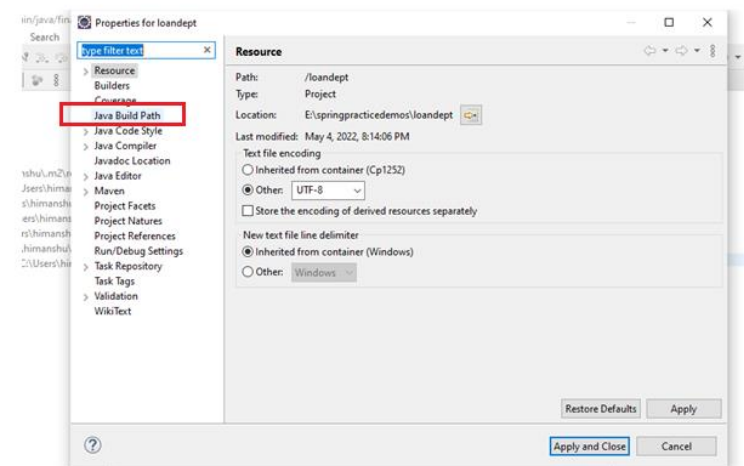
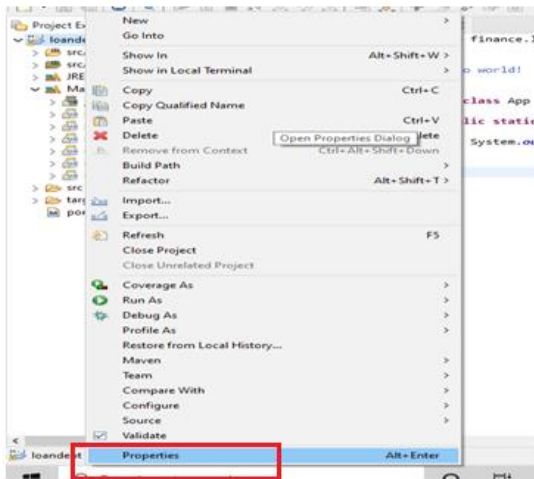
Local repo:- path-> c:\Users\codemines\.m2\repository

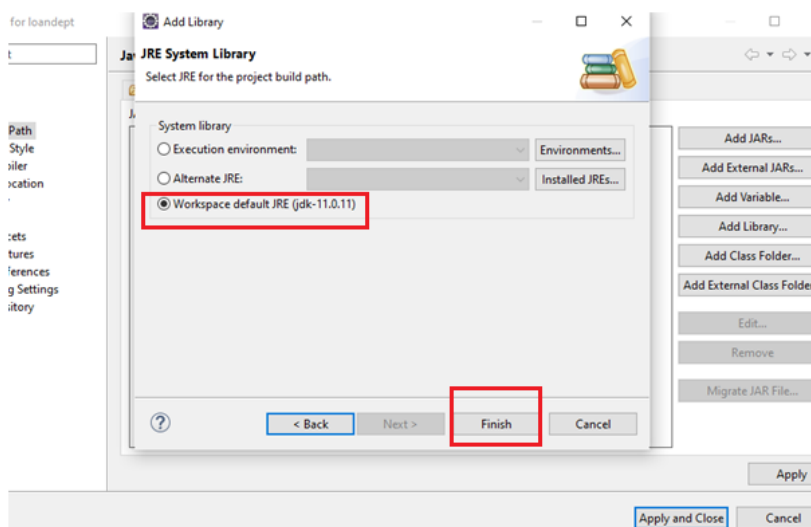
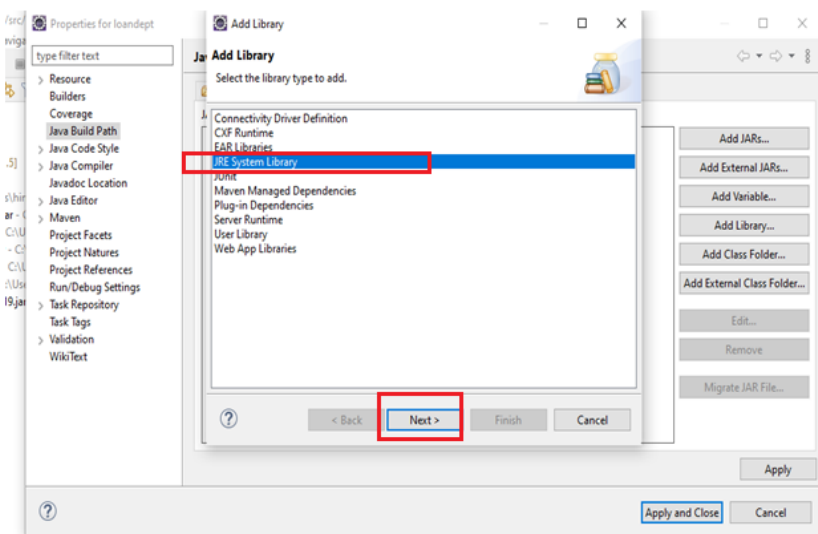
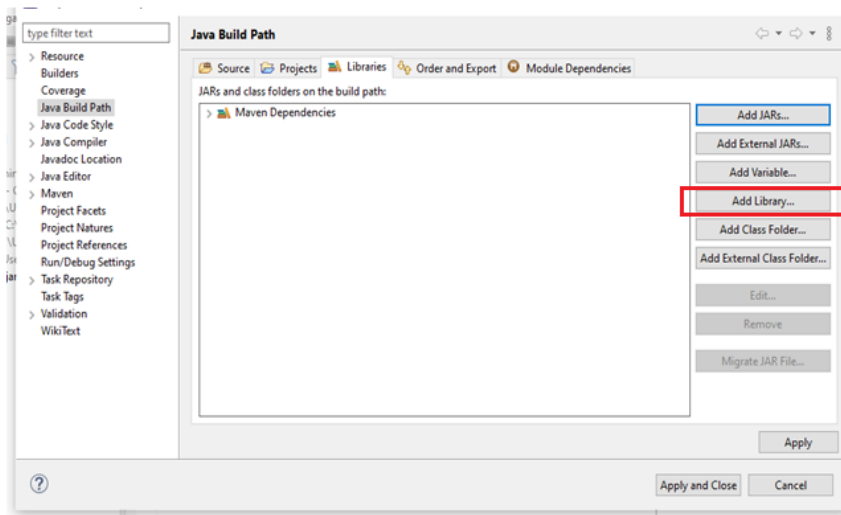


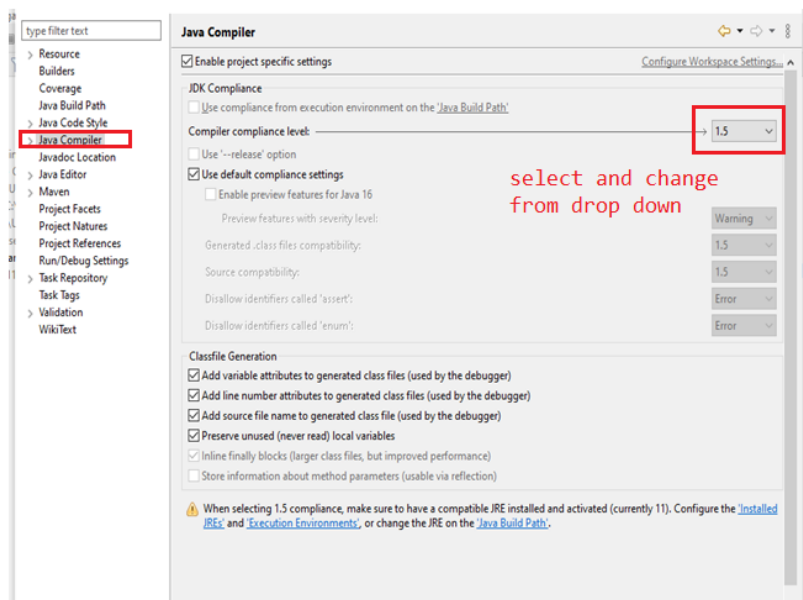
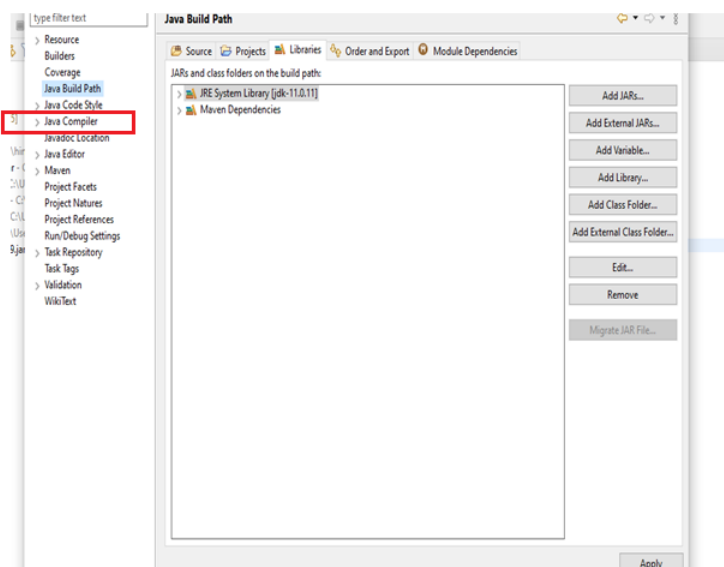
spring context gives all the dependent lib.

Now to change default JDK configuration in eclipse if required then follow these steps.









In above we have change old jdk to new jdk as well as compiler config also and here we are ready to go ahead.

->as of now we know how to create maven project and how to make it spring project by adding spring-context dependency using POM file with help of maven build tool.

->we have also seen about IOC container .what is IOC ,what it responsibility now we will see these topics indeep.

Overll small picture get cleared now go deep dive inside these topics..

Configuration file

->as we know in spring .IOC will create bean and provide it in our application but we need to defined configuration regarding that means what kind of bean we need so that IOC container/application context can full fill our demand.

->we can give configuration in two ways

1)XML based approach

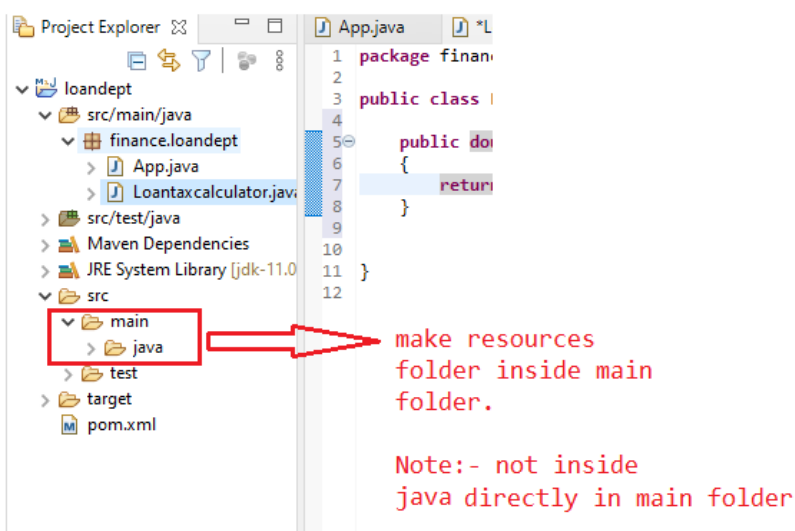
2)annotation based approach (recommended)

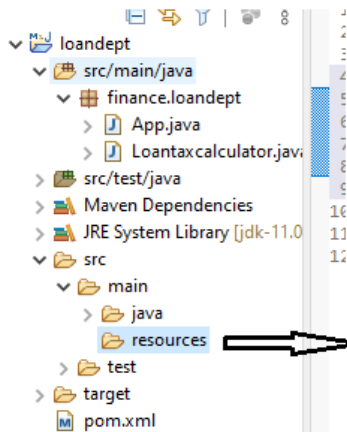
XML based approach

->in this approach we will make one xml file and defined all bean related configuration there and IOC container will read that configuration from that XML file and create & initiate bean depending on configuration you have written in that XML file.

->but where to keep that configuration XML file because we get one project structure and there are to lots of different –different packages and folder..

So to keep XML configuration file we need to make one “**resources**” folder in eclipse. In spring boot bydefault this “**resources**” folder comes we no need to create there.





folder name should
always "resources"
only.

you can now make xml
configuration file
inside it. you can
take any name for
configuration file
but in XML approach
extension should be
.xml

When you create xml file for configuration then we need to write xml tags inside it for configuration so to start any tag first we need to write basic xml schema which you no need to remember you can just see it from net .

See basic schema

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<!-- write beans configuration here -->

</beans>
```

Now we have one class suppose

```
package finance.loandept;

public class Loantaxcalculator
{
    public double myloan()
    {
        return 20.30;
    }
}
```

->we have Loantaxcalculator class and we need its object but we don't want to create manually. we want that IOC container will create and give to us then we will define that in configuration xml file that we want that Loantaxcalculator bean.

myappconfig.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="
http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-
beans.xsd">
  <bean id="loantax" class="finance.loandep.Loantaxcalculator" />
</beans>
```

to create bean we use <bean> tag

which class bean you want we need to provide that class fullyqualified name in class attribute.

this id will act as a reference variable for that bean. you can give any id but it should be unique.

->we have created configuration XML file and also define <bean> tag that we need this class bean, now give this configuration file to IOC container/application context.

->inside main class

```
App.java | Loantaxcalculator.java | myappconfig.xml
1 package finance.loandep;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5
6 public class App
7 {
8     public static void main( String[] args )
9     {
10         ApplicationContext context= new ClassPathXmlApplicationContext("myappconfig.xml");
11         Loantaxcalculator ltc=(Loantaxcalculator) context.getBean("loantax");
12         System.out.println(ltc.myloan());
13     }
14 }
15
```

ApplicationContext context= new ClassPathXmlApplicationContext("myappconfig.xml");

ClassPathXmlApplicationContext: This container takes the path of xml file from the classpath and loads the definitions of the beans from an xml file. means **classpathxmlapplicaiioncontext** will recognize where this **myappconfig.xml** is in the classpath means in the project ..and read that configuration file and create its object.

->**ApplicationContext** is nothing but IOC container. Now in above code your bean get created .now to use that bean we need to get it from IOC container so we use **getBean(unique_bean_id)** method.

```
1 package finance.loandep;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5
6 public class App
7 {
8     public static void main( String[] args )
9     {
10         ApplicationContext context= new ClassPathXmlApplicationContext("myappconfig.xml");
11         Loantaxcalculator ltc=(Loantaxcalculator) context.getBean("loantax");
12         System.out.println(ltc.myloan());
13     }
14 }
15 }
```

Some steps summary

- >first create maven project and add spring-context dependency.
- >make resource folder if it is not there.
- >define configuration xml file.
- >create main class and create ApplicationContext as well and get bean with that unique bean id.

You can also use :-

Bean Factory Container

Resource resource = new FileSystemResource("appbeans.xml");

BeanFactory factory = new XmlBeanFactory(resource);

=>If bean configuration file available outside of the project then we can load that file using FileSystemResource class like.

```
Resource resource = new FileSystemResource("D://mydemo//myconfig.xml");
```

```
BeanFactory factory = new XmlBeanFactory(resource);
```

```
Loantaxcalculator ltc=factory.getBean(Loantaxcalculator.class);
```

->ApplicationContext is more preferred way and also recommended.

->applicationContext will follow eager loading that means whenever IOC starts immediately it will create bean objects.

->BeanFactory will create bean object on demand basis that means when we call `getBean()` method then only it will create bean object (lazy loading).

->in general if you need to set class property value then we can simply set by using setter and getter , but we want so set from configuration xml file then we can set it by **<property>** tag inside bean.

Ex:

```
public class LoanTaxCalculator {  
  
    public int loanAmount;  
    private String name;  
  
    public int getLoanAmount() {  
        return loanAmount;  
    }  
  
    public void setLoanAmount(int loanAmount) {  
        this.loanAmount = loanAmount;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
}
```

Myconfig.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="  
http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-  
beans.xsd">  
  
    <bean id="Loan" class="finance.Loandepartment.LoanTaxCalculator">  
        <property name="loanAmount" value="250"/></property>  
    </bean>  
  
</beans>
```

Main class

```
public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );

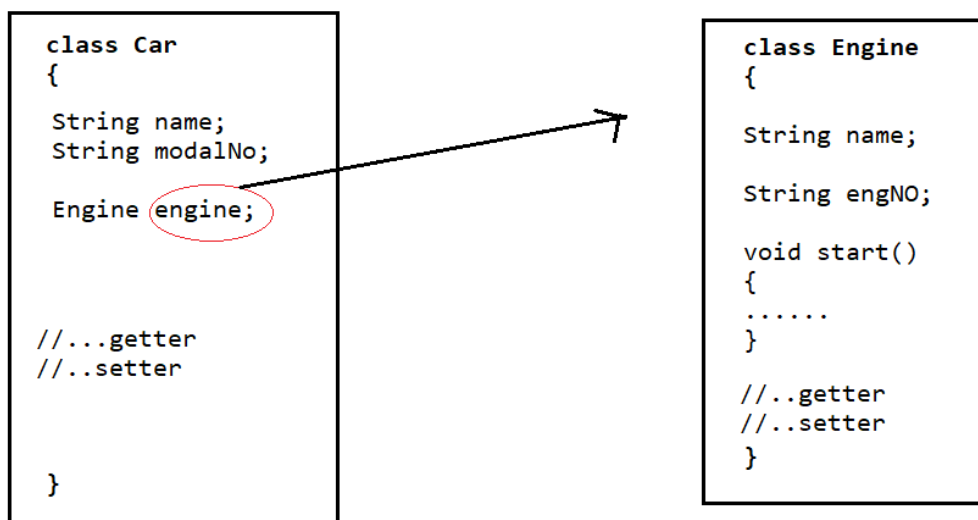
        ApplicationContext context=new ClassPathXmlApplicationContext("myconfig.xml");
        LoanTaxCalculator cal=(LoanTaxCalculator)context.getBean("loan");
        System.out.println(cal.getLoanAmount());
    }
}
```

Dependency injection

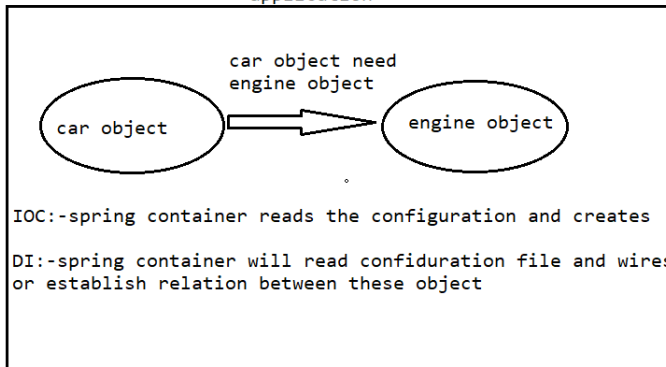
->as off now we have seen bean creation using configuration xml file. Now we deep dive into most important topic that is DI (dependency injection).

Let first know what is dependency means. See if one class object requires or need another class object for some operation or business logic then we can say that one class is dependent on another class.

Ex:- we have one Car class and we have one Engine class then car class requires Engine class object which means Car class is dependent on Engine class.



application



->spring uses dependency injection to inject that dependent object

->it is has a relation ship.

IOC:-spring container reads the configuration and creates

DI:-spring container will read confiduration file and wires or establish relation between these object

Ex:-

normal property ←

```
public class Car {  
    private String name;  
    private String carModal;  
    private int vehno;  
  
    private Engine engine; → dependent object  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getCarModal() {  
        return carModal;  
    }  
  
    public void setCarModal(String carModal) {  
        this.carModal = carModal;  
    }  
  
    public int getVehno() {  
        return vehno;  
    }  
  
    public void setVehno(int vehno) {  
        this.vehno = vehno;  
    }  
  
    public Engine getEngine() {  
        return engine;  
    }  
  
    public void setEngine(Engine engine) { ← dependency injection using setter  
        this.engine = engine;  
    }  
}
```

```
public class Engine {  
  
    private String engNo;  
    private String engineModal;  
  
    public void start() {  
        System.out.println("boom car started !!!!");  
    }  
  
    public String getEngNo() {  
        return engNo;  
    }  
  
    public void setEngNo(String engNo) {  
        this.engNo = engNo;  
    }  
  
    public String getEngineModal() {  
        return engineModal;  
    }  
  
    public void setEngineModal(String engineModal) {  
        this.engineModal = engineModal;  
    }  
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="myeng" class="finance.Loandepartment.modal.Engine">
<property name="engNo" value="102de"></property>
<property name="engineModal" value="suzuki-itch"></property>
</bean>
```

⇒ IOC will create Engine bean and also provide property value .to set its value IOC uses setter method .

injection

```
<bean id="mycar" class="finance.Loandepartment.modal.Car">
<property name="engine" ref="myeng"></property>
</bean>
```

⇒ IOC will create Car class bean and also it will see that this Car class is dependent on Engine class and it will see **ref attribute** and search that bean if available then IOC will inject it using setter method

Note:- IOC will perform this injection using setter method so we can also called it setter injection.

```
</beans>
```

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");

        ApplicationContext context = new ClassPathXmlApplicationContext("myconfig.xml");
        Car car=(Car)context.getBean("mycar");

        System.out.println(car.getEngine().getEngineModal());
        System.out.println(car.getEngine().getEngNo());
        car.getEngine().start();

    }
}
```

Note:-if we use **<property>** tag for injection then it uses setter method .which means setter injection happens. The property name should match with the setter method property name of the class. The ref bean value should match with bean id value in the xml file.

```
public class Car {
    private String name;
    private String carModal;
    private int vehno;

    private Engine engine;

    public Engine getEngine() {
        return engine;
    }

    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

```
<bean id="mycar" class="finance.Loandepartment.modal.Car">
    <property name="engine" ref="myeng"></property>
</bean>
```

← The property name should match with the setter method property name of the class

We have seen how to set normal property using <property> tag and how to inject dependent bean using ref attribute. Setter method injection is one way we have another way also ..

Dependency injection using constructor

->as we know that we can set class property by using setter method right core java concept and we also know that we can also use constructor to initialize class property. So we can also use constructor to perform dependency injection.

Constructor arg:

<constructor-arg>

<ref bean= "org" >

</constructor-arg>

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="myeng" class="finance.Loandepartment.modal.Engine">
        <property name="engNo" value="102de"></property>
        <property name="engineModal" value="suzuki-itch"></property>
    </bean>

    <bean id="mycar" class="finance.Loandepartment.modal.Car">
        <constructor-arg value="scorpio"/>
        <constructor-arg value="mahindra-zzm"/>
        <constructor-arg value="2250"/>
        <constructor-arg ref="myeng"/>
    </bean>

</beans>
```

```
public class Car {
    private String name;
    private String carModal;
    private int vehno;
    private Engine engine;

    public Car(String name, String carModal, int vehno, Engine engine)
    {
        this.name = name;
        this.carModal = carModal;
        this.vehno = vehno;
        this.engine = engine;
    }
}
```

->it will inject that bean using constructor . you can use both either setter or constructor but in constructor injection you need to maintain constructor parameter order in which order parameter are present in constructor in same order you need to pass value from xml file.

->in constructor if by mistake parameter order changes then it will cause issue so to solve it we can use one more attribute called **index**.

->now using index attribute it is not mandatory to maintain order just you need to write correct index

```

<bean id="mycar" class="finance.Loandepartment.modal.Car">
  <constructor-arg index="0" value="scorpio"/>
  <constructor-arg index="1" value="mahindra-zzm"/>
  <constructor-arg index="2" value="2250"/>
  <constructor-arg index="3" ref="myeng"/>
</bean>

public class Car {
    private String name;
    private String carModal;
    private int vehno;

    private Engine engine;

    public Car(String name, int vehno, String carModal, Engine engine) {
        this.name = name;
        this.carModal = carModal;
        this.vehno = vehno;
        this.engine = engine;
    }
}

```

->as off now we have seen how to inject single dependency now we will see how to inject list of dependencies

Using setter

```

<bean id="myeng1" class="finance.Loandepartment.modal.Engine">
  <property name="engNo" value="102de"/>
  <property name="engineModal" value="suzuki-itch"/>
</bean>

<bean id="myeng2" class="finance.Loandepartment.modal.Engine">
  <property name="engNo" value="103de"/>
  <property name="engineModal" value="hundai-itch"/>
</bean>

<bean id="myeng3" class="finance.Loandepartment.modal.Engine">
  <property name="engNo" value="104de"/>
  <property name="engineModal" value="kia-itch"/>
</bean>

<bean id="mycar" class="finance.Loandepartment.modal.Car">
  <property name="engineList">
    <list>
      <ref bean="myeng1" />
      <ref bean="myeng2" />
      <ref bean="myeng3" />
    </list>
  </property>
</bean>

public class Car {
    private String name;
    private String carModal;
    private int vehno;

    private List<Engine> engineList;

    public void setEngineList(List<Engine> engineList) {
        this.engineList = engineList;
    }
}

```

Same thing you can perform with constructor injection also..

```

<bean id="myeng1" class="finance.Loandepartment.modal.Engine">
  <property name="engNo" value="102de"/>
  <property name="engineModal" value="suzuki-itch"/>
</bean>

<bean id="myeng2" class="finance.Loandepartment.modal.Engine">
  <property name="engNo" value="103de"/>
  <property name="engineModal" value="hundai-itch"/>
</bean>

<bean id="myeng3" class="finance.Loandepartment.modal.Engine">
  <property name="engNo" value="104de"/>
  <property name="engineModal" value="kia-itch"/>
</bean>

<bean id="mycar" class="finance.Loandepartment.modal.Car">
  <constructor-arg values="ritz"/>
  <constructor-arg values="ritz-202"/>
  <constructor-arg values="1102"/>
  <constructor-arg>
    <list>
      <ref bean="myeng1" />
      <ref bean="myeng2" />
      <ref bean="myeng3" />
    </list>
  </constructor-arg>
</bean>

public class Car {
    private String name;
    private String carModal;
    private int vehno;

    private List<Engine> engineList;

    public Car(String name, String carModal, int vehno, List<Engine> engineList) {
        super();
        this.name = name;
        this.carModal = carModal;
        this.vehno = vehno;
        this.engineList = engineList;
    }
}

```

```

<bean id="myeng1" class="finance.loandepartment.modal.Engine">
  <property name="engNo" value="102de"></property>
  <property name="engineModal" value="suzuki-itch"></property>
</bean>

<bean id="myeng2" class="finance.loandepartment.modal.Engine">
  <property name="engNo" value="103de"></property>
  <property name="engineModal" value="hundai-itch"></property>
</bean>

<bean id="myeng3" class="finance.loandepartment.modal.Engine">
  <property name="engNo" value="104de"></property>
  <property name="engineModal" value="kia-itch"></property>
</bean>

<bean id="mycar" class="finance.loandepartment.modal.Car">
  <property name="engine">
    <set>
      <ref bean="myeng1" />
      <ref bean="myeng2" />
      <ref bean="myeng3" />
    </set>
  </property>
</bean>

```

for set type

```

private String name;
private String carModal;
private int vehno;
private Set<Engine> engine;

```

->same logic we can perform with constructor injection ...

->for map value

```

<bean id="myeng1" class="finance.loandepartment.modal.Engine">
  <property name="engNo" value="102de"></property>
  <property name="engineModal" value="suzuki-itch"></property>
</bean>

<bean id="myeng2" class="finance.loandepartment.modal.Engine">
  <property name="engNo" value="103de"></property>
  <property name="engineModal" value="hundai-itch"></property>
</bean>

<bean id="myeng3" class="finance.loandepartment.modal.Engine">
  <property name="engNo" value="104de"></property>
  <property name="engineModal" value="kia-itch"></property>
</bean>

<bean id="mycar" class="finance.loandepartment.modal.Car">
  <property name="engine">
    <map>
      <entry key="old engine" value-ref="myeng1" />
    </map>
  </property>
</bean>

```

```

private String name;
private String carModal;
private int vehno;
private Map<String,Engine> engine;

```

->same concept apply using constructor injection..

Autowiring

->as off now in above cases we are doing manually wiring of objects means we are injecting beans using <ref> tag and all inside property tag or constructor arg tag.

->but for small number of objects it's ok but in real time there are lots of object we require to inject then this manual wiring approach is not recommended just for ex if we have 50 object then we need to write 50 bean tag and inject it .it become complex .

->to solve this autowiring came into picture. Auto wiring solve all the manual dependencies specified in the configuration and make DI simple.

Now in autowiring we have 3 types:

1)byname

2)byType

3)constructor

Autowiring byName

-> The dependent property name (variable) in the class is matched with bean id in the configuration file and if they are matched then ioc will inject that particular bean by using setters simple

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd"
xmlns:util="http://www.springframework.org/schema/util">

<bean id="engine" class="finance.Loandepartment.modal.Engine" />

<bean id="car" class="finance.Loandepartment.modal.Car" autowire="byName" />

</beans>
```

```
public class Car {

    private String name;
    private String carModel;
    private int vehno;

    private Engine engine;

    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

IOC will search is there any engine bean available if present inject it



ioc will create engine bean and ioc will create car bean as IOC will see autowire byName it go to Car class and see what dependent object should be inject then it match that property name with bean id ,if any bean id with that name available then IOC will inject that bean. IOC also check bean Type is matched or not then injects

ex: here in Car class IOC found property name 'engine' then ioc will see is there any bean id matching with name 'engine' if available then ioc will inject it.

->above we just mention **autowire="byname"** IOC automatically inject that dependent bean.

Autowire by Type

->in this dependent property type in the class is matched with the bean class type. If both are matched then IOC container will inject that specific bean. We need to use **autowire="byType"**.

->**Note:-** if more than 1 bean is found with same class type in XML configuration file, autowire byType give error. We can fix this in annotation based configuration.

->autowired byType means ico will match type not bean id so you can give any bean id.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd"
       xmlns:util="http://www.springframework.org/schema/util">

    <bean id="tataeng" class="finance.Loandepartment.modal.Engine" />

    <bean id="car" class="finance.Loandepartment.modal.Car" autowire="byType" />

</beans>
```

```
public class Car {

    private String name;
    private String carModal;
    private int vehno;

    private Engine engine;

    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

matching type

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");

        ApplicationContext context = new
            ClassPathXmlApplicationContext("myconfig.xml");
        Car car=(Car)context.getBean("car");
        car.getEngine().setEngineModal("kia1007");

        Engine engine = car.getEngine();
        System.out.println(engine.getEngineModal());

    }
}
```

```
<bean id="tataeng" class="finance.Loandepartment.modal.Engine" />
<bean id="suzukieng" class="finance.Loandepartment.modal.Engine" />
<bean id="car" class="finance.Loandepartment.modal.Car" autowire="byType" />
```

two bean having same type now ioc will confuse in byType injection . it will raise NoBeanQualifi. exception.

Autowire by constructor

->autowire by constructor uses constructor for dependency injection.

->autowire by constructor first inject bean based on its type means if type matched then it inject that particular bean.

Note:- we have seen that if more than 1 bean having same type then **autowire byType** generate exception. But when we use **autowire by Constructor**. Then first it matched on the basis of by Type but if ioc found more than 1 bean with same type then IOC will try to match constructor parameter name with bean id .if matched then it will perform the injection otherwise raise an exception.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd"
       xmlns:util="http://www.springframework.org/schema/util">
```

```
    <bean id="tataeng" class="finance.Loandepartment.modal.Engine" />
    <bean id="nexoneng" class="finance.Loandepartment.modal.Engine" />
    <bean id="car" class="finance.Loandepartment.modal.Car" autowire="constructor" />
```

```
</beans>
```

in autowireby constructor first matching happen on basis of type . if more than 1 bean with same type then constructor try to match with constructor parameter name with bean id

```
public class Car {
    private String name;
    private String carModel;
    private int vehno;
    private Engine nexoneng1;
    public Car(Engine nexoneng) {
        this.nexoneng1 = nexoneng;
    }
}
```

Annotations based configuration

->as we seen in XML based configuration that we need to create separate XML file and IOC will read it and perform DI etc. But this is little stressful for developer , developer need to remember lots of XML tags as well as need to maintain separate XML file .

->to solve above problem annotation came into picture. Actually in java annotation introduced from java 5. Now spring provides lots of annotation and now IOC will read that annotation and perform its task now no need to maintain separate XML file for that.

->let discuss some of most important annotation .

Some Important annotations in Spring IOC and DI

@Configuration : @Configuration indicates that the class has @Bean definition methods or @Component scan to scan for the @Component so that Spring Container can process the class and generates Spring Beans tied to application Context

@Bean : Method level annotation used to explicitly declare a Spring Bean

@Component: Class Level annotation which will be detected by @ComponentScan for creating Spring Beans

@Autowired: Enables to inject the object dependency, can be used on Setter method, field, Constructor

@Qualifier: if there is more than 1 bean with same class type, using @Qualifier we can match the bean id to be injected

@Qualifier is placed along with @Autowired

@Primary: If we can declare @Primary in any of the @Bean/@Component, then by default the bean

with @Primary gets injected if there is a conflict in autowiring of the beans with same class type provided there is no @Qualifier matching present

@Scope : Indicates the name of the bean scope to be used like Prototype(default is singleton if no @Scope is present). Can be used along with @Bean or @Component

@Lazy : Indicates the lazy loading of Bean , can be used along with @Bean or @Component

@ComponentScan: Scans for the @Components in the specified base package, default base package is the package where @Configuration class is present

->in annotation based approach we no need to make any separate XML file .instead of that we make our java class as a configuration by using @Configuration annotation at class level.

->like in XML approach in xml configuration file we declare bean by using <bean> tag. Similarly in annotation approach we create bean annotated method by using @Bean annotation,so IOC will call that @Bean annotated method and return the object.

In main class to load configuration class we need to use

ApplicationContext context=new
AnnotationConfigApplicationContext(AppConfig.class);

IOC will read this configuration class and call the methods with @Bean annotated, in that we will write code for creating new object,we are writing code for creating object but method is called by IOC..



```
@Configuration
public class AppConfig {    method name is act as
                             bean id

    @Bean
    public Employee employee() {

        Employee emp = new Employee();
        emp.setEaddress("jbp");
        emp.setEmpId(101);
        emp.setName("akash");
        return emp;
    }
}
```

```
public class Employee {

    private int empId;
    private String ename;
    private String eaddress;
    public int getEmpId() {
        return empId;
    }
    public void setEmpId(int empId) {
        this.empId = empId;
    }
    public String getEname() {
        return ename;
    }
    public void setName(String ename) {
        this.ename = ename;
    }
    public String getEaddress() {
        return eaddress;
    }
    public void setAddress(String eaddress) {
        this.eaddress = eaddress;
    }
}
```

```
public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );

        ApplicationContext context=new AnnotationConfigApplicationContext(AppConfig.class);
        Employee emp =(Employee) context.getBean("employee");

        System.out.println(emp.getEmpId());
        System.out.println(emp.getEname());
        System.out.println(emp.getEaddress());
    }
}
```

->now if we want to inject some dependent object using annotation then we can go with

@Autowired

```
public class Employee {  
  
    private int empId;  
    private String ename;  
    private String eaddress;  
  
    @Autowired  
    private Department department;  
  
    public Department getDepartment() {  
        return department;  
    }  
  
    public void setDepartment(Department department) {  
        this.department = department;  
    }  
  
    //.....getter  
    //setter  
}
```

Employee is dependent on department

```
public class Department {  
    private int deptId;  
    private String deptName;  
    private String deptHead;  
  
    //setter  
    //getter  
}
```

@Configuration

```
public class AppConfig {
```

@Bean

```
public Employee employee() {
```

```
    Employee emp = new Employee();  
    emp.setEaddress("jbp");  
    emp.setEmpId(101);  
    emp.setEname("akash");  
    return emp;  
}
```

@Bean

```
public Department department() {  
    Department dept = new Department();  
    dept.setDeptHead("boss");  
    dept.setDeptId(102);  
    dept.setDeptName("health");  
    return dept;  
}
```

```
}//
```

IOC calls this @Bean annotated method and create bean of Employee class and IOC found one dependent object i.e. Department in Employee class with @Autowired annotation

->then IOC will again come to configuration class and search for department bean as it found it will inject it.

```
public class Employee {
```

```
    private int empId;  
    private String ename;  
    private String eaddress;
```

@Autowired

```
    private Department department;
```

IOC will find department bean in configuration class if available IOC will inject it

```

public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");

        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        Employee emp = (Employee) context.getBean("employee");

        Department depart = emp.getDepartment();

        System.out.println(depart.getDeptHead() + " " + depart.getDeptId() + " " + depart.getDeptName());

        System.out.println(emp.getEmpId());
        System.out.println(emp.getEname());
        System.out.println(emp.getEaddress());
    }
}

```

Note:- by default @Autowired inject bean by matching 'byType' if same type of bean is present then it inject on basis of byname. In annotation this single annotation will work both byType ,byname.(in xml we need to mention separate byname,ByType)

->as we know @Autowired work bydefault byType . if more then one bean existed with same type then it goes with byname.

->if in case ioc not get any bean byname also then it cause issue.

```

public class Employee {

    private int empId;
    private String ename;
    private String eaddress;

    @Autowired
    private Department department;
}

```

bydefault @Autowired works byType
but here Department type 2 beans existed
now in this case @Autowired works byName
,now it will
match property name with @Bean annotated
method
name.

```

@Configuration
public class AppConfig {

    @Bean
    public Employee employee() {
        Employee emp = new Employee();
        emp.setEaddress("jbp");
        emp.setEmpId(101);
        emp.setEname("akash");
        return emp;
    }

    @Bean
    public Department department() {
        Department dept = new Department();
        dept.setDeptHead("himanshu");
        dept.setDeptId(108);
        dept.setDeptName("management");
        return dept;
    }

    @Bean
    public Department department1() {
        Department dept = new Department();
        dept.setDeptHead("boss");
        dept.setDeptId(102);
        dept.setDeptName("management");
    }
}

```

```

@Component("myemp")
public class Employee {

    private int empId;
    private String ename;
    private String eaddress;

    @Autowired
    private Department department;

    public Department getDepartment() {
        return department;
    }

}

```

if you not create any setter method then spring will generate its own setter methods and inject the department object.

```

@Component("myemp")
public class Employee {

    private int empId;
    private String ename;
    private String eaddress;

    private Department department;

    public Department getDepartment() {
        return department;
    }

    @Autowired
    public void setDepartment(Department department) {
        this.department = department;
    }

}

```

spring will call the setter method written by you and inject the object

Note:- if you not create any setter then it implicitly inject using reflection api concept.

@Qualifier

-> if there is more than 1 bean with same class type, using @Qualifier we can match the bean id to be injected. **@Qualifier** is placed along with @Autowired.

-> means if we have more than 1 bean of same type then @Autowired will work byName but if bean name also not match in that case it will cause issue. We can use **@Qualifier** annotation and specify the name so now qualifier name must match with bean name, it doesn't matter what ever is property name

```

public class Employee {

    private int empId;
    private String ename;
    private String eaddress;

    @Autowired
    @Qualifier("new-dept")
    private Department department;

}

```

now qualifier name will be matched with bean name instead of property.

```

@Configuration
public class AppConfig {

    @Bean
    public Employee employee() {
        Employee emp = new Employee();
        emp.setEaddress("jbp");
        emp.setEmpId(101);
        emp.setName("akash");
        return emp;
    }

    @Bean("new-dept")
    public Department department() {
        Department dept = new Department();
        dept.setDeptHead("stranger");
        dept.setDeptId(1011);
        dept.setDeptName("payment");
        return dept;
    }

    @Bean
    public Department department1() {
        Department dept = new Department();
        dept.setDeptHead("boss");
        dept.setDeptId(102);
        return dept;
    }

}

```

```

public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");

        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        Employee emp = (Employee) context.getBean("employee");

        Department depart = emp.getDepartment();

        System.out.println(depart.getDeptHead() + " " + depart.getDeptId() + " " + depart.getDeptName());

        System.out.println(emp.getEmpId());
        System.out.println(emp.getEname());
        System.out.println(emp.getEaddress());

    }
}

```

@Component and @ComponentScan

->**@Component**: Class Level annotation which will be detected by @ComponentScan for creating Spring Beans.

->**@ComponentScan**: Scans for the @Components in the specified base package, default base package is the package where @Configuration class is present.

->in case of @Configuration we need to declare @Bean annotated method to create bean ,but suppose we need 50 bean then we need to create 50 bean methods . but here we have one another class level annotations i.e @ComponentScan and @Component

->@ComponentScan will scan for @Component and wherever it sees @Component ,spring ioc will create the bean automatically.

->both **@ComponentScan** and **@Component** are class level annotations.means we can use it on class level only.

->**@ComponentScan** is always associated with **@Configuration** annotations. Means when we make our configuration class by using @Configuration then we need to write @ComponentScan annotation also .

->now if you want bean of any class then declare that class with @Component annotation so that IOC will automatically scan it and create bean for you.

this is our configuration class because
@Configuration annotation is there.

@ComponentScan is also there which means now
IOC will search for the classes with @Component
annotation and create bean .

by default class name is our bean Id .



```
@Configuration
@ComponentScan
public class AppConfig {

} //
```

```
@Component
public class Employee {

    private int empId;
    private String ename;
    private String eaddress;

    @Autowired
    private Department department;

    //setter
    //getter

}
```

```
@Component
public class Department {
    private int deptId;
    private String deptName;
    private String deptHead;

    //setter
    //getter

}
```

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");

        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        Employee emp = (Employee) context.getBean("employee");
        emp.setEaddress("jbp");
        emp.setEname("john");

        emp.getDepartment().setDeptHead("codemines");
        emp.getDepartment().setDeptName("tech");

        System.out.println(emp.getDepartment().getDeptHead() + " " + emp.getDepartment().getDeptName());

        System.out.println(emp.getEmpId());
        System.out.println(emp.getEname());
        System.out.println(emp.getEaddress());

    }
}
```

->if you want to give another name of bean instead of class name then you can
provide in **@Component("name")**.

```

public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");

        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        Employee emp = (Employee) context.getBean("myemp");
        emp.setEaddress("jbp");
        emp.setName("john");

        emp.getDepartment().setDeptHead("codemines");
        emp.getDepartment().setDeptName("tech");

        System.out.println(emp.getDepartment().getDeptHead() + " " +
            emp.getDepartment().getDeptName());

        System.out.println(emp.getEmpId());
        System.out.println(emp.getEname());
        System.out.println(emp.getEaddress());
    }
}

@Component("myemp")
public class Employee {

    private int empId;
    private String ename;
    private String eaddress;

    @Autowired
    private Department department;

    //setter
    //getter
}

```

@Primary: If we can declare @Primary in any of the @Bean/@Component, then by default the bean

with @Primary gets injected if there is a conflict in autowiring of the beans with same class type provided there is no @Qualifier matching present

```

@Configuration
public class AppConfig {

    @Bean("dept")
    public Department department() {
        Department dept = new Department();
        dept.setDeptId("D100");
        dept.setDeptName("Engineering");
        return dept;
    }

    @Bean("dept1")
    @Primary
    public Department department1() {
        Department dept = new Department();
        dept.setDeptId("D102");
        dept.setDeptName("software");
        return dept;
    }

    @Bean
    public Employee employee() {
        Employee emp = new Employee();
        emp.setEmpId("E100");
        emp.setEmpName("Rajsekar");
        return emp;
    }
}

public class Department {

    private String deptId;
    private String deptName;

    public class Employee {

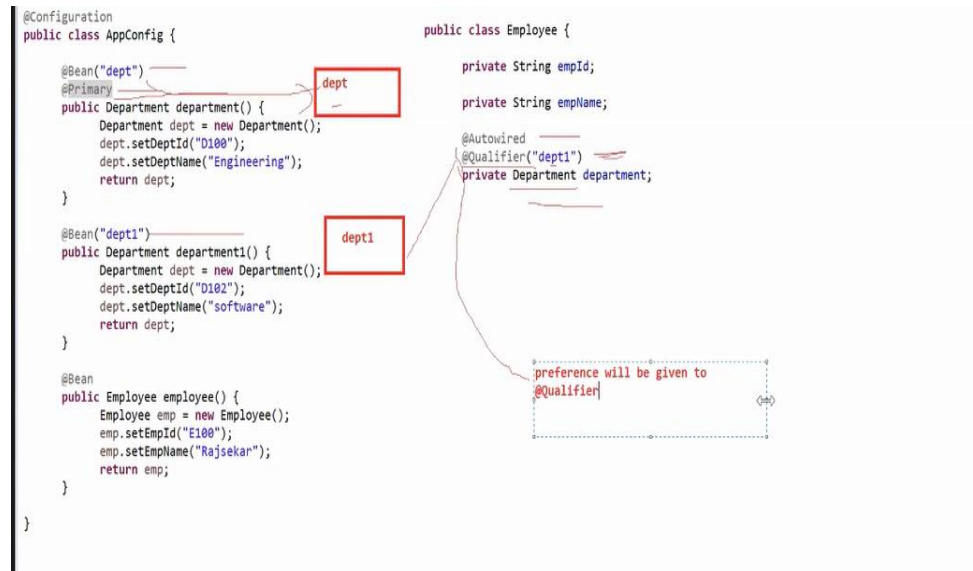
        private String empId;
        private String empName;

        @Autowired
        private Department department;
    }
}

```

1. byType

Above example two bean with same type ,we have given bean name like dept,dept1 but there is no any @qualifer annotation so now ioc will confuse .but we have also provided @Primary so IOC will give more priority to 'dept1' bean and inject it.



->if you have used both @Qualifier as well as @primary annotation then IOC will give preference to @Qualifier first then @Primary.

Bean Scope

Singleton – Same object returned for the same bean id per application context.

Prototype – Different object returned for the same bean id per application context.

-> by default every bean is singleton ,if we need to define the scope of bean as prototype ,we should declare the scope of bean as prototype during bean declaration.

as we know we have 2 approaches to create configuration

->XML based approach.

->annotation based approach.

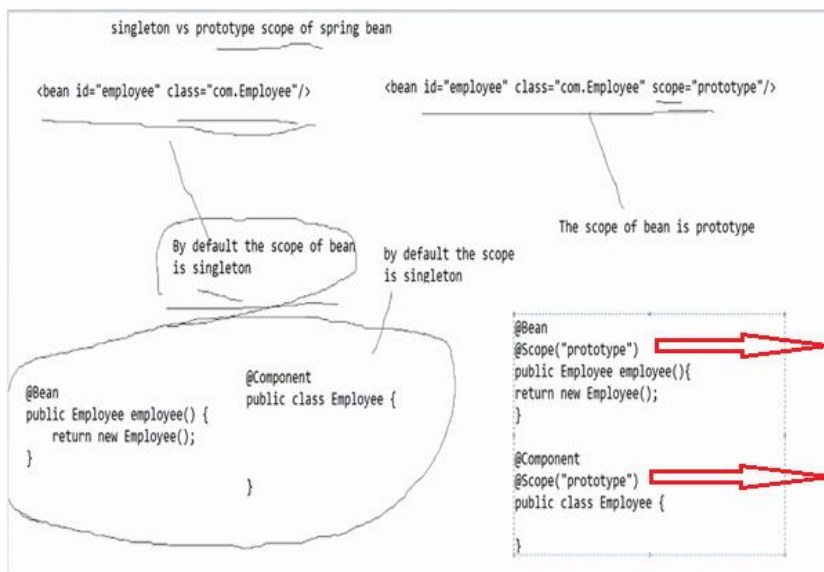
Xml

`<bean id="emp" class="com.Employee"/>` ➡ here there is no scope defined so by default it follows singleton scope .

`<bean id="emp" class="com.Employee" scope="prototype" />`

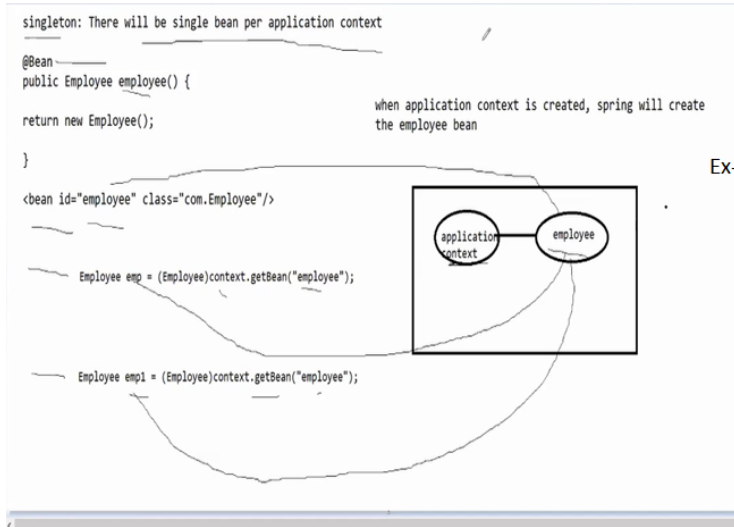
here scope="prototype" is defined so here scope of this bean is prototype

annotation based approach



with @Bean we can use like this.

with @component we can use like this.

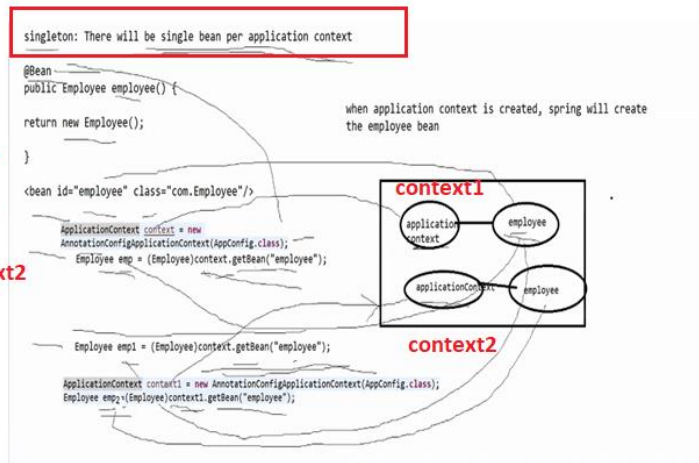


Ex- `Employee emp = (Employee)context.getBean("employee");`

`Employee emp1 = (Employee)context.getBean("employee");`

both the time we will get same bean because by default scope is singleton so if we ask same bean id bean then it will give same bean per application context

per application context means here scope is singleton only but we have create two application context so we will get different bean for employee one from context1 and another from context2



Prototype scope

->in this different bean will created every time in per application context.

```

@Component
@Scope("prototype")
public class Employee {
    private String name;
    private String city;
}

```

scope is prototype means now whenever we do `getBean("employee")` we will get new bean everytime.

```

ApplicationContext context=new
AnnotationConfigApplicationContext(Appcontext.class);

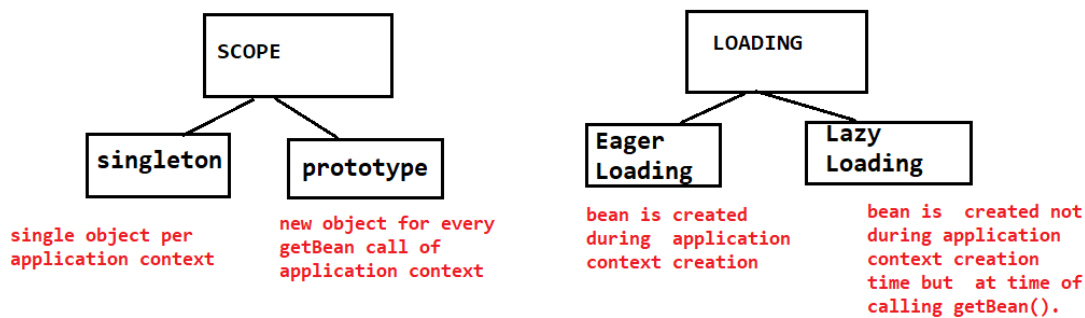
Employee e=(Employee) context.getBean("employee");
e.setName("himmu");
e.setCity("banglore");

Employee emp=(Employee)context.getBean("employee");

```

Lazy and eager Loading

SPRING



CODEMINESTECH

Eager Loading – Spring Beans are created during the application context creation.

Lazy loading – Spring Beans are created during the `context.getBean` creation.

->by default it follows eager loading. Means bean are created during the time of application context creation.

```

@Configuration
public class AppConfig {

    @Bean
    @Lazy
    //the scope of the bean is singleton as
    scope is not defined
    public Employee employee() {
        return new Employee();
    }
}

```

The bean scope is singleton
the bean loading is lazy

```

@Configuration
class AppConfig {

    @Bean
    public Employee employee() {
        return new Employee();
    }

}

```

This bean is eagerly loaded, the bean is created during the application context creation

```

@Configuration
class AppConfig{

    @Bean
    @Lazy
    public Employee employee() {
        return new Employee();
    }

}

```

The bean is lazily loaded, the bean is created during the `context.getBean` and not during the application context creation

```

<bean id="employee" class="com.Employee" lazy-init="true"/>

```

```

@Configuration
@ComponentScan
class AppConfig
{

}

@Component
@Lazy
class Employee {

}

```

CASE 1 -if scope is singleton . bean loading is eager.

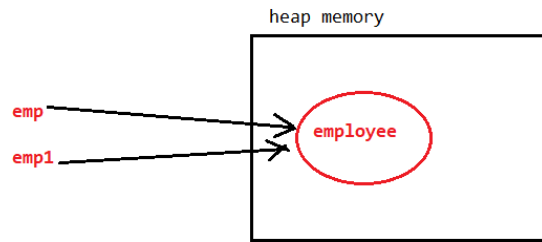
```
@Configuration
public class AppConfig
{
    @Bean
    public Employee employee()
    {
        //----//
    }
}
```

```
ApplicationContext context =new
AnnotationConfigApplicationContext(AppConfig.class);

Employee emp = (Employee) context.getBean("employee");

Employee emp1=(Employee) context.getBean("employee");
```

emp and emp1 both are point to same employee bean because employee bean is singleton.



CASE 2 - scope of bean is singleton bean loading is lazy

```
@Configuration
public class AppConfig
{
    @Bean
    @Lazy
    public Employee employee()
    {
        //----//
    }
}
```

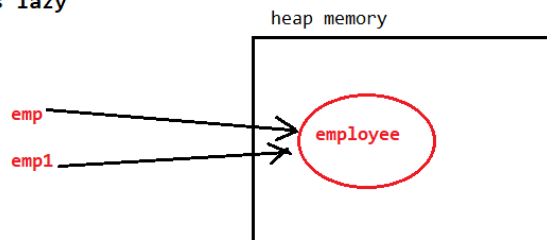
```
ApplicationContext context =new
AnnotationConfigApplicationContext(AppConfig.class);

Employee emp = (Employee) context.getBean("employee");

Employee emp1=(Employee) context.getBean("employee");
```

as the bean is lazily loaded,the bean is not created during application context creation

bean is created during the context.getBean context.



CASE 3 -scope of the bean is prototype bean loading is eager.

```
@Configuration
public class AppConfig
{
    @Bean
    @Scope("prototype")
    public Employee employee()
    {
        //----//
    }
}
```

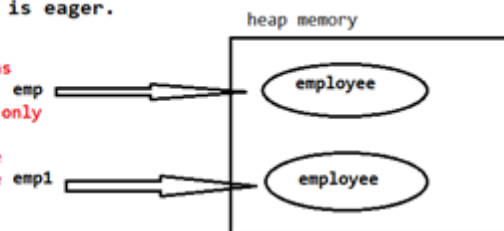
bean scope is prototype means
the new bean is created
during context.getBean call only

spring will give preference
only to bean scope prototype

```
ApplicationContext context =new
AnnotationConfigApplicationContext(AppConfig.class);

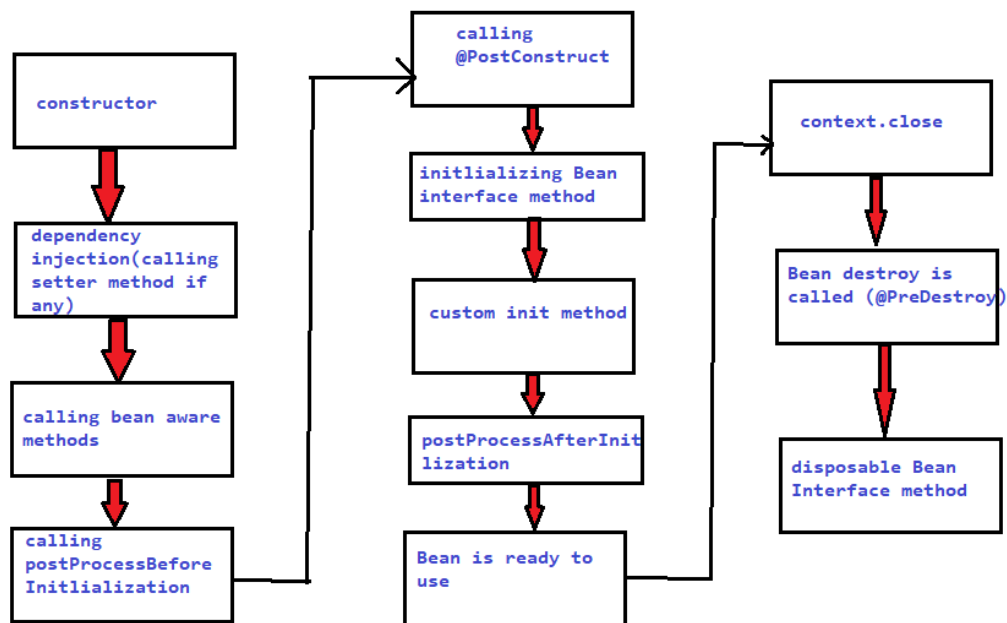
Employee emp = (Employee) context.getBean("employee");

Employee emp1=(Employee) context.getBean("employee");
```



Bean Life cycle

Bean Life Cycle in spring



@Autowired on Constructor

case 1:

in the class with @Component .suppose if we have exactly 1 parameterized constructor, this gets called even if we have @Autowired or do'nt have @Autowired on the constructor

case 2:

in the class with @Component if we have more than 1 parametrized constructor, then one of the constructor should be @Autowired otherwise it will throw error.

case 3:

in the class with @Component if we have more than 1 parametrized constructor, then if we decalre @Autowired on more than 1 parametrized constructor, it will throw error

case 4:

in the class with @Component if we have 1 parameterized constructor and 1 default constructor(constructor without arguments) then default constructor gets called if the parameterized constructor does not have @Autowired otherwise the parameterized constructor gets called where @Autowired is present.

CASE 1

```
@Component("myemp")
public class Employee {
```

```
    private int empId;
    private String ename;
    private String eaddress;
```

```
    private Department department;
```

```
    public Department getDepartment() {
        return department;
    }
```

```
    @Autowired
    public Employee(Department department) {
        System.out.println("1 paramtere constructor..");
        this.department = department;
    }
}
```

suppose if we have exactly 1 parameterized constructor, this gets called even if we have @Autowired or do'nt have @Autowired on the constructor

this will be called first

```
@Component("myemp")
public class Employee {
```

CASE 2

```
    private int empId;
    private String ename;
    private String eaddress;
```

```
    private Department department;
```

```
    private Department dept2;
```

```
    public Employee(Department department) {
        System.out.println("1 paramtere constructor..");
        this.department = department;
    }
```

```
    public Employee(Department department, Department dept2) {
        System.out.println("2 paramter constructor");
        this.department = department;
        this.dept2 = dept2;
    }
}
```

in the class with @Component if we have more than 1 parametrized constructor, then one of the constructor should be @Autowired otherwise it will throw error.

here no @autowired at any of constructor.

```
@Component("myemp")
public class Employee {
```

CASE 3

```
    private int empId;
    private String ename;
    private String eaddress;
```

```
    private Department department;
```

```
    private Department dept2;
```

```
    @Autowired
    public Employee(Department department) {
        System.out.println("1 paramtere constructor..");
        this.department = department;
    }
```

```
    @Autowired
    public Employee(Department department, Department dept2) {
        System.out.println("2 paramter constructor");
        this.department = department;
        this.dept2 = dept2;
    }
}
```

in the class with @Component if we have more than 1 parametrized constructor, then if we decalre @Autowired on more than 1 parametrized constructor, it will throw error

```

public class Employee {
    private int empId;
    private String ename;
    private String eaddress;

    private Department department;
    private Department dept2;

    public Employee() {
        System.out.println("employee zero cons..");
    }

    public Employee(Department department) {
        System.out.println("1 paramtere constructor..");
        this.department = department;
    }

    public Employee(Department department, Department dept2) {
        System.out.println("2 paramter constructor");
        this.department = department;
        this.dept2 = dept2;
    }
}

```

in the class with **@Component** if we have 1 parmeterized constructor and 1 default constructor(constructor without arguments) then default constructor gets called if the parmeterized constructor does not have **@Autowired** otherwise the parameterized constructor gets called where **@Autowired** is present.

Spring boot

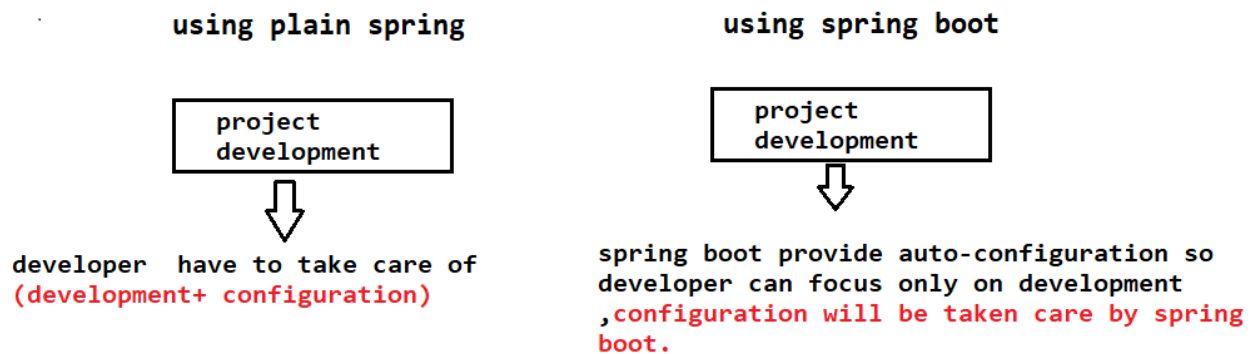
- >Spring boot is an open source java-based framework developed by Pivotal Team.
- >spring boot makes developer life easy by auto-configuring everything and helps to create production grade spring based application.
- >spring boot simplifies the bootstrapping and development.
- >spring boot was designed on the top of existed spring framework only it internally uses all spring modules .
- >spring boot can used to develop spring based application either by JAVA/kotlin or by Groovy.
- >it improves productivity and reduces development time.
- >Spring boot follows “Opinionated default configuration” approach to reduce developer efforts ,that is Spring boot avoids writing lot of boilerplate code,annotations and configuration.
- >Spring boot provides embedded HTTP servers like Tomcat,Jetty etc.
- >spring boot also provide build tool like Maven , Gradle to develop application.

->it also provide In-Memory database also ex:- H2

->it provides YAML support also for properties.

Note:- if we are using plain spring then developer need to take care of everything i.e business logic + lots of configuration.

Note:- if we are using spring boot then developer can focus on its business logic and rest all configuration will be taken care by spring boot.



->spring boot is famous for auto-configuration feature which is one of the main feature of spring boot.

->based on starter POMs it will identify what configuration required.

Ex:- if we define web-starter ,spring boot will provide web related feature like give support of server.

Ex:-if we define datajpa-starter ,spring boot will provide persistence layer features.

Ex:-if we define security-starter,spring boot will secure our application

..etc.

Note:- in spring boot we no need to download Tomcat server externally separate like we do in advance java .here spring boot provide embedded server.

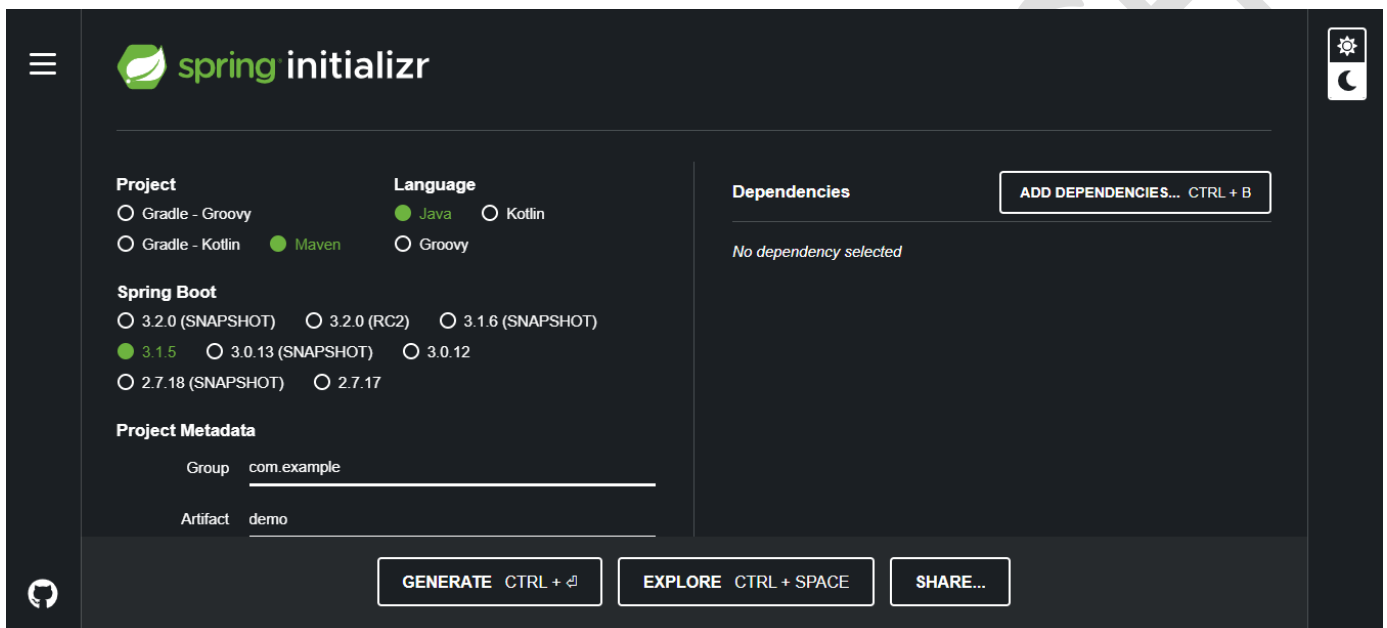
->How to create spring boot project?

We can create spring boot project in two ways.

- spring initlizer website(start.spring.io)
- using IDE like STS(spring tool suit) recommended

spring initlizer website

->you need to go to browser and search <https://start.spring.io/> you will get



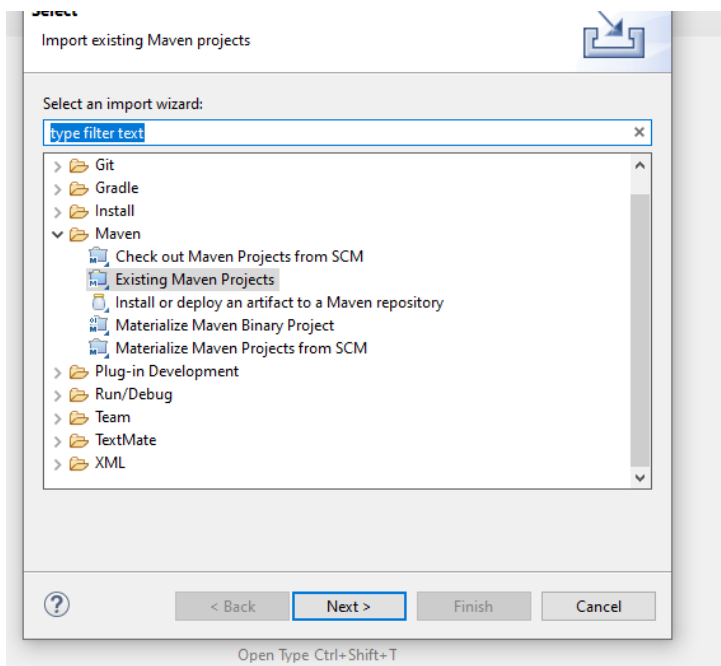
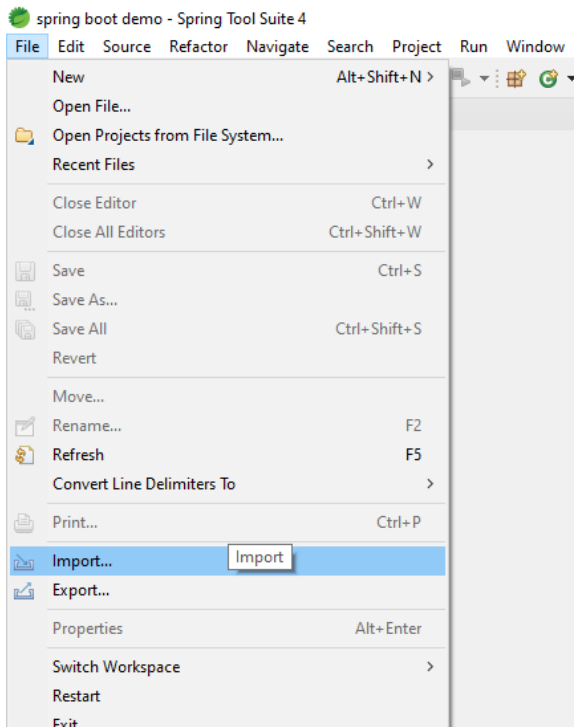
The screenshot shows the Spring Initializr web application. It has a dark theme with a sidebar on the left containing a hamburger menu and a GitHub icon. The main content area is divided into several sections: 'Project' with radio buttons for 'Gradle - Groovy' and 'Gradle - Kotlin', and 'Language' with radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'. Below this is the 'Spring Boot' section with radio buttons for versions: '3.2.0 (SNAPSHOT)', '3.2.0 (RC2)', '3.1.6 (SNAPSHOT)', '3.1.5' (selected), '3.0.13 (SNAPSHOT)', '3.0.12', '2.7.18 (SNAPSHOT)', and '2.7.17'. The 'Project Metadata' section has input fields for 'Group' (containing 'com.example') and 'Artifact' (containing 'demo'). On the right, there is a 'Dependencies' section with a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'. At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'. A settings icon and a moon icon are visible in the top right corner.

->now just provide necessary information for your project like group,artefact id,package name ..etc we already know about this in spring core.

->add required dependency if needed.

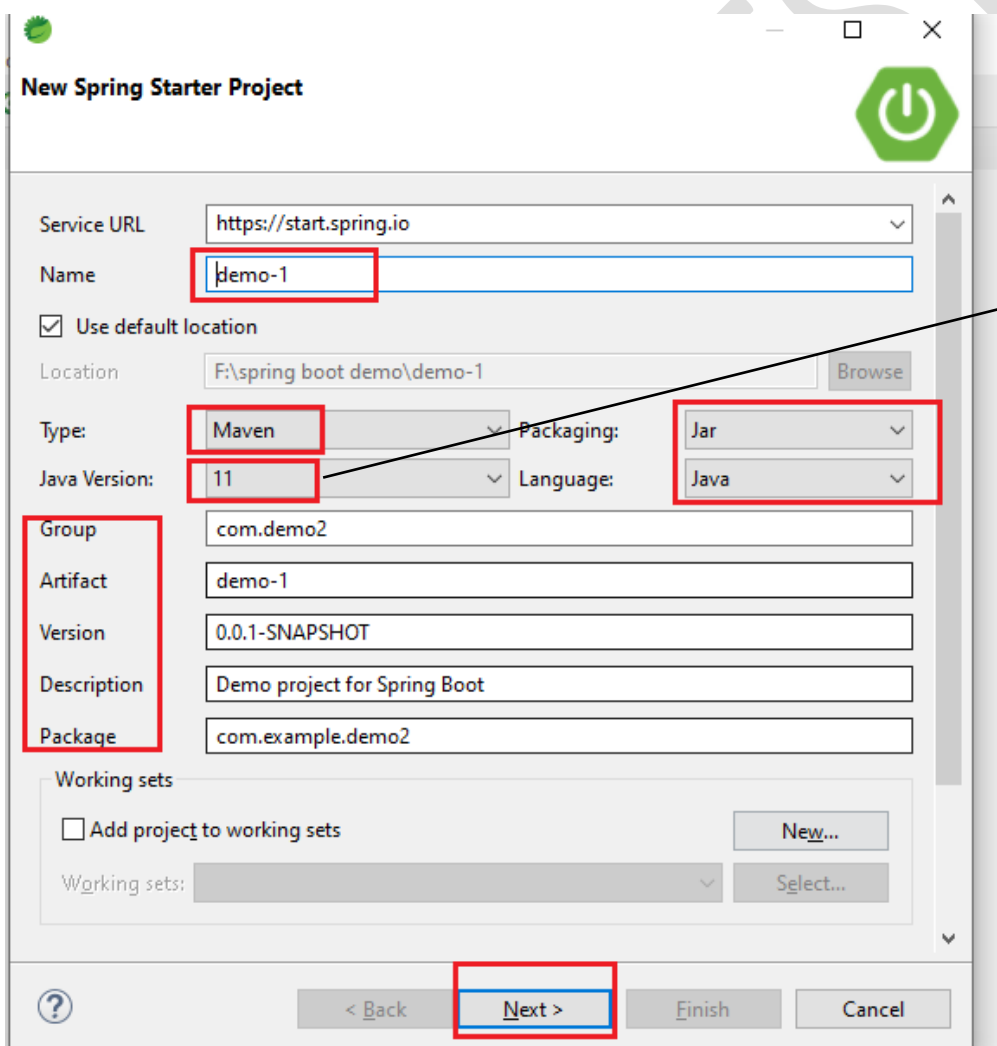
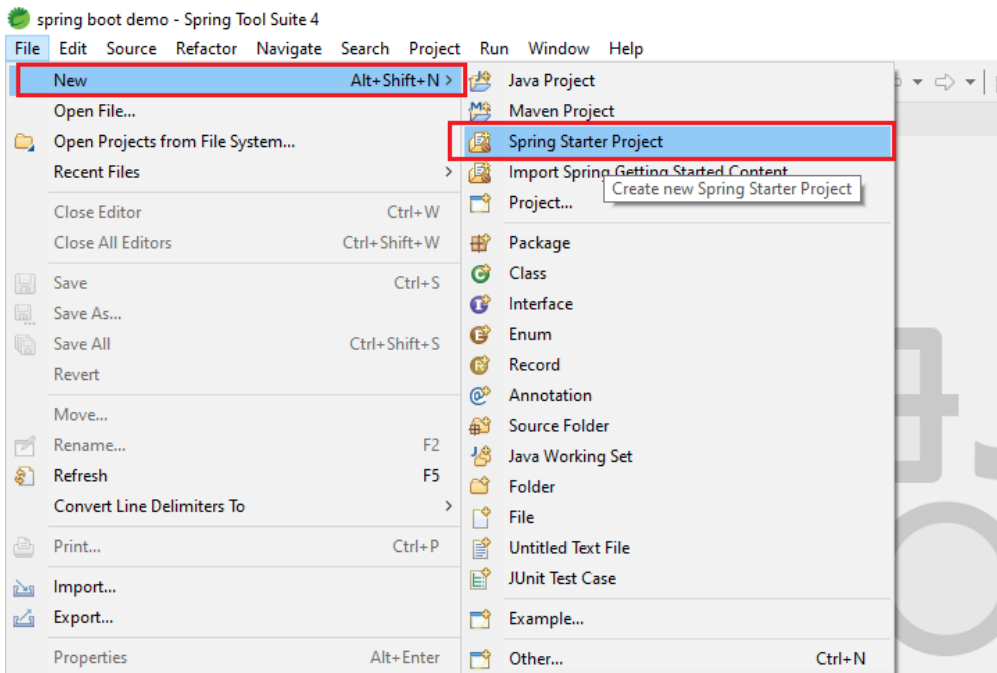
->click on generate button and your project get downloaded

->now you just need to open any IDE in our case (STS) and import it by clicking on existing maven project .



By spring spring-tool-suite (STS)

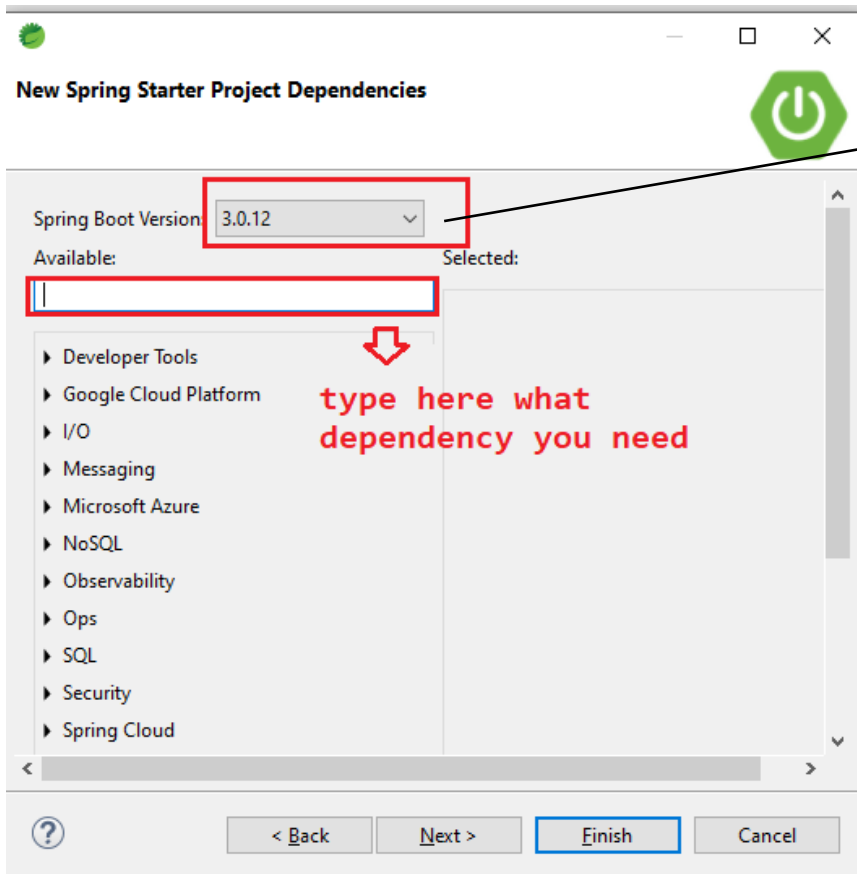
- >click on new then select spring starter project.
- >provide all the information of project like name,artifact id,group,version,etc
- >select dependency which required .
- >at last finish.



Note:- you can choose any java version above 8 or 8 itself. In spring boot this drop down show different- different java version time to time. May be after few days it show version like 17,21..etc .. so you can select any java version and you can also change java version from POM.xml file .

Note:- for above 3.0.x version use minimum JDK 17

Note:- try to understand spring and spring boot concepts .version may change time to time but concepts remain same .concepts always remain same . focus on concepts and use any version.

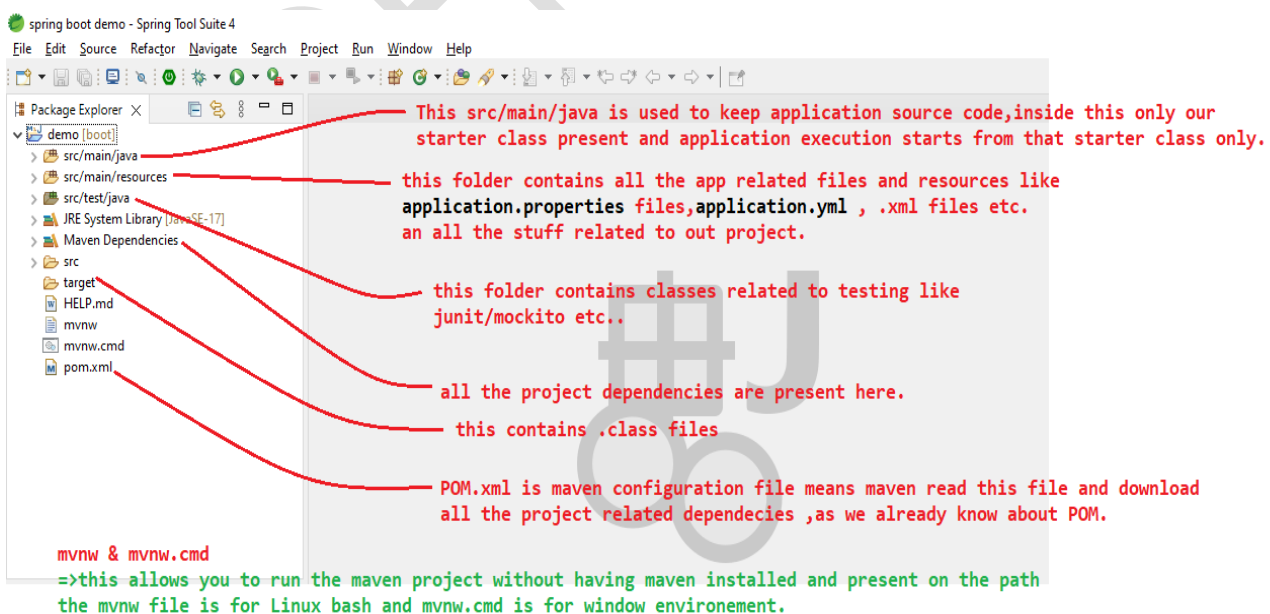


Note:- spring boot version change time to time . try to understand concepts of spring boot . version may change on time but concepts always remains same. You can use any spring boot version , spring boot automatic handles version conflicts so focus on concepts .

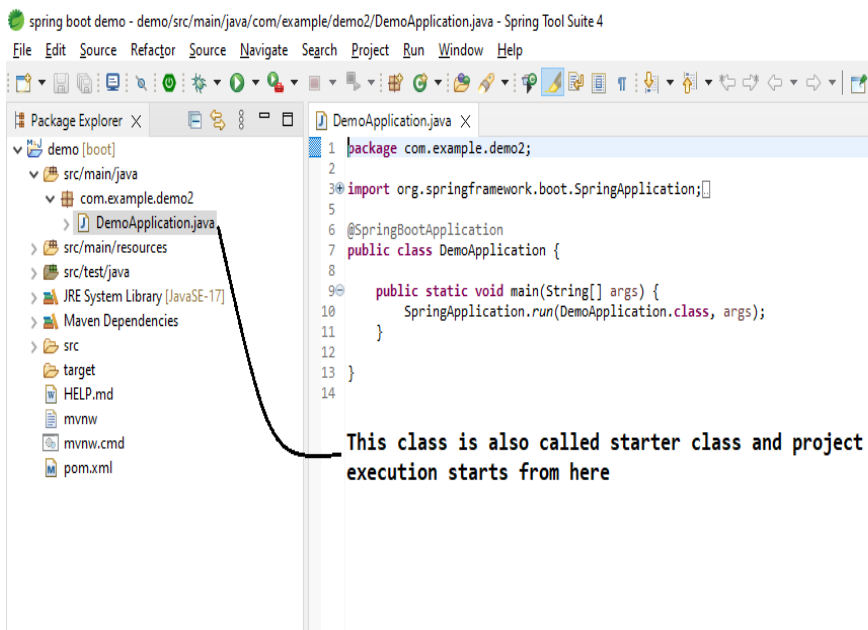
->click on finish done.

Note:-STS Ide will internally connect with 'start.spring.io' website only to create boot application.

Folder structure spring boot



->when we create spring boot project then by default we get one predefined class called **ProjectNameApplication.java** inside src/main/java /package name..



```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

->**@SpringBootApplication** annotation is equals to 3 annotations

@SpringBootApplication = **@SpringBootConfiguration** + **@EnableAutoConfiguration** + **@ComponentScan**

->**@SpringBootConfiguration** annotation represent start class as configuration class.

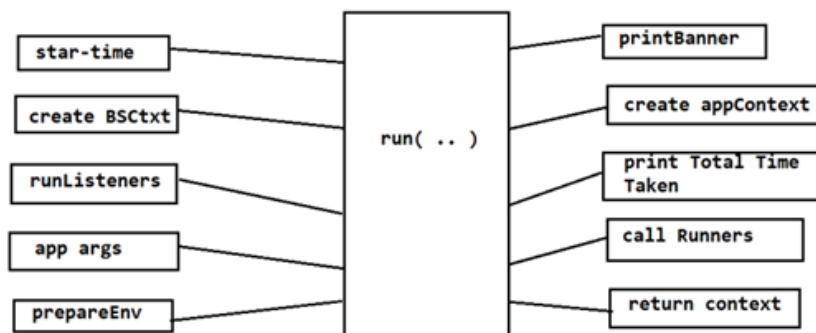
->**@EnableAutoConfiguration** annotation will enable auto-configuration features in spring boot.

->**@ComponentScan** annotation is used to identify spring beans available in the application.

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {

        SpringApplication.run(DemoApplication.class, args);
    }
}
```



This `run()` method performs various tasks related to spring boot application

Note:- as we know `@SpringBootApplication` annotation contains `@ComponentScan` also which means IOC will scan and create bean/object for all the component classes.

Note:- IOC will first search bean in base package and then sub-packages. If bean is available in these 2 places then IOC will create bean. If bean is present in any other separate package then IOC will not scan those beans until and unless we not specify.

base package contains starter class

com.codeminestech	⇒ base package (component scanning will start from here.)
com.codeminestech.services	⇒ This package will be scanned because it is sub-package
com.codeminestech.dao	⇒ This package will be scanned because it is sub-package
com.codeminestech.config	⇒ This package will be scanned because it is sub-package
com.util.helper	⇒ this package will not be scanned because this is not sub-package or base package

->but if we want that spring will scan different package also then we need to override @ComponentScan() with basepackage parameter ,so that our IOC can now able to scan all the packages specify there.

```
@SpringBootApplication
@ComponentScan(basePackages = {"com.codeminestech", "com.util"})
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

specify all the package including base package in which starter class present so IOC will scan all those package which is specify here.

->now lets move ahead. We already understand lots of annotation in spring core let see it again with some new annotation.

@Component:-as we already know to represent any java class as spring bean we will use @Component annotations.

@Service:-to represent java class as spring bean which contains some business logic we use @Service. This represent class as bean only but main difference is that we write business logic in that class .any class where we write business logic is nothing but service only right so we use this annotation.

@Repository:-to represent java class as spring bean which contains persistence logic (related to DB operation) we will use @Repository annotation. This @Repository provide beans but in addition internally it have some extra feature due to which it able to handle DB related exceptions. Thats the reason we use @Repository for DAO classes .

@Controller:-it represent a class as a Controller class,in web application to handle incoming requests we will use @Controller annotation over our controller class. Ex:- used in MVC

@RestController:- To develop distributed application we need to develop Rest Controller (api and all) in that scenario we use @RestController we will see it later..

We have also learnt about DI and autowiring in spring core section ..

@Qualifier: if there is more than 1 bean with **same class type**, using @Qualifier we can match the bean id to be injected .@Qualifier is placed along with @Autowired

@Primary: If we can declare @Primary in any of the @Bean/@Component, then by default the bean with @Primary gets injected if there is a conflict in autowiring of the beans with same class type provided there is no @Qualifier matching present

->we already learnt about these annotation in spring core which will be used in spring boot also..

Spring banner

->when we run spring boot application we can see that spring logo is printing on the console that spring logo is nothing but banner.

->we can also customize spring boot banner by creating **banner.txt** file under **src/main/resources**

->if you want banner in same style then you can generate **ASCII text** and keep it in **banner.txt file**.

->we have 3 modes for spring boot banner

- off
- log
- console

off :- means banner will not be printed

log:- means banner will be in log file (if we use logging)

console:- banner will print on console (by default mode)

Now to configure banner modes we need to define it on **application.properties** file

Ex:- spring.main.banner-mode=off

Runner in spring boot

->runner are used to execute any logic only one time once boot application got started.means if you want to execute to perform any task which should run only one time when execution starts then you go for runner.

->In spring boot we have 2 types of Runners

- ApplicationRunner
- CommandLineRunner

Note:- These ApplicationRunner and CommandLineRunner are Functional Interface (functional interface is an interface which contains only one abstract method)

->to use runner just you need to implements these runner Interface to you classes where you need to implement .

->and write the logic what you want

```

@Component
public class Employee implements ApplicationRunner{

    @Override
    public void run(ApplicationArguments args) throws Exception {
        System.out.println("inside application runner run method");
    }

    public Employee() {
        System.out.println("employee class constructor..");
    }

}

```

```

@Component
public class Employee implements CommandLineRunner{

    @Override
    public void run(String... args) throws Exception {
        System.out.println("inside commandline run method");
    }

    public Employee() {
        System.out.println("employee class constructor..");
    }

}

```

Difference between both

->ApplicationRunner **run()** method taking ApplicationArgument object as a parameter .

->CommandLineRunner **run()** method taking String[] as a parameter.

Use case:-

->we can write cache logic on runner run() method means load data to cache only once when application started.

->send any email or notification once our application get started.

->to store some common data like admin role and all when application get started etc.

Spring Data JPA

->Data JPA is used to develop persistence layer in our application

->persistence layer used to communicate with database.

In Java Frameworks we already have some technologies to make a persistence layer like **JDBC Api** , **Spring JDBC** , **hibernate framework** , **spring Orm**.

->if we talk about JDBC developer need to take care about everything like developer need to **create connection, prepare statement for queries ,fire queries using inbuilt methods**, then at last close connection.

->spring JDBC built on top of jdbc only and it reduces lot of developer efforts by taking care of db connections and db exceptions.. we just need to use it.

->above both api deals with data in **text format**

->To represent our application data in form of object we will go for ORM framework like hibernate.

->by using hibernate we can develop persistence logic.

->In hibernate developer needs to take care of session-factory , session, transaction..etc.

->developer need to write lot of boilerplate code ,developer need to create mapping file then configuration file etc for each entity.

->to overcome this Spring ORM came into picture by providing abstraction layer for hibernate. Spring Orm provide predefined Classes to execute methods directly but still there is some limitation. predefined methods redundant issue may come.

->To overcome this problem Spring Data Jpa came into picture.

->spring Data Jpa providing ready-made methods to perform CRUD operations . Which means we no need to write methods for CRUD operations because Data Jpa provide for us.

->spring Data JPA provides ready made methods to perform CRUD operations. Means developer no need to write methods to perform CRUD operation it is taken care by Data Jpa.

->Data Jpa is used to developed persistence layer in our applications.

->by using Data Jpa it is very easy and clear to implement persistence layer.

->Data Jpa is auto configured and self logic implemented for CRUD operation.

->Data Jpa provides “**Embedded Database Support**” . means database provided in application itself. Ex like h2.

->we can use this embedded database for temporary purpose.

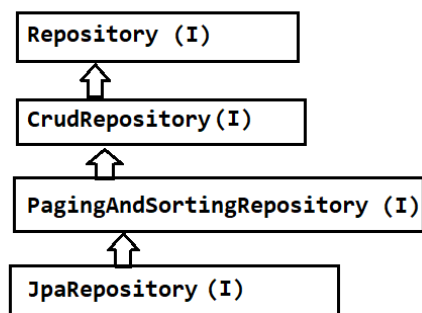
->These embedded database are used in both development and testing purpose but not for production.

->spring boot also supports both sql and No-sql database also.

->Data Jpa supports Easy Connection Pooling concept(auto-config).

->Data Jpa Supports Cache Management (Auto Config).

->Data Jpa has provided Repository Interfaces in package “**org.springframework.data.repository**”.



Steps to Create first program using Data JPA

Step1. Create database in your Mysql database (in ourcase we are using Mysql).

Step2:-create spring boot application with below dependencies.

- Datajpa-starter
- Jdbc driver (in my case mysql if you are using Oracle then use that)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

mysql connector may be vary depends on versions..

Step3:-create Entity class with following annotations.

example

```
@Entity
@Table(name = "USERS")
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long uId;

    private String userName;
    private String password;

    @Column(name = "USER_FIRST")
    private String firstName;
    private String lastName;
    private String email;
    private String phone;
    //getters
    //setters
}
```

defined annotation based on your requirement

There are lots of annotations related to entity class provided by JPA define according to your requirement.

@Entity:- it is a class level annotations, it can be used to declare a class as an Entity class.

@Table(name="tablename"):- it is a class level annotation, it is used to configure table name for the entity class. Where "name" attribute is used to provide table name. And also used to map our class with db table name.

@Id:- it is a field level annotation. it is used to declare a property as a ID. And it represent the field mapped with primary column in table.

@column(name="columnname"):- it is used to provide database table column name. what ever name you provide it will create column with that name and map with it.

Note:- if in database table name and our entity class name is same then @Table is optional.

Note:- if database column name and entity class field name is same then @Column is optional.

Step4:- create Repository interface for Entity class.

Data Jpa provide 2 Interface to create Repository

- CrudRepository
- JpaRepository

CRUD Repository

-> first create one interface and extends this repository .

```
public interface UserRepo extends CrudRepository<User, Integer>{  
    }  
    entity name      primary column type  
    ↑                ↑
```

We can able to use all method specified in Crud Repository.

Step5:configure datasource in application.properties file.

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdemo
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.platform=mysql

spring.jpa.hibernate.ddl-auto=create
```

Step6:- call the repository method and test it by save data in db as off now we are calling it from main class in future we will call it from service.

```
@SpringBootApplication
public class project1Application {

    public static void main(String[] args) {
        ConfigurableApplicationContext context = SpringApplication.run(project1Application.class, args);

        UserRepo userrepo = context.getBean(UserRepo.class);

        User user=new User();
        user.setFirstname("codemines");
        user.setLastname("jbp");
        user.setUserId(101);
        user.setUsername("cmdtech");

        User saved = userrepo.save(user);
    }
}
```

this save method is from crud repository we are just calling it to save our user data

Save(obj):- this save method is from crudRepository(I) which is used to perform save or update operation.

=>if primary key value is Null or not exist in DB then perform insert operation,else if record is already available then update operation.

Note:-It is optional to write @Repository annotation for our repository interface because when we have defined data-jpa starter in POM.xml file then it will scan our project and it will identify JPA repository and it will provide implementation for those repository. Save() is also called upsert() method.

So far we have done:-

->we developed one application using data JPA.

->we configure data-jpa starter in POM.xml.

->we have created one Entity class and used following annotations like @Table,@Column,@Id,@Entity.

->we have created one interface and extends it with CrudRepository to make Repo Layer.

Note:-for CrudRepository data Jpa will implement our interface using proxy design pattern.

Let see following methods by Crudrepository.

Save(T obj):-to insert / update one record into table.

saveAll(Iterable obj):-to insert/update multiple record into table.

findById(id):-to retrieve record using primary key.

findAllById(Iterable<ID> ids):-To retrieve records using multiple primary keys.

findAll():- To retrieve all records from the table.

existsById(id):- To check presence of record in the table.

Count():-To check total records count in the table.

deleteById(id):-To delete record based on primary key.

deleteAllById(Iterable id):-To delete records based on multiple primary key.

delete(obj):-to delete record based on entity object.

deleteAll() :-To delete All records from table.

=> In data jpa we are calling methods to perform CRUD operations. To print queries generated by Data JPA on the console we can write below property in application.properties file

```
spring.jpa.show-sql=true
```

=> To generate schema dynamically we can write below property in application.properties file (tables will be created dynamically in database)

```
spring.jpa.hibernate.ddl-auto=update
```

Note:- spring.jpa.hibernate.ddl-auto=create

->from above table will create automatically and every time new table will create. if you set it update then it will update to existing table if table already present otherwise create and update it always.

```
UserRepo userrepo = context.getBean(UserRepo.class);
```

```
User user=new User();  
user.setFirstname("codemines");  
user.setLastname("jbp");  
user.setUserId(101);  
user.setUsername("cmdtech");
```

```
User saved = userrepo.save(user);
```

```
Iterable<User> itr = userrepo.findAll();  
itr.forEach(d->{  
    System.out.println(d.getFirstname()+" "+d.getLastname()+" "+d.getUsername());  
});
```

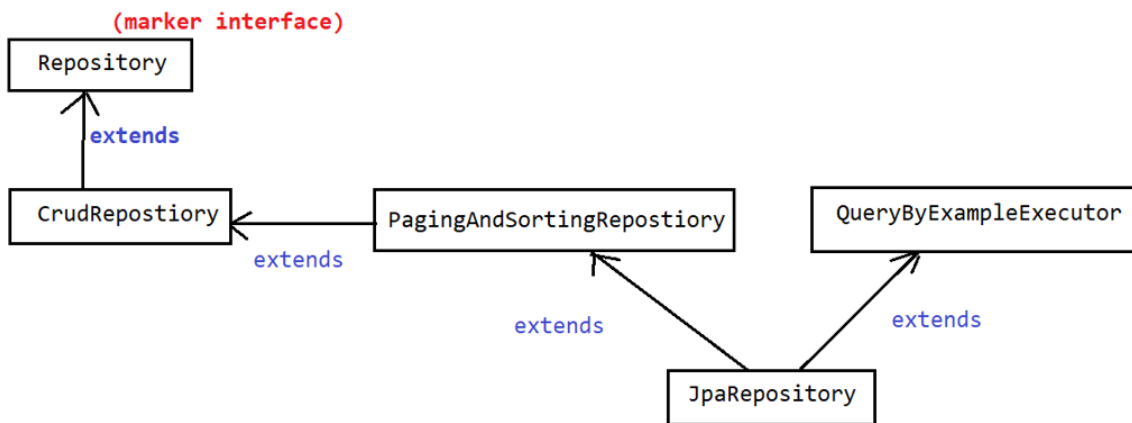
```
Optional<User> opt = userrepo.findById(101);  
if(opt.isPresent())  
{  
    User user1 = opt.get();  
    System.out.println(user1.getUsername()+" "+user1.getFirstname()+" "+user1.getLastname());  
}
```

```
userrepo.delete(user);
```

->above diagram shows example of few methods we have used to perform crud operation.

->we can create repository using by CrudRepository or by JpaRepository

->JpaRepository having additional features like **Pagination** , **Sorting** and **QueryByExample** (QBE).



we recommend to use JpaRepository because it contains all the features of CrudRepository as well as some additional features also.

-> JpaRepository also used to perform sorting.

It is used to retrieve records based on sorting order.

Ascending order

Descending order

```

public static void main(String[] args) {
    ConfigurableApplicationContext context = SpringApplication.run(project1Application.class,
args);

    UserRepo userrepo = context.getBean(UserRepo.class);

    List<User> users = userrepo.findAll(Sort.by("firstname").ascending());

    for(User u:users)
    {
        System.out.println(u.getFirstname());
    }

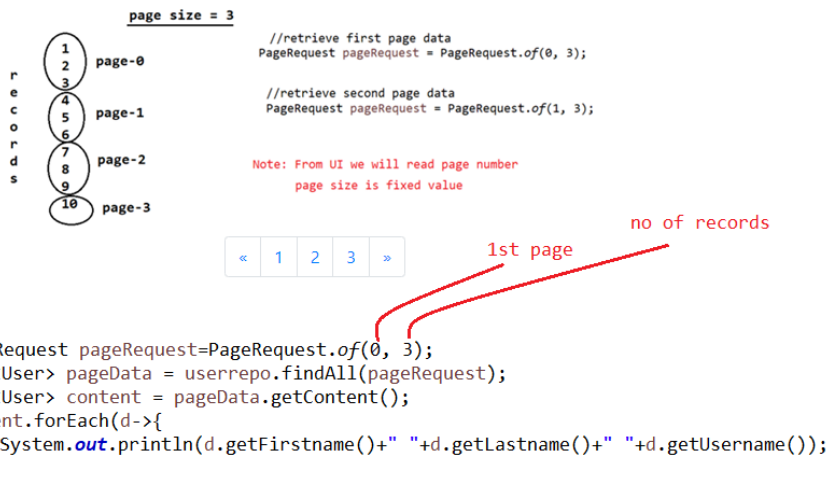
    System.out.println("-----desc-----");
    List<User> users1 = userrepo.findAll(Sort.by("firstname").descending());

    for(User u:users1)
    {
        System.out.println(u.getFirstname());
    }

}
}

```

Jpa is used for Pagination



Query by Example (QBE)

->it is used to retrieve records based on conditions just for filter kind of thing.

```
@Entity
public class Products {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Integer pid;
    private String prodName;
    private String prodTitle;
    private String prodDesc;
    private Double price;
    //setter
    //getter
}
```

```
Products prod=new Products();
prod.setProdName("samsung");

Example<Products> ex=Example.of(prod);
List<Products> all = repo.findAll(ex);

System.out.println(all);

all.forEach(d->{
    System.out.println(d);
    System.out.println(d.getPid()+" "+d.getProdName()+" "+d.getProdTitle());
});
}
```

->as off now we have seen CrudRepository as well as JpaRepository and few methods like save(),findAll(),findById(),delete(),deleteAll(),deleteById() etc.

->**Note:-** take entity id in Wrapper class format like (Integer,Long..etc) to get proper result in Query by example

->now these methods are ok to deal with normal conditions but in real time different-different scenario comes to deal with that data-jpa provides facility to create custom method .

->we can create custom methods like findByXXx(..).

->Spring Data internally generates a query based on method written in Repository by developer.

findByXXX () methods

-> findByXXX () methods are used to construct the query based on method name in the repository

-> We need to write findByXXX () method using entity class property name (variable)

Note: Method name is very important here because based on method name only jpa will construct the query and it will execute that query.

In simple word we need to use **findBy.....**

examples

findBy____(parameter);

you can write according to your need.

```
findByName(String name)
findByAgeGreaterThan(int age)
findByEmailContaining(String email)
findBySalaryGreaterThanEqual(double salary)
findByLastNameOrderByFirstNameAsc(String lastName)
findByCityAndIsActive(String city, boolean isActive)
findByNameIn(List<String> names)
findByNameLikeAndAgeLessThan(String name, int age)
findByLastNameStartingWith(String prefix)
findByAddressNotNull()
findByEmailEndingWithIgnoreCase(String suffix)
findByNameStartingWithIgnoreCase(String prefix)
findByNameNot(String name)
findByLastNameLikeAndAgeGreaterThanEqual(String pattern, int minAge)
```

```
findByNameLike(String pattern)
findByEmailLikeIgnoreCase(String pattern)
findByLastNameNotLike(String pattern)
```

```
ex:-findBy**Name**Like("codemines%");
```

```

List<User> user = userrepo.findByAmountGreaterThan(200);
user.forEach(data->{
    System.out.println(data.getFirstname()+" "+data.getLastname()+" "+data.getAmount());
});

System.out.println("based on lastname");
List<User> users = userrepo.findByLastname("kumar");
users.forEach(data->{
    System.out.println(data.getFirstname()+" "+data.getLastname()+" "+data.getAmount());
});

System.out.println("based on first name and lastname");
List<User> users2 = userrepo.findByFirstnameAndLastname("tom", "kumar");
users2.forEach(data->{
    System.out.println(data.getFirstname()+" "+data.getLastname()+" "+data.getAmount());
});
}

```

repository layer

```

@Repository
public interface UserRepo extends JpaRepository<User,Integer>{

    public List<User> findByLastname(String lastname);

    public List<User> findByFirstnameAndLastname(String firstname, String lastname);

    public List<User> findByAmountGreaterThan(int amount);

    List<User> findByAmountIsNull();

}

```

->as off now we have seen about custome method provided by Data-Jpa.

Now let's see Custom Queries in Data JPA

-> Custom Query means we can write the query and we can ask data jpa to execute our query.

-> We can write custom queries in 2 ways

- Native SQL (plain sql queries)
- HQL (Hibernate Query Language)

Native Sql queries

->In native style Sql queries will use table names and columns directly in the query.

->native Sql queries are database dependent .

->database dependent means one DB sql syntax may not supported by other DB .

->Native sql queries is not recommended because if we change our database in future then we need to change our sql querries syntax

according to that database. As well as we need to again test everything it will consume lot of time and efforts .

->To Overcome this Native Sql problem ,hibernate introduced its own query language that is called **HQL**.

->**HQL** stands for hibernate query language.

->**HQL** queries are database in-dependent queries.

->internally HQL converted to Sql only using dialect .

->for every database hibernate have dialect classes.

->**HQL** we will prepare queries by using **POJO/entity class** name and their **properties**.

->HQL queries syntax looks similar to SQL only but instead of table name and column name we use entity class name and property(variable) name.

SQL : `select * from user`

HQL : `from user`

SQL : `select * from user where userid=101`

HQL : `from user where userId=105`

SQL : `select * from user where firstname="john" and lastname="doe"`

HQL : `from user where firstName="john" and lastName="doe"`

...etc

SQL

`table name`
`column name`

HQL

`entity/pojo name`
`property(variable) name`

To execute custom queries we will use **@Query** annotation.

```

@Repository
public interface UserRepo extends JpaRepository<User,Integer>{

    @Query("from User")
    public List<User> myusers();

    @Query("from User where amount>200")
    public List<User> getUserwhoIsHavingAmountMoreThen200(int amount);
}

```

now you can give any name to method jpa will generate queries based on HQL provided in @Query()

```

@Query(value="select * from my_user", nativeQuery = true)
public User user();

```

if you want to use native queries then in @Query use value & nativeQuery=true



```

@Query("from User")
public List<User> myusers();

@Query("from User where amount>:amount")
public List<User> getUserwhoIsHavingAmountMoreThen200(int amount);

@Query(value="select * from my_user where user_id=?", nativeQuery = true)
public User user(int id);

@Query(value="select * from my_user where user_id=:id", nativeQuery = true)
public User user(int id);

@Query("from User where amount>?1")
public List<User> getUserwhoIsHavingAmountMoreThen200(int amount);

```

you can pass parameter like ?1, ?2, ?3

or

Named paramter like :id , :amount (but paramter name must be same as variable name)

Timestamping in Data Jpa

->When we insert or update a record it is very important to track when that record get inserted or when that record get updated. so to track this it is highly recommended to maintain columns in database related to insert and update records like

Created_date

Updated_date

->Created_date column will represent when that record got inserted into table.

->UPDATED_DATE column will represent when that record got updated in the table.

These column will help developer for analysis of insertion and updation in future.

Ex:- provide all customer details register on last month.

->In Hibernate we have TimeStamping concept to insert created_date & updated_date values.

@CreationTimeStamp

@UpdateTimestamp.

Ex:-

```

@Entity
public class Employee {

    @Id
    private int empid;

    private String ename;

    private String address;

    @Column(name="created_date" , updatable = false)
    @CreationTimestamp
    private LocalDateTime createdDate;

    @Column(name ="updated_date" , insertable = false)
    @UpdateTimestamp
    private LocalDateTime updatedDate;

    //setter
    //getter

```

updatable=false
means ignore this column when update query getting generated. ex:- ignore created_date column when update query fire.

insertable=false
means ignore this column when insert query getting generated . ex:- ignore updated_date column when insert query fire.

Generators

-> Generators are used to generate the value for primary key columns in the database.

->As we all already discussed in our database portion about primary key ,what is primary key,use of primary key and etc.

->as we already know **Primary key is Unique and Not null.**

->To represent primary key we use **@Id** annotation.

->in real time when we develop application we should not ask end user to enter value for primary key because end users may enter duplicate value which may create ambiguity in our records.

->So we will use Generators to generate primary key column values with in the application.

->we use **@GeneratedValue** annotation to apply generator for primary key value.

@Id

@GeneratedValue

@Column(name = "student_id")

-> When we insert record into table, sid value will be generated through **@GeneratedValue** annotation

Note: If we use oracle database then it will use hibernate_sequence by default to get primary key column value (select hibernate_sequence.nextval from dual)

Note: hibernate_sequence is the default sequence created by hibernate in oracle database.

Note: MySQL Database will not support for sequences. MySQL DB will use Auto Increment to generate the value for Primary key Columns

```
@Entity
public class Employee {

    @Id
    ➡ @GeneratedValue(strategy=GenerationType.AUTO)
    private int empid;

    private String ename;

    private String address;

    @Column(name="created_date" , updatable = false)
    @CreationTimestamp
    private LocalDateTime createdDate;

    @Column(name ="updated_date" , insertable = false)
    @UpdateTimestamp
    private LocalDateTime updatedDate;

    //getter
    //setter
}
```

Some more annotations related to JPA

@OneToOne:-used to define one-to-one mapping between two entity beans.

@OneToMany:-used to define one – to – many unidirectional entity mapping.

@ManyToMany:-it represent many –to-many relationship between two entites. It is typically used to map a collection of child entites to a collection of parent entites.

Transcation

->To understand transcation let understand this example

->we want to transfer money from source account to target account.

->To transfer money we need to perform two logic

1)we need to deduct money from source account

2)we need to credit /add money to target account.

->if both operation happen successfully means **transaction is succesfull.**

But if we transfer the amount and amount deducted from source account successfully but failed to credit the amount in target account then our **Transcation got Failed.**

->**we also discussed about Atomicity . that either All or None.**

Means if amount failed to credit to target account then that amount should come back source account this is called rollback.

->in Data JPA when we perform Non-Select Operation then Transaction will be committed by default.

->it is not automatically rollback.

->if we want to rollback the transcation then we should use **@Transactional** annotation.

Ex:-

main class

```
public static void main(String[] args) {  
    ConfigurableApplicationContext context = SpringApplication.run(Jwtproject1Application.class, args);  
    DepartmentService deptserv = context.getBean(DepartmentService.class);  
  
    Employee emp = new Employee();  
    emp.setEmpaddress("mohan");  
    emp.setEmpName("abhi");  
    deptserv.addEmployee(emp);  
  
} //
```

Department entity

```
@Entity  
public class Department {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.AUTO)  
    private int deptId;  
  
    private String deptName;  
  
    @OneToOne  
    private Employee employee;  
  
    //getter  
    //setter  
  
}
```

Employee entity

```
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.AUTO)  
    private int empId;  
    private String empName;  
    private String empaddress;  
  
    //setter  
    //getter  
}
```

Repos

department repo

```
@Repository
public interface DepartmentRepo extends JpaRepository<Department, Integer> {
}
```

Employee Repo

```
@Repository
public interface EmployeeRepo extends JpaRepository<Employee, Integer>{
}
```

```
@Service
public class DepartmentService {

    @Autowired
    private DepartmentRepo departmentRepo;
```

```
    @Autowired
    private EmployeeRepo employeeRepo;
```

```
    @Transactional
    public Employee addEmployee(Employee emp)
    {
        Employee employee = employeeRepo.save(emp);

        Department dept=null;
        dept.setDeptName("computer science");
        dept.setEmployee(emp);
        departmentRepo.save(dept);

        return employee;
    }
```

here @Transactional annotation is defined in this addEmployee() method. here our Transactional is successfull when both the object is saved otherwise transaction fail not even one table data will store which means either both or none.

Spring Web MVC

->Spring Web Mvc is one module available in Spring framework.

->in plain spring developer need to take care lot of boilerplate code but spring boot reduces lot of work because of “Auto-Configuration” functionality.

->using MVC module we can develop 2 types of applications

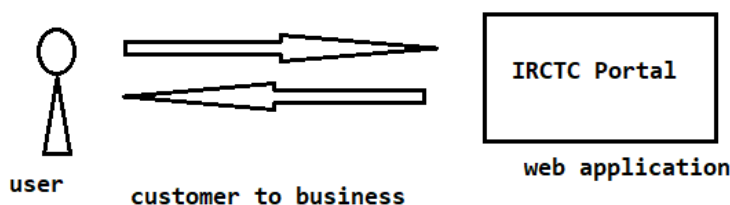
- Web Application
- Distributed Application

->The application which can be access from browser is called **web application**.

Ex:- facebook,twitter,IRCTC,MakeMyTrip..etc

->generally Web application are used for C2B business which means customer to business model.

Ex:- users < ----- > IRCTC

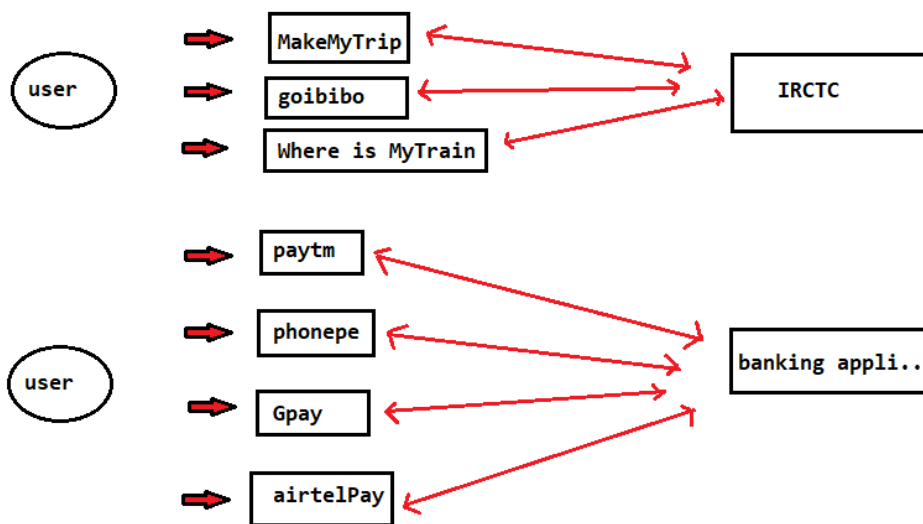


->The application which is communicating with other application are called **distributed application**.

->Distibuted Applications are used for Business To Business Communication. Which means one application will not directly contact with customer instead of it communicate with another application .

Ex:- makeMyTrip is communicating with IRCTC

Ex:- goibibo is communicating with any airline portal for flight booking



what is the need of distributed application ?

->just imagine if you want to develop weather forecast application or you need to start your own business which help full to book train ticket. If you do every thing from zero like to establish whole infrastructure as well as whole communication system then lot of budget will required. lot of money power,man power needed.

So this not smart way to make money , the smart way is the companies who already have big infrastructure as well as man-power and whole eco-system . just contact with them and pay amount to them so that your application can communicate with their application .so you are doing here B2B business.

Ex:- phonepe,googlepay,paytm are one application using which you can transfer money very easily so you are using these application but internally these application are communicating with banking application ,if suppose paytm will not communicate with banking application then paytm need to build it's own infrastructure and needed lot of man-power and moneypower .better option is to deal

with banks and give some percentage to them and use their application.

In simple you can say instead of re-developing everything from scratch we can reuse service of one –application in another application.

To make Distributed application we have 2 approaches

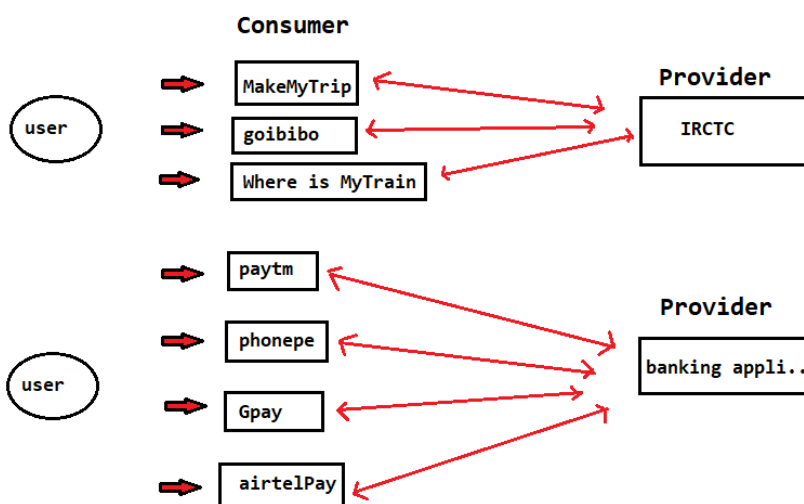
- SOAP based webservices (legacy very old almost dead now)
- Rest Full Webservices (trending and latest)

->when we are developing Distributed application then 2 terminology we mostly see.

- Provider
- Consumer

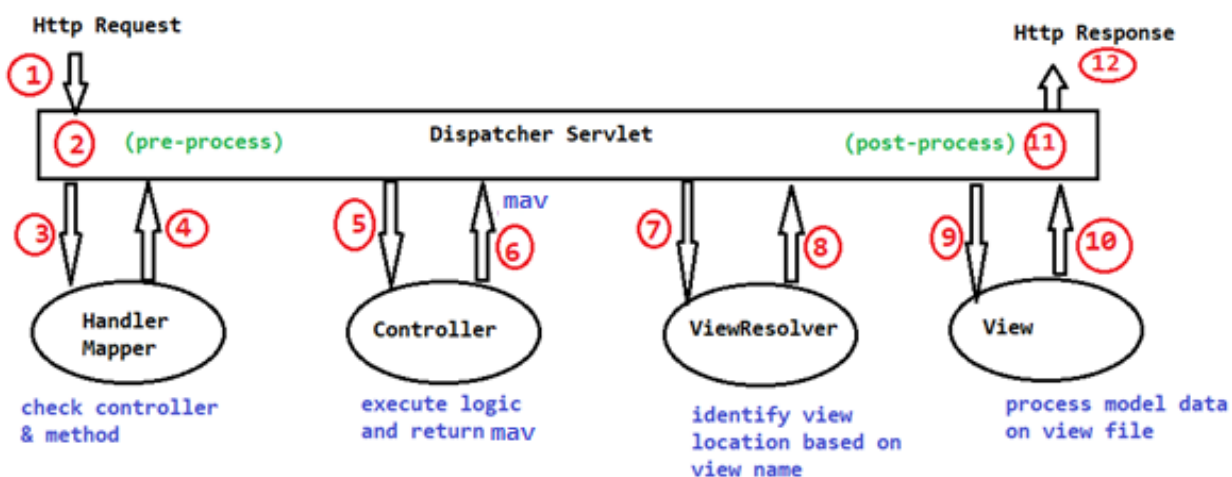
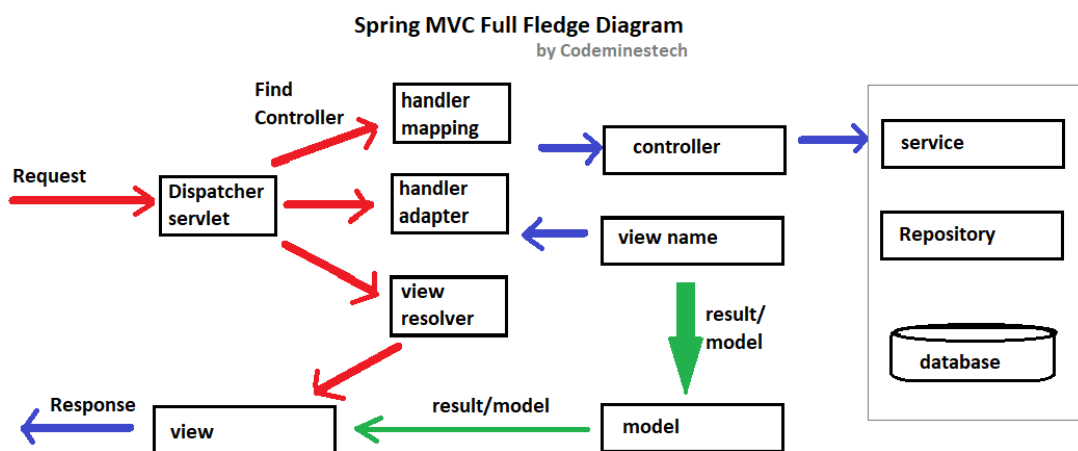
Provider:-The application which is providing business services to other applications is called as provider.

Consumer:-The application which is accessing business services from another application is called as Consumer.



->distributed application are language and platform independent which means application developed on different-different language using different –different platform can communicate each other these feature is also called “intereoperable”.

Spring Web MVC Architecture



Dispatcher Servlet(FrontController):-dispatcher servlet is a predefined servlet class which will act as a front controller. It is responsible to perform Pre-Processing and Post-Processing of a request.

Handler Mapper:-it is used to identify which request should be processed by which controller method in our application.

Controller:- it is used to handle a request .

ModelAndView:-modelandview is a predefined class and model holds data in key-value format and view refers logical view name.

->**ViewResolver** is used to identify physical location of view files available in the project.

->**view** is used to render Model Data in the view component.

->mvc working in springboot in simple words:-request first comes to the dispatcher servlet which is front-controller provided by spring boot. Dispatcher servlet pre-process this incoming request (by identifying headers, body and data of request) and after pre-processing it pass request to handler mapper. Now handlermapper role come into picture and it identify which controller method need to invoke .so after identify that method it passes that request again back to dispatcher servlet . and now dispatcher servlet execute that controller methods. Now controller method will return ModelAndView Object to dispatcher servlet. Now dispatcher servlet passes controller over ViewResolver and view resolver find the view file location in our project and finally all model data rendered in view and send response to client.

->Building First web application using spring web mvc

->To create web project spring boot provides starter i.e “**web-starter**” dependency.

->web-starter provides embedded-server(bydefault tomcat).

->Embedded servers are built-in servers which are used to run our application. Like in advance java here we no need to install apache tomcat separate and add to project.

->add “**tomcat-embed-jasper**’ dependency in POM.xml file ,

Note:- when we are using JSP as a view then only this jasper dependency needed otherwise this step is not required (because springboot not provide support of JSP so we need this).

```
<dependency>
```

```
<groupId>org.apache.tomcat.embed</groupId>
```

```
<artifactId>tomcat-embed-jasper</artifactId>
```

```
</dependency>
```

Note:- if we not add this jasper dependency and we are trying to using JSP file then if we send request to our controller it will download that file instead of displaying response.

->as we know in JSP we can write in normal JSP scriplet tag as well as JSTL also. So we need to add one more dependency if we want to use JSTL .

```
<dependency>
```

```
<groupId>javax.servlet</groupId>
```

```
<artifactId>jstl</artifactId>
```

```
</dependency>
```

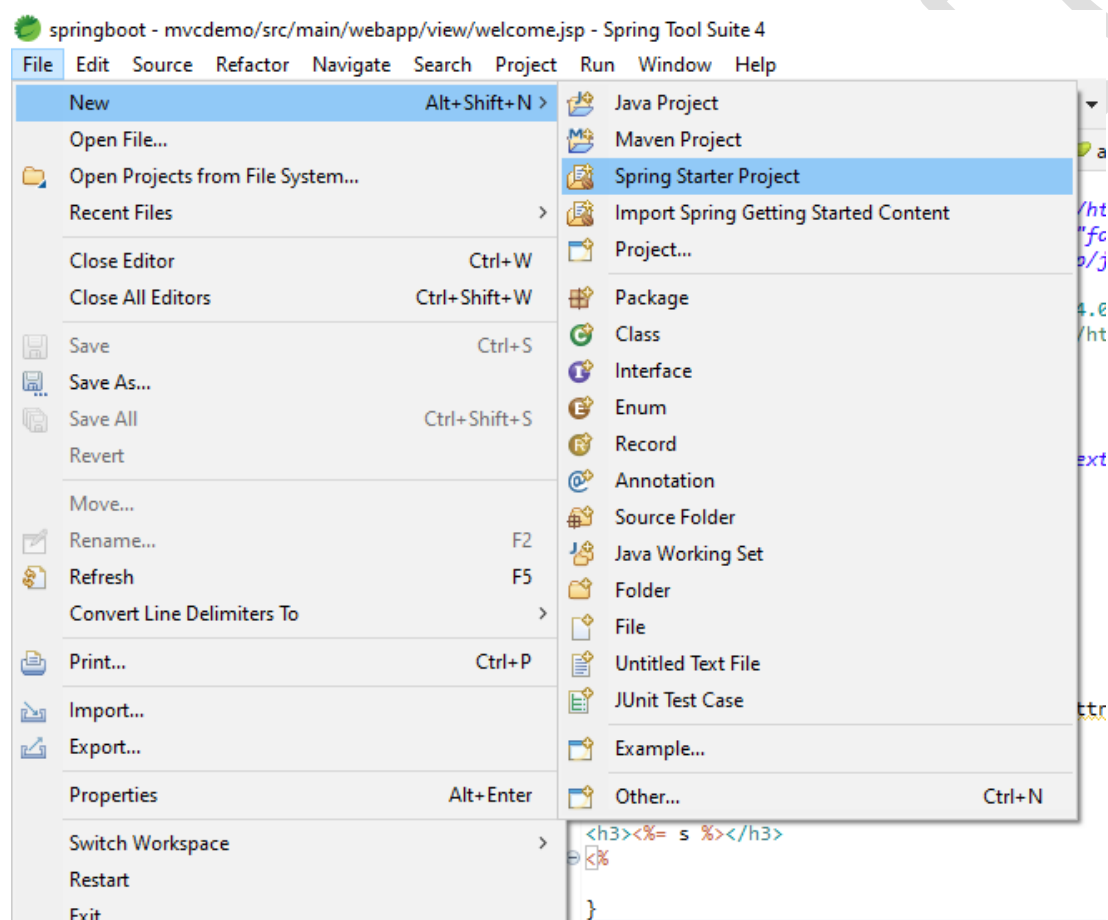
->select “**war**” instead of jar . if you select war then springboot will provide webapp folder. Otherwise you need to create it manually.

->we need to use @Controller annotation over our Controller class to make it controller.

-> Bind controller class method to HTTP GET Request using @GetMapping annotation.

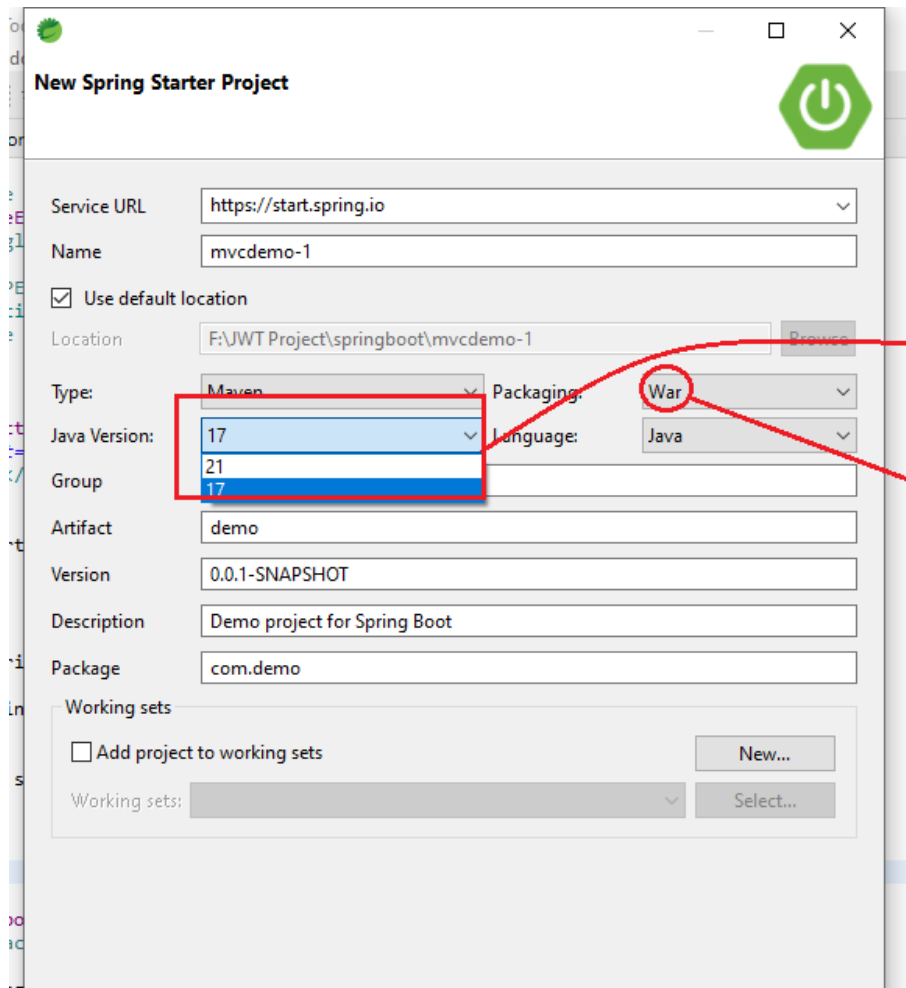
->create view file inside webapp location.

Step1:-



Step2

Select java version and war, and select starter-web.



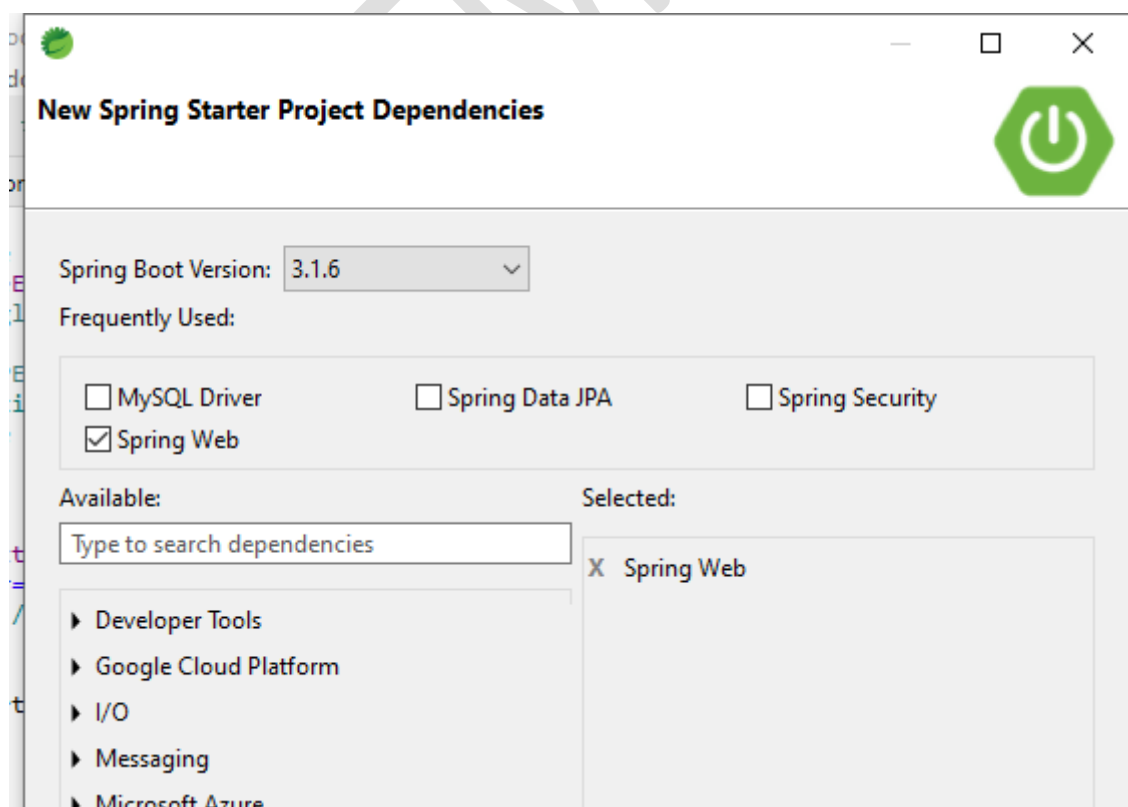
The 'New Spring Starter Project' dialog box is shown. It contains the following fields and options:

- Service URL: `https://start.spring.io`
- Name: `mvcdemo-1`
- ☒ Use default location
- Location: `F:\JWT Project\springboot\mvcdemo-1` (with a 'Browse...' button)
- Type: `Maven`
- Java Version: `17` (with a dropdown menu showing `17`, `21`, and `17`)
- Packaging: `War` (with a dropdown menu showing `War`)
- Language: `Java`
- Group: `com.demo`
- Artifact: `demo`
- Version: `0.0.1-SNAPSHOT`
- Description: `Demo project for Spring Boot`
- Package: `com.demo`
- Working sets: ☐ Add project to working sets (with 'New...' and 'Select...' buttons)

we are using java 11
,here in latest
release it is not
showing but we can
configure after
project creation
from POM file

for web mvc select
war project

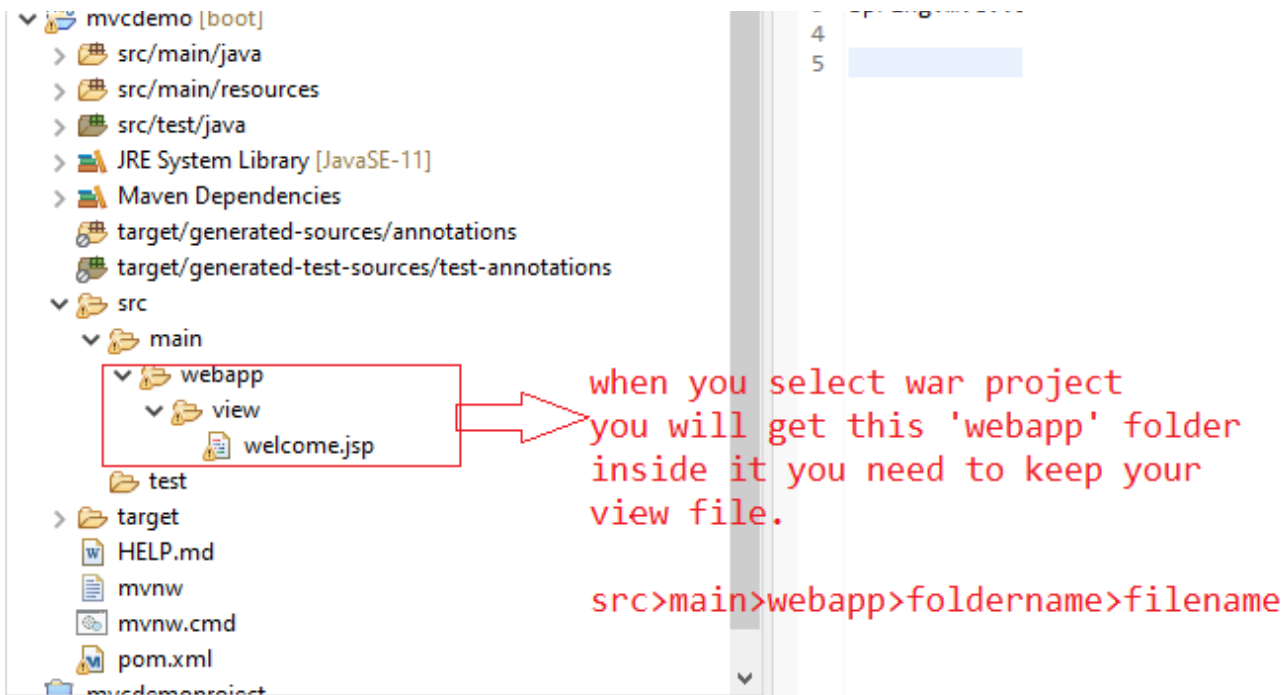
Step3



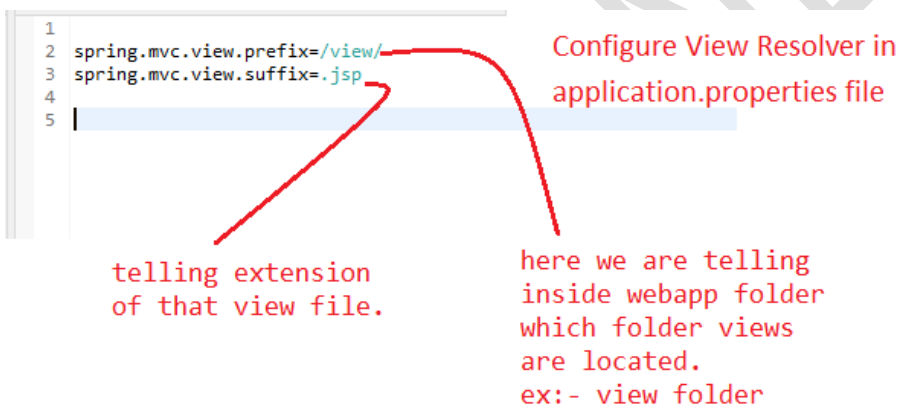
The 'New Spring Starter Project Dependencies' dialog box is shown. It contains the following fields and options:

- Spring Boot Version: `3.1.6`
- Frequently Used:
 - ☐ MySQL Driver
 - ☐ Spring Data JPA
 - ☐ Spring Security
 - ☒ Spring Web
- Available:
 - Type to search dependencies
 - Developer Tools
 - Google Cloud Platform
 - I/O
 - Messaging
 - Microsoft Azure
- Selected:
 - X Spring Web

Step4:-



Step5:- add these to application.properties file



Step6:-

```

@Controller
public class MyFirstController {

    @GetMapping("/welcome")
    public ModelAndView displayingWelcome()
    {

        String msg="Welcome to Codeminestech bro !!!";
        ModelAndView mav=new ModelAndView();
        mav.addObject("msg",msg);
        mav.setViewName("welcome");

        return mav;
    }

    @GetMapping("/allmsg")
    public ModelAndView getListOfMessages()
    {

        List<String> messages=new ArrayList<>();
        messages.add("java is very good");
        messages.add("html is related to UI");
        messages.add("spring boot");

        ModelAndView mav=new ModelAndView();
        mav.addObject("lists",messages);
        mav.setViewName("welcome");
        return mav;
    }

}

```

view file name

our controller method will return ModelAndView object

view file name

our controller method will return ModelAndView object

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1" isELIgnored="false"%>

```

```

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>

```

to use JSTL in our JSP we need to add this taglib

```

<!DOCTYPE html PUBLIC "-//W3C// DTD HTML 4.01
    Transitional// EN" "http://www.w3.org/TR/html4/loose.dtd">
<%@ page import="java.util.*"%>

```

```

<html>
<head>
<title></title>
</head>
<body>
<h1>Start here</h1>

```

```

<%
List<String> li = (List<String>) request.getAttribute("lists");

for (String s : li) {
%>
<h3><%=s%></h3>
<%
}
%>

```

scriptlet

```

<table border="1">
<c:forEach items="${lists}" var="msgs">
<tr>
<td>${msgs}</td>
</tr>
</c:forEach>

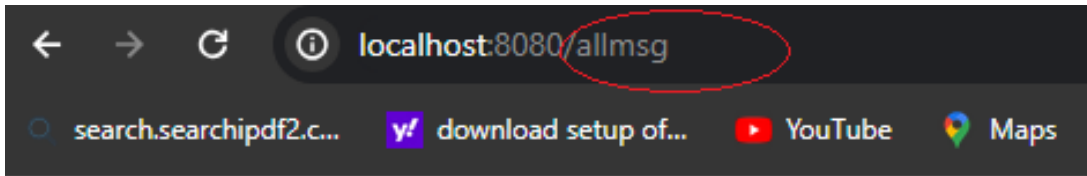
```

JSTL code

```

</table>

```



Start here

java is very good

html is related to UI

spring boot

java is very good
html is related to UI
spring boot

Using Lombok

->as we know we need to create getter and setters and if required then zero args constructor and parametrized constructor. But to avoid all these writing part we can use Lombok also,so that Lombok will take care of getter,setter,constructor and all , just we need to use Lombok annotations.

->using Lombok is not mandatory it's depend on developer requirement. Just to reduce code we can use it.

add this to pom.xml

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <scope>provided</scope>
</dependency>
```

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Department {
```

```
    private int deptId;
    private String deptName;
```

```
    }
    }
    here no
    getter, setter, constructor
    etc internally taken care
    by lombok
```

these are lombok annotations

```
@Controller
public class MyFirstController {

    @GetMapping("/deptlist")
    public ModelAndView getAllDeptDetails()
    {
        Department dept=new Department(101,"ravi");
        Department dept2=new Department(101,"arjun");
        Department dept3=new Department(102,"vijay");
        Department dept4=new Department(104,"himanshu");

        List<Department> deptlist=new ArrayList<>();
        deptlist.add(dept);
        deptlist.add(dept2);
        deptlist.add(dept3);
        deptlist.add(dept4);

        ModelAndView mav=new ModelAndView();
        mav.addObject("departments",deptlist);
        mav.setViewName("welcome");

        return mav;
    }
}
```

```
<table border="1">
  <c:forEach items="${departments}" var="dept">
    <tr>
      <td>${dept.deptId}</td>
      <td>${dept.deptName}</td>
    </tr>
  </c:forEach>
</table>
```

Sending Data From UI to Controller

->We can send data from UI to Controller in 3 ways.

- Query Params
- Path Params
- Request Body

Note:-Query Params & PathParams will represent data in URL.

Query Paramteres.

->Query parameter represent data in key-value format.

->Query parameter will present only at end of the URL.

->Query Parameter will start with '?'.

->TO use multiple query parameter we can use '&' as a separator.

->To read query parameters from URL we will use **@RequestParam** annotations.

Ex:- <https://www.youtube.com/watch> ==> This is URL

Ex:-<https://www.youtube.com/watch?v=pGxvU9EZct8>

key value

↑ ↑

↓

this represent
query parameter

Ex: www.codeminestech.com/course?name=java&fee=7999

↓

separator for
multiple parameter

Ex:-

<http://localhost:8080/course?name=angular>

```
@Controller
public class MyController {

    @GetMapping("/course")
    public ModelAndView getCoursesData(@RequestParam("name") String name)
    {
        String detail="";
        if(name.equals("java"))
        {
            detail="java is very popular lang, duration 3months,fee 3999/";
        }else if(name.equals("angular"))
        {
            detail="angular is very popular frontend framework";
        }
        else if(name.equals("springboot"))
        {
            detail="spring boot is very popular and robust backend framework";
        }else
        {
            detail="please select correct course";
        }

        ModelAndView mav=new ModelAndView();
        mav.addObject("course",detail);
        mav.setViewName("welcome");
        return mav;
    }
}
```

it will get name
parameter value from
request and put in
String name variable

http://localhost:8080/coursename?name=angular&teach=t2

```
@Controller
public class MyController {

    @GetMapping("/coursename")
    public ModelAndView getCoursesData(@RequestParam("name") String name,@RequestParam("teach") String teacher)
    {
        String detail="";
        if(name.equals("java")&&teacher.equals("t1"))
        {
            detail="java is very popular lang, duration 3months,fee 3999/";
        }else if(name.equals("angular")&&teacher.equals("t2"))
        {
            detail="angular is very popular frontend framework";
        }
        else if(name.equals("springboot")&&teacher.equals("t3"))
        {
            detail="spring boot is very popular and robust backend framework";
        }else
        {
            detail="please select correct course";
        }

        ModelAndView mav=new ModelAndView();
        mav.addObject("course",detail);
        mav.setViewName("welcome");
        return mav;
    }
}
```

Path Parameters

- >Path parameter is also used to send data to server in URL.
- >path params will represent data directly.(no key-value format)
- >path parameter can be present any where in the URL ,not necessary to keep at end you can place anywhere.
- >to represent data in URL using path param we need to use placeholder.{}

Note:- to read path parameter from URL we need to use **@PathVariable** annotations.

http://localhost:8080/coursename/java

- 1.no key-value pair.
- 2.use {} palceholder.
- 3.use @PathVariable

```
@Controller
public class MyController {

    @GetMapping("/coursename/{name}")
    public ModelAndView getCoursesData(@PathVariable("name") String name)
    {
        String detail="";
        if(name.equals("java"))
        {
            detail="java is very popular lang, duration 3months,fee 3999/";
        }else if(name.equals("angular"))
        {
            detail="angular is very popular frontend framework";
        }
        else if(name.equals("springboot"))
        {
            detail="spring boot is very popular and robust backend framework";
        }else
        {
            detail="please select correct course";
        }

        ModelAndView mav=new ModelAndView();
        mav.addObject("course",detail);
        mav.setViewName("welcome");
        return mav;
    }
}
```

http://localhost:8080/angular/coursename

- 1.no key-value pair.
- 2.use {} palceholder.
- 3.use @PathVariable
- 4.you can place data anywhere but placeholder should match position.

```
@Controller
public class MyController {

    @GetMapping("/{name}/coursename")
    public ModelAndView getCoursesData(@PathVariable("name") String name)
    {
        String detail="";
        if(name.equals("java"))
        {
            detail="java is very popular lang, duration 3months,fee 3999/";
        }else if(name.equals("angular"))
        {
            detail="angular is very popular frontend framework";
        }
        else if(name.equals("springboot"))
        {
            detail="spring boot is very popular and robust backend framework";
        }else
        {
            detail="please select correct course";
        }

        ModelAndView mav=new ModelAndView();
        mav.addObject("course",detail);
        mav.setViewName("welcome");
        return mav;
    }
}
```

http://localhost:8080/coursename/springboot/t3

```
@Controller
public class MyController {

    @GetMapping("/coursename/{name}/{teach}")
    public ModelAndView getCoursesData(@PathVariable("name") String name,@PathVariable("teach") String teacher)
    {
        String detail="";
        if(name.equals("java")&&teacher.equals("t1"))
        {
            detail="java is very popular lang, duration 3months,fee 3999/";
        }else if(name.equals("angular")&&teacher.equals("t2"))
        {
            detail="angular is very popular frontend framework";
        }
        else if(name.equals("springboot")&&teacher.equals("t3"))
        {
            detail="spring boot is very popular and robust backend framework";
        }else
        {
            detail="please select correct course";
        }

        ModelAndView mav=new ModelAndView();
        mav.addObject("course",detail);
        mav.setViewName("welcome");
        return mav;
    }
}
```

- 1.no key-value pair.
- 2.use {} palceholder.
- 3.use @PathVariable
- 4.you can place data anywhere but placeholder should match position.

Note:-Path Parameter & Query Parameter will represent data in URL.

->we can send only text data in URL not supported for binary data.

->in both the cases data is displaying in URL, not recommended for sensitive data.ex(password,pin,code,id..etc).

->there is some data limit in URL depends on browser.

->if we want to send any simple and text format data then we go for these both concept.

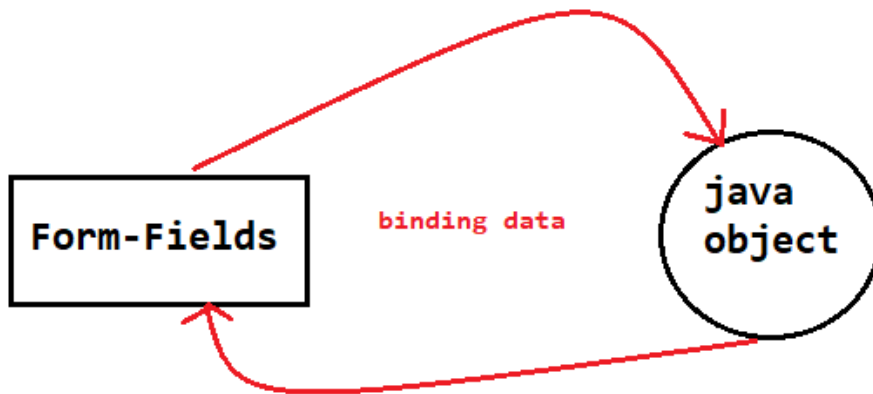
Note:- as we seen both query param and path param are used to send data in URL. So question arise where to use which.

In general if we want to retrieve multiple records then go for **Query Param**.

And if we want to retrieve unique records then go for **Path Param**.

Example related to Form based application in MVC

- >as we know form is an very essential part of any web applications
- >Form are used to collect data from users and send it to server.
- >spring web-MVC provides “**form-tag-library**” to simplify forms development.



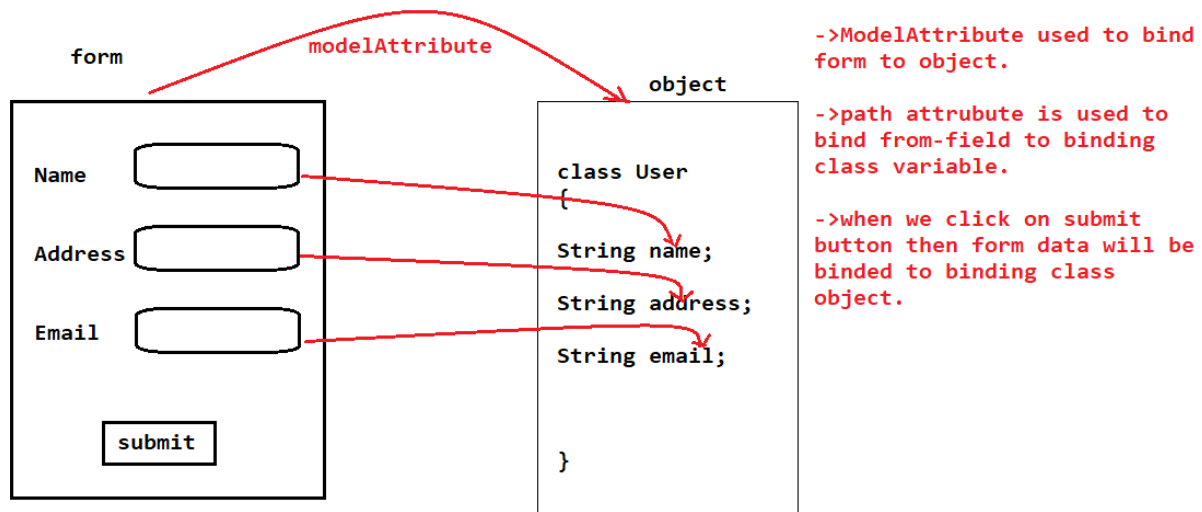
- >this spring web-mvc tag lib contains lots of tag which help to deal with form data.

Ex:

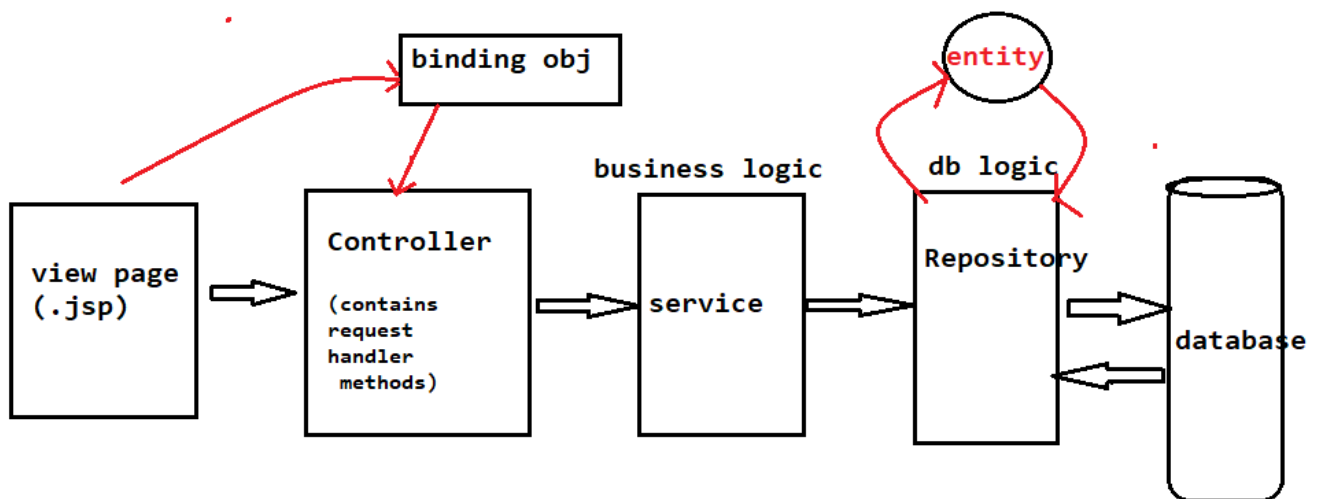
```
<form:form/>
<form:input/>
<form:password/>
<form:select/>
<form:option/> & <form:options/>
<form:radioButton/> & <form:radioButtons/>
<form:checkbox/> & <form:checkboxes/>
<form:hidden/>
<form:error/>
```

->To bind java object to form we will use '**modelAttribute**' attribute.

->To bind Form-fields(controls) to Object variable we use '**Path**' attribute.



Task1:- collecting data from form and store to db(mysql)



->we will follow this architect , here we are making service layer direct class , we can also achieve loose cupling by using interface also if required.

application.properties

```
spring.mvc.view.prefix=/view/  
spring.mvc.view.suffix=.jsp  
  
spring.datasource.url=jdbc:mysql://localhost:3306/userdb  
spring.datasource.username=root  
spring.datasource.password=root  
spring.jpa.hibernate.ddl-auto=update
```

Controller

```
@Controller  
public class MyController {  
  
    @Autowired  
    private UserService userService;  
  
    //load form  
    @GetMapping("/")  
    public ModelAndView loadForm() {  
  
        ModelAndView mav = new ModelAndView();  
        mav.addObject("user", new User());  
        mav.setViewName("index");  
        return mav;  
    }  
  
    @PostMapping("/user")  
    public ModelAndView registerUser(User newUser) {  
  
        String msg =  
            userService.saveUserData(newUser);  
        ModelAndView mav = new ModelAndView();  
        mav.addObject("msg", msg);  
        mav.setViewName("index");  
        return mav;  
    }  
  
}
```

we are making Post request from Form as we click submit button form will submit request to this handler method and bind to the object

this method load form related view and here "user" key is bind with modelAttribute in form means one User object is bind to form

```

<!DOCTYPE html>
<%@ taglib prefix="spring" uri="http://www.springframework.org/tags"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<html lang="en">
<head>
<style>
.formHead
{
text-align: center;
}
</style>
</head>
<body>

<div>
<div class="formHead"><span>Register Here</span></div>
<div class="formHead">
<form:form method="POST" action="user" modelAttribute="user">
<div><label>Enter Name</label>
<form:input type="text" path="userName" placeholder="enter name"/>
</div>

<div><label>Enter Address</label>
<form:input type="text" path="userAddress" placeholder="enter address" />
</div>

<div><label>Enter password</label>
<form:input type="password" path="userPassword" placeholder="enter password" />
</div>

<div>
<input type="submit" value="register">
</div>

```

⇒ form tag lib

entity

```

@Entity
public class User {

@Id
@GeneratedValue
private int userId;

private String userName;
private String userAddress;
private String userPassword;

---//setter
---//getter

```

@Repository layer for db logic

```
@Repository
public interface UserRepo extends JpaRepository<User,Integer> {

}
```

service

```
@Service
public class UserService {

    @Autowired
    private UserRepo userRepo;

    public String saveUserData(User user)
    {
        User saved = userRepo.save(user);
        if(saved!=null)
        {
            return "user saved Successfully!!";
        }else
        {
            return "something went wrong";
        }
    }

}

} //
```

```
<name>mvcdemo</name>
<description>Demo project for Spring Boot</description>
<properties>
    <java.version>11</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-tomcat</artifactId>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-jasper</artifactId>
    </dependency>

    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>jstl</artifactId>
    </dependency>

    <!--
    https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-data-jpa -->
    ..
```

We can also use H2 database , H2 database is embedded database provided by spring boot.

->to use H2 database we need to add H2 dependency.

->Note:- H2 database is for POC(proof of concept) means just for temporary purpose to test your concept as you shutdown your server you will loose everything)

```
<dependency>

    <groupId>com.h2database</groupId>

    <artifactId>h2</artifactId>

</dependency>
```

->configure H2 datasource properties in application.properties file

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
</dependency>
```

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.username=sa
spring.datasource.password=sa
spring.datasource.driver-class-name=org.h2.Driver
spring.h2.console.enabled=true(to see GUI you need to enable this true)
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

what ever username & password we provide here same we need to pass when we open our h2 db GUI

To see hr database GUI make request to h2-console

ex: -URL : <http://localhost:8080/h2-console>

```
@Entity
@Table(name="myuser")
public class User {

    @Id
    @GeneratedValue
    private int userId;

    private String userName;
    private String userAddress;
    private String userPassword;
}
```

in h2 db user is already one predefined schema so i have change table name using @Table annotation.

H2 db console

English ▼ [Preferences](#) [Tools](#) [Help](#)

Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:testdb

User Name: sa

Password:

Connect Test Connection

🔗

🔗

☒ Auto commit

🔗

🔗

Max rows: 1000 ▼

🟢

🟢

🔴

📄

Auto complete Off ▼

Auto select On ▼

?

📁 jdbc:h2:mem:testdb

📁 MYUSER

- 📄 USER_ID
- 📄 USER_ADDRESS
- 📄 USER_NAME
- 📄 USER_PASSWORD
- 📄 Indexes

📁 INFORMATION_SCHEMA

📄 Sequences

👤 Users

📄 H2 2.1.212 (2022-04-09)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM MYUSER|

SELECT * FROM MYUSER;

USER_ID

USER_ADDRESS

USER_NAME

USER_PASSWORD

(no rows, 2 ms)

Edit

Model interface

- >like we have seen how spring mvc uses ModelAndView object and how we are adding data and view name to it, like wise
- >we have one Model Interface and we can send data from Controller to UI using Model Object.
- >this also represent data in key-value pair.
- >we can write controller using 'Model' interface like below to send data from Controller to UI.
- >in Model we no need to set view name by using any method . directly return String your view name it is automatically understood.

```
@Controller
public class MyController {

    @Autowired
    private UserService userService;

    //load form
    @GetMapping("/")
    public ModelAndView loadForm() {

        ModelAndView mav = new ModelAndView();
        mav.addObject("user", new User());
        mav.setViewName("index");
        return mav;
    }

    @PostMapping("/user")
    public String registerUser(Model model, User newUser) {

        String msg = userService.saveUserData(newUser);
        model.addAttribute("msg", msg);
        return "index";
    }
}
```

 directly return view name in string format it is understood by spring

- >as off now we have seen spring MVC .that either you use **ModelAndView** class or **Model** Interface they both return view file

name and dispatcher servlet give this view file name to View Resolver to get location.

->means what ever string you return dispatcher thinks it like a view file name and it passes it to View Resolver and then it is identified by view resolver.

->**but if you want to send response directly without any view File. Then we need to bypass the step of ViewResolver means dispatcher servlet directly give response no need to pass it to viewResolver.**

->So to send response directly to User without any View File then we need to use annotation called **@ResponseBody**.

```
@Controller
public class MyController {

    @GetMapping("/alldata")
    @ResponseBody
    public String getUserData()
    {
        String data="all users are very good..";
        return data;
    }

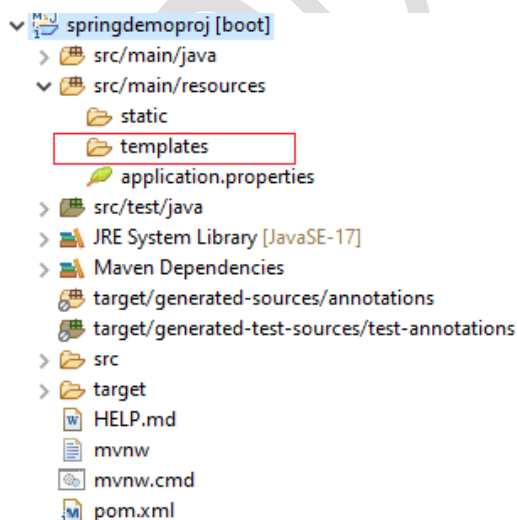
}
```

Note : @Controller + @ResponseBody = @RestController

->here we have used @ResponseBody so here response will go directly no any View file like JSP and all.

Thymeleaf

- >we used JSP as a presentation technology in our web mvc project.
- >JSP can't be executed in browser directly means when request comes to JSP then internally JSP will be converted to Servlet and Servlet will send response to browser.
- >so When we use JSP for presentation logic the burden will increase on server because every JSP should converted into Servlet and number of conversion may increase.
- >To overcome that **Thymeleaf** come into picture. And it is used for presentation logic.
- >Thymeleaf is a template engine that can be used in HTML directly.
- > as compared with JSP Thymeleaf is fast .
- >Thymeleaf provides dynamic nature to our HTML page.
- > to use thymeleaf in spring boot we need to add **thymeleaf-starter**.
- >create thymeleaf template in **Src/main/resources/templates** folder.



->file name with .html extension.

Note: No need to configure view resolver because Spring Boot will detect thymeleaf template files and will process them.

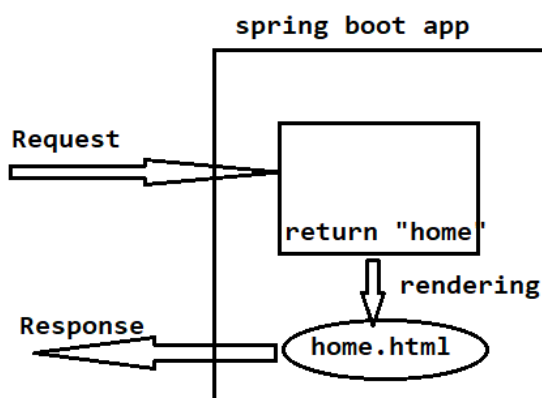
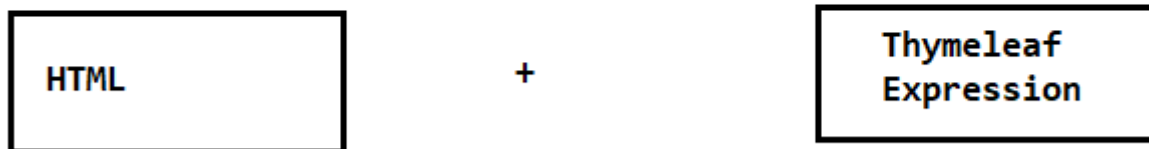
Thymeleaf is a modern server-side Java template engine for both web and standalone environments, capable of processing **HTML, XML, JavaScript, CSS**, and even **plain text**.

Thymeleaf Engine

Thymeleaf Engine will parse Thymeleaf Template. It uses Java model data to replace the positions marked on the Thymeleaf Template to create a new text in the HTML page.

- Well, in web applications, the Thymeleaf is processed on the server.
- Results are included in the HTML and returned to the browser.

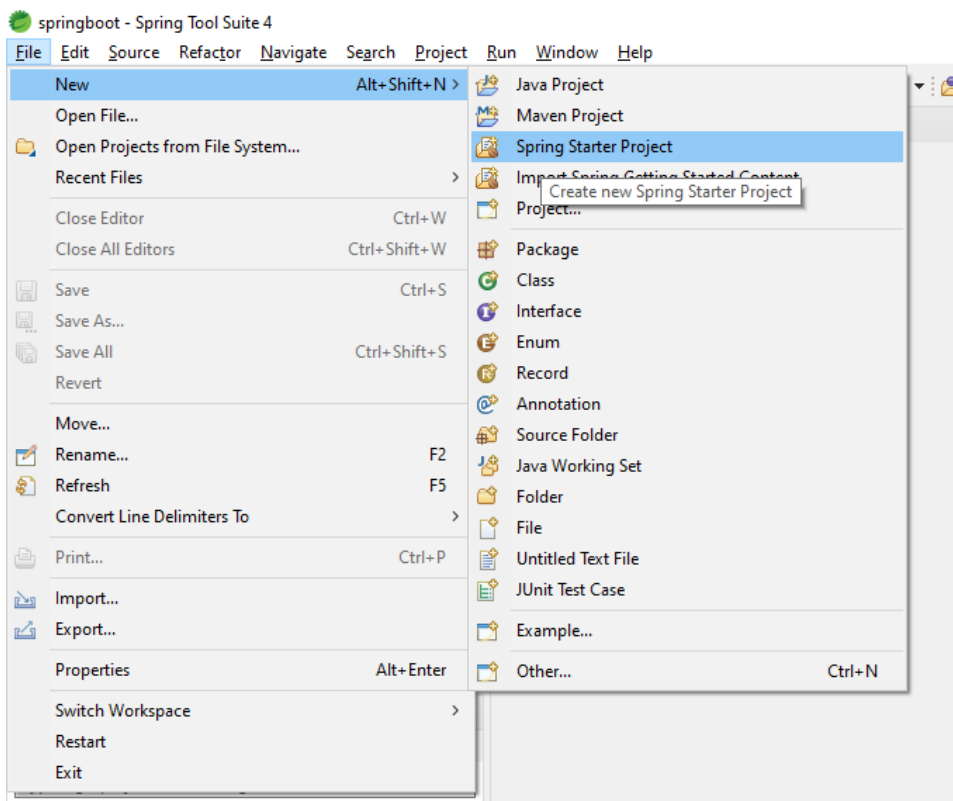
Thymeleaf Template



In Thymeleaf rendering happens in server side and then generate HTML as response

Steps to create Thymeleaf project

Step1.



Step2

New Spring Starter Project Dependencies



Spring Boot Version: 3.1.6

Frequently Used:

☐ MySQL Driver	☐ Spring Data JPA	☐ Spring Security
☒ Spring Web	☒ Thymeleaf	

Available:

thy

Template Engines

☒ Thymeleaf

Selected:

X Thymeleaf

X Spring Web

Step3

```

<artifactId>demo</artifactId>
<version>0.0.1-SNAPSHOT</version>
<packaging>war</packaging>
<name>springthymleafproj</name>
<description>Demo project for Spring Boot</description>
<properties>
    <java.version>11</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
</dependencies>

```

Step4.create controller

```

@Controller
public class MyController {

    @GetMapping("/about")
    public String myData(Model model)
    {
        System.out.println("inside about ..");

        model.addAttribute("name", "codemienstech");

        Employee emp1=new Employee(101,"emp1","addr1");
        Employee emp2=new Employee(102,"emp2","addr2");
        Employee emp3=new Employee(103,"emp3","addr3");
        Employee emp4=new Employee(104,"emp4","addr4");
        Employee emp5=new Employee(101,"emp5","addr5");
        Employee emp6=new Employee(101,"emp6","addr6");

        List<Employee> emplist=new ArrayList<>();
        emplist.add(emp1);
        emplist.add(emp2);
        emplist.add(emp3);
        emplist.add(emp4);
        emplist.add(emp5);
        emplist.add(emp6);

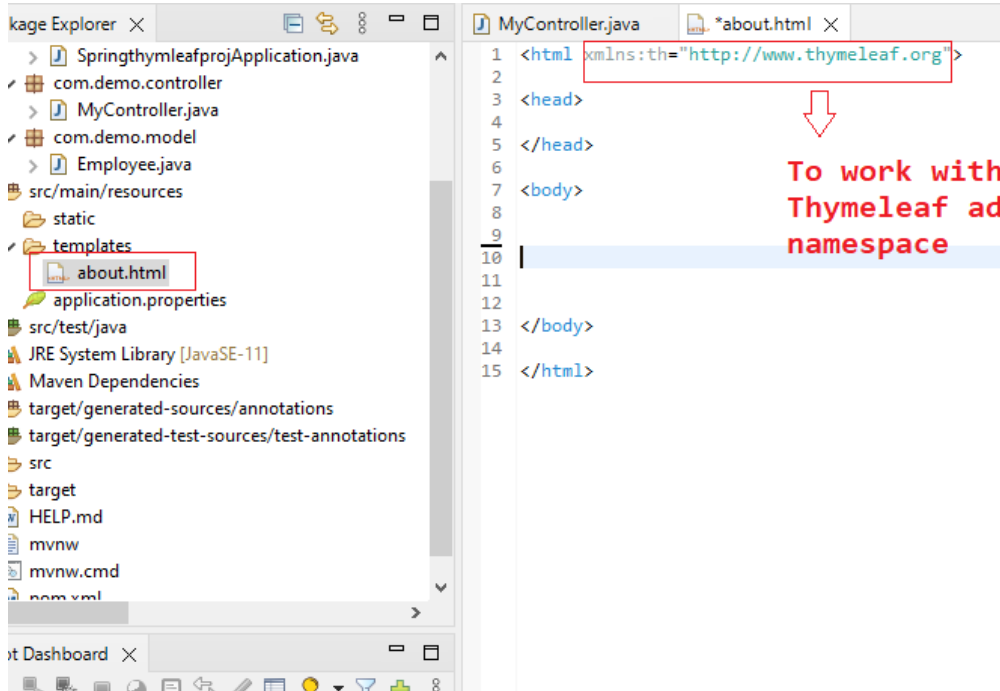
        model.addAttribute("list",emplist);

        return "about";
    }
}

```

Step5: create html file in template depending upon what view name you are returning means here i am returning "about" .

So file name must be about.html.



To work with Thymeleaf add this namespace

```
<html xmlns:th="http://www.thymeleaf.org">
```

this is namespace we need to write to work with thymeleaf.

```
<head>
```

To use Thymeleaf in html we need to use it's name space "th"

```
</head>
```

```
<body>
```

th:text is used to render text data to get dynamic data we use \${variable}

```
<h1>about us </h1>
```

```
<span th:text="${name}"></span>
```

if we want to use java predefined classes like string ,list..etc then we need to use #classname.mthod()

```
<h3>Name : <span th:text="${name}"></span></h3>
```

```
<h3>Name : <span th:text="${#strings.toUpperCase(name)}"></span></h3>
```

```
<h3>Name : <span th:text="${#strings.length(name)}"></span></h3>
```

here we are performing uppercase and counting lenght of our name variable content using predefined string class method (#)

if want to create variable on thymeleaf then we can create it using th:with . we can use these variable in same context only means in this div

```
<div th:with="a=5,b=6,n=${name}">
  <h1 th:text="${a}"></h1>
  <h1 th:text="${b}"></h1>
  <h1 th:text="${n}"></h1>
</div>
```

To iterate something in thymeleaf we need to use
`th:each`

```
<table border="1">
  <tr>
    <td>id</td>
    <td>name</td>
    <td>address</td>
  </tr>
  <tr th:each="val:${list}">
    <td th:text="${val.empId}"></td>
    <td th:text="${val.empName}"></td>
    <td th:text="${val.empAddress}"></td>
  </tr>
</table>
```

this is a list type variable rendering from controller

we get following status for iterative items
index: give current index (starts from 0)
count: give the number of elements processed so far
size: the total number of elements in list
even/odd: checks if the current iteration index is even or odd
first: checks if the current iteration is the first one
last: checks if the current iteration is the last one.

```
<table border="1">
  <tr>
    <td>id</td>
    <td>name</td>
    <td>address</td>
  </tr>
  <tr th:each="val,status:${list}">
    <td th:text="${status.index}+''+${val.empId}"></td>
    <td th:style="${status.even?'color:blue':''}" th:text="${status.index}+''+${val.empName}"></td>
    <td th:text="${status.index}+''+${val.empAddress}"></td>
  </tr>
</table>
```

to give css style we can use `th:style`

Conditionals in Thymeleaf

Elvis Operator
(`con?"true":"false"`)

If-Unless
`th:if`
`th:unless`

Switch-case

Conditionals in Thymeleaf

Elvis Operator
(con?"true":"false")

If-Unless
th:if
th:unless

Switch-case

ex:-
<h1 th:if="\${gender}=='M'">Male</h1>
<h1 th:unless="\${gender}=='M'">Female</h1>

```
<div th:switch="${num}">
<h1 th:case="'0'">zero</h1>
<h1 th:case="'1'">one</h1>
<h1 th:case="'2'">two</h1>
</div>
```

```
<span th:text="${empid}?'user is there':'something went wrong'"></span>
```

```
public class Employee {
    private int empId;
    private String empName;
    private String empAddress;

    public Employee(int empId, String empName, String empAddress) {
        super();
        this.empId = empId;
        this.empName = empName;
        this.empAddress = empAddress;
    }
}

<form th:action="@{/createUser}" th:object="${employee}" method="post">
    <input type="text" placeholder="enter name" th:field="**{empId}" />
    <input type="text" placeholder="enter name" th:field="**{empName}" />
    <input type="text" placeholder="enter address" th:field="**{empAddress}" />
    <input type="submit" value="submit" />
</form>

@GetMapping("/about")
public String myData(Model model)
{
    //to load about page where form is present
    model.addAttribute("employee", new Employee(0, null, null));
    return "about";
}

@PostMapping("/createUser")
public ModelAndView createEmp(Employee employee)
{
    ModelAndView mav=new ModelAndView();

    if(employee!=null)
    {
        mav.addObject("employee",employee);
        mav.setViewName("success");
        return mav;
    }
    else
    {
        mav.addObject("msg","something went wrong");
        mav.setViewName("error");
        return mav;
    }
}
```

Spring Boot + Thymeleaf + Form validations – Example

We have one dependency for Form validation we need to add that so that we can perform Form Validations.

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-validation</artifactId>

</dependency>

```

|
public class Person {

    @NotNull
    @Size(min = 3, max = 8)
    private String name;

    @NotNull
    @Min(18)
    private Integer age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Person [name=" + name + ", age=" + age + "]";
    }

}

```

@NotNull ,@Size,@Min ..etc annotations
validate our object data if we violate it it will generate issue

```

@Controller
public class PersonController {

    @GetMapping("/person")
    public String displayForm(Model model) {
        Person personObj = new Person();
        model.addAttribute("person", personObj);
        return "index";
    }

    @PostMapping("/savePerson")
    public String savePerson(@Valid Person person, BindingResult result, Model model) {
        System.out.println(person);

        if (result.hasErrors()) {
            return "index";
        }

        model.addAttribute("msg", person.getName() + " record saved successfully");
        return "data";
    }

}

```

as we have define annotation related to validation in our pojo/entity classes then in controller handler method we need to pass **@Valid** annotation in front of pojo/entity paramter and **bindingResult** object

BindingResult object provides method support through which we can check error is exist or not depend on that we can provide response

```

<!DOCTYPE html>
<html xmlns:th="https://www.thymeleaf.org">
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>

    <form th:action="@{/savePerson}" th:object="${person}" method="POST">
        <table>
            <tr>
                <td>Name :</td>
                <td><input type="text" th:field="*{name}" /></td>
                <td th:if="${#fields.hasErrors('name')}" th:errors="*{name}">Name
                    Error</td>
            </tr>
            <tr>
                <td>Age :</td>
                <td><input type="text" th:field="*{age}" /></td>
                <td th:if="${#fields.hasErrors('age')}" th:errors="*{age}">Age
                    Error</td>
            </tr>
            <tr>
                <td></td>
                <td><input type="submit" value="Save" /></td>
            </tr>
        </table>
    </form>

</body>
</html>

```

we can display error in frontend using conditional statement as well as **#fields.hasErrors** properties

difference b/w application.properties & application.yml

- >When we create spring boot application, by default project provides **application.properties** files under **src/main/resources** folder.
- >all the project configuration we do inside **application.properties** file.
- >**application.properties** file supports configuration details in **key-value** format.

Ex:-

application.properties

```

spring.mvc.view.prefix=/view/
spring.mvc.view.suffix=.jsp

spring.datasource.url=jdbc:mysql://localhost:3306/userdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update

```

- >application.properties file will represent data in sequential manner.

.YML files

- >it is an alternative of application.properties file.
- >YML stands for **yet another markup language**.
- >YML Files will represent data in **hierarchical** manner.
- >YML files is more readable then .properties files.

application.properties

```
spring.mvc.view.prefix=/view/  
spring.mvc.view.suffix=.jsp
```

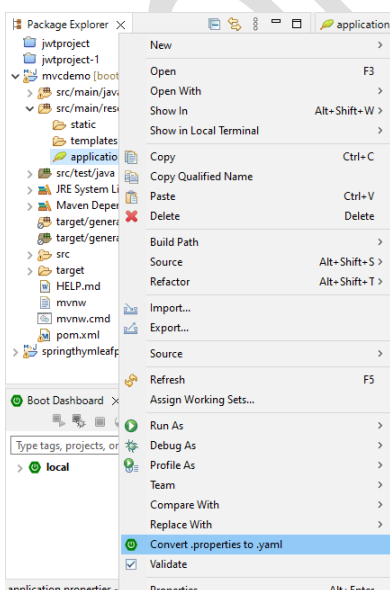
application.yml

```
spring:  
  mvc:  
    view:  
      prefix: /view/  
      suffix: .jsp
```

inside view we have prefix ,suffix so in yml .we no need to write 2 separate line. in one hierarachy we defined it

Note:- be careful of spaces while writing .YML files .give proper spaces .

->**we can also convert Our application.properties files to application.Yml files**



Note:- `application.properties` file will support only java where as `yml` files are supported by multiple programming languages.

->In profiles concept , for every profile we need to create separate `application.properties` file.

->but in case of `YML` , **single `yml` file is sufficient to deal with all profiles.**

Profiles:- In real time application are developed and tested in various environments like `dev`,`uat`,`local`..etc.

->so we need to make separate configuration files for every environment if we are using `.properties` file like

`Application.properties`,`application-dev.properties`,`application-uat.properties`,`application-prod.properties`.

`application.properties` $\Rightarrow$ base file
`application-dev.properties` $\Rightarrow$ dev environemnt
`application-prod.properties` $\Rightarrow$ prod environment

`application.yml` $\Rightarrow$ base file
`application-dev.yml` $\Rightarrow$ dev environment
`application-prod.yml` $\Rightarrow$ prod environment

for `YML` approach you can either make
separte files as well as in one `yml` file
also you can manage

Same we can also defined in `.YML` approach. In `.YML` we have two option either you can create separate file for different profile or in same file also you can manage.

->we need to define profile in base file that spring need to use which environment file for configuration .

->if we not use any profile concept then we need to change our configuration file every time depends on environment but it is very bad approach to change our configuration file in real time because if by mistake something went wrong then it may cause big issue.

Ex:- suppose in real time we have 3 environment dev, test, prod environment and for every environment we have separate database for our application for production we have production db config which connects with database which store production data.

For test we have separate database config which related with database which stores test data used for testing.

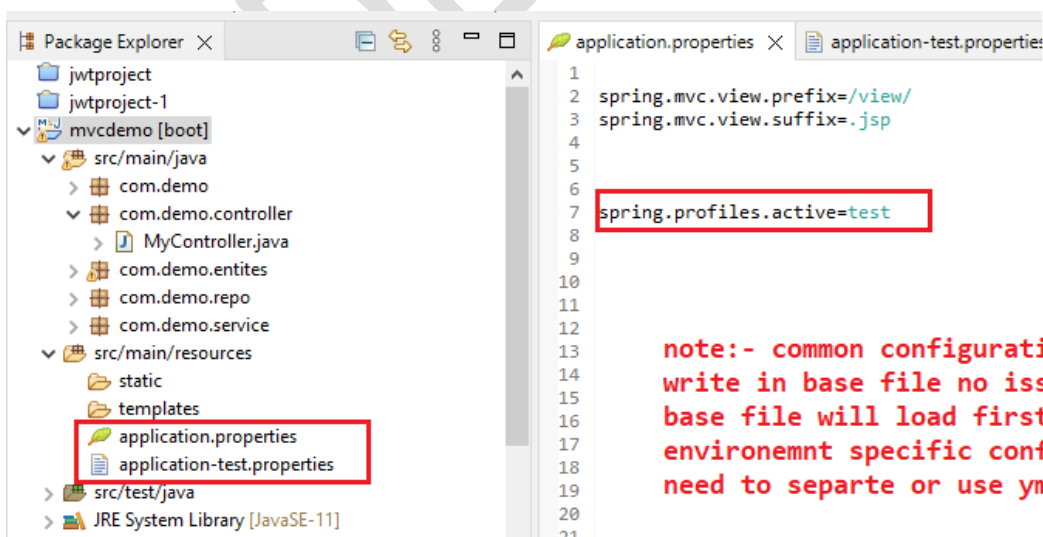
Similarly we have separate database in local system which used for development purpose so we have separate config related to that dev config.

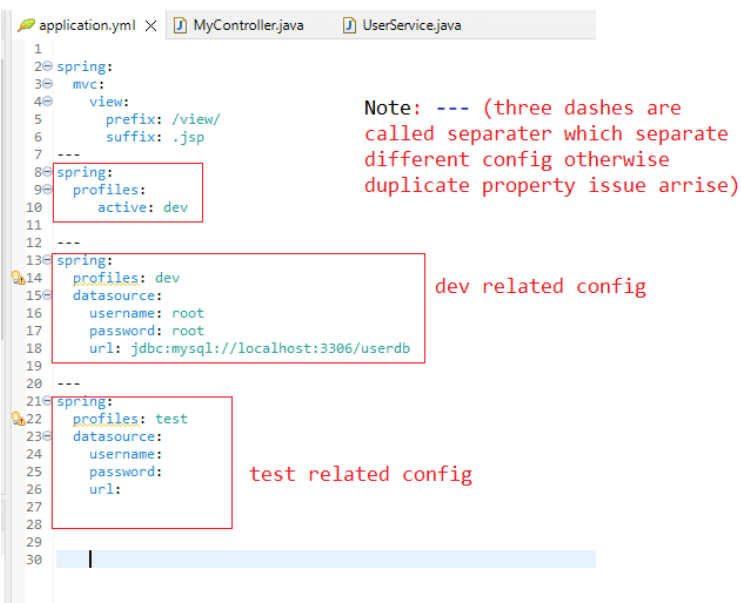
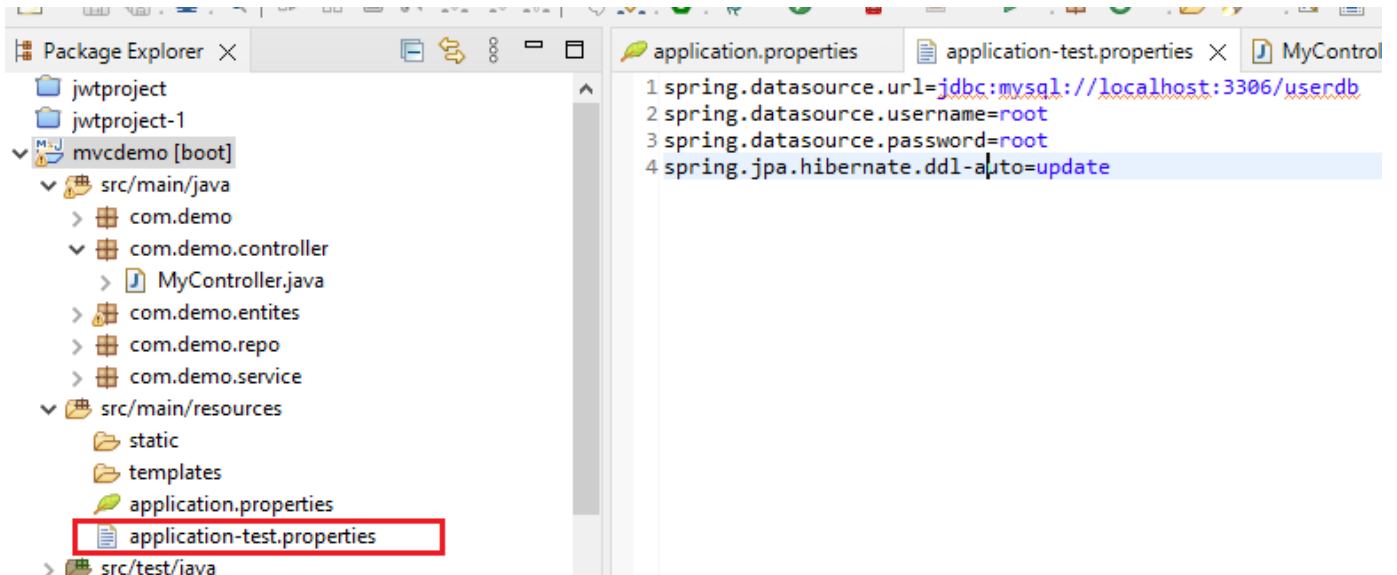
So if we change every time then it become headic for developer as well as chances of getting issue is high.

So to overcome this we can use **Profiles** concepts.

->by default spring boot loads application.properties file i.e base file.

->so in base file you need to specify which environment specific file project should used.





->in latest version you can config like below

The main editor displays the content of `application.yml` with a more detailed configuration:

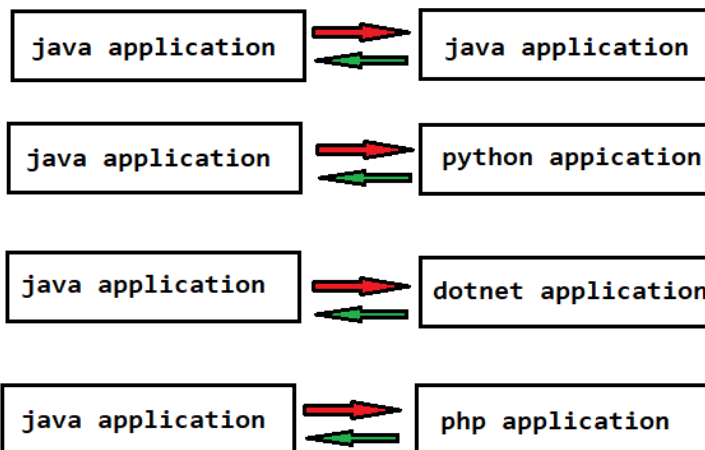
```
1 spring:
2   application:
3     name: myfifthprojectwithprofiles
4   ---
5   spring:
6     profiles:
7       active: prod
8   ---
9   spring:
10    config:
11      activate:
12        on-profile: dev
13      datasource:
14        url: jdbc:mysql://localhost:3306/thirdproject
15        username: root
16        password: root
17      jpa:
18        hibernate:
19          ddl-auto: update
20          show-sql: true
21  ---
22  spring:
23    config:
24      activate:
25        on-profile: test
26      datasource:
27        url: jdbc:mysql://localhost:3306/mytestdb
28        username: root
29        password: root
30  jpa:
```

What are RESTful Services ?

->Restful Services are used to develop “**Distributed Application**”. Which is independent of any programming language as we already discussed above distributed languages.

->as we already know if one application is communicating with another application then we call it as distributed application.

->distributed applications are interoperable means irrespective of the platform and language if application are communicating then we call them as interoperable applications.



Interoperability

irrespective of the platform and language if applications are communicating then we will call them as Interoperable applications.

Why we should use Distributed Application?

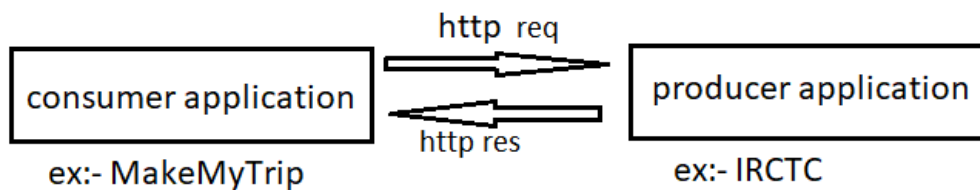
->As we already discuss what is distributed application and it's use case.

->we have already discussed about B2B how companies are doing B2B using distributed applications.

->we already know what is **producer** application and what is **consumer** application.

->The application which is providing business services to other applications is called as Provider

->The application which is consuming business services from other applications is called as Consumer.



Consumer application will send request to Provider application with some data. Provider application will process that data and will send response to consumer.

->application can exchange data over network in **XML,JSON** format

->**JSON** format is trending in market as well all latest tech stack supports JSON format.

Now days JSON is ruling over market.

->lets see something related to XML approach first.

XML

->XML stands for Extensible Markup Language.

->it is governed by w3C.

->XML is also tag based language. Which represent data in elements like

Ex:- <id>111</id>

->elements tag are paired means contains start tag and end tags.

-> Every xml will start with prolog

```
<?xml version="1.0" encoding="UTF-8">
```

-> Prolog represents processing instructions

Note : Every XML should contain only one root element

```
<?xml version="1.0" encoding="UTF-8">
```

```
<student>
  <id>101</id>
  <name>John</name>
  <rank>101</rank>
</student>
```

This is root tag which represent java side binding class

this tag represent binding class property name.

-> In the above xml we have 2 types of elements

- 1) Simple Element
- 2) Compound Element

What is Simple Element?

The element which represents data directly is called as Simple Element

Ex: `<id>111</id>`

What is Compound Element?

The Element which represents child elements is called as Compound Element.

```
<student>
  <id>111</id>
</student>
```

-> We can use attributes also in the xml

ex: `<book-name type="java">Spring</book-name>`

-> Attributes are used to provide supplement information for elements

-> XML is interoperable and we already discussed about interoperability.

-> SOAP Webservices will support only xml format for exchanging data

-> RESTful services supports for xml and some other formats also like json.

->so to transfer/exchange data from one app to another app we need to convert one type format to respective type Object.

Lets see how to convert **XML to java Object** and **java Object to XML**.

Marshalling & Un-Marshalling

->Marshalling & Un-Marshalling are called Runtime Operation.

->The process of converting java Object into XML is called as **Marshalling**.

->The process of converting XML into java Object is called as **Un-Marshalling**.

Note: These operations will happen when the application is running

Note: To perform Runtime operations Binding classes are mandatory.

Working with JAX-B

-> JAX-B api & its implementations are part of JDK only (directley we can use it)

-> From Java 11 onwards JAX-B removed from JDK, we have to add JAX-B dependency in pom.xml.

Marshalling program using jaxb api.

->in program we are just focusing on marshalling (java obj-- > XML).

->create one java class.

->add jaxb dependency to POM.xml

->depends on spring version jaxb may differ.

->then in main class create Marshall object and just pass object and file name with xml extension it will create file .

TestprojectApplication.java application.yml testproject/pom.xml X Student.java student.xml

```
7 <groupId>org.springframework.boot</groupId>
8 <artifactId>spring-boot-starter-parent</artifactId>
9 <version>3.1.6</version>
10 <relativePath /> <!-- lookup parent from repository -->
11 </parent>
12 <groupId>com.demo</groupId>
13 <artifactId>demo</artifactId>
14 <version>0.0.1-SNAPSHOT</version>
15 <name>testproject</name>
16 <description>Demo project for Spring Boot</description>
17 <properties>
18 <java.version>11</java.version>
19 </properties>
20 <dependencies>
21
22 <dependency>
23 <groupId>org.springframework.boot</groupId>
24 <artifactId>spring-boot-starter-web</artifactId>
25 </dependency>
26
27
28 <dependency>
29 <groupId>org.springframework.boot</groupId>
30 <artifactId>spring-boot-starter-test</artifactId>
31 <scope>test</scope>
32 </dependency>
33
34 <dependency>
35 <groupId>com.sun.xml.bind</groupId>
36 <artifactId>jaxb-impl</artifactId>
37 </dependency>
38
39 <dependency>
40 <groupId>jakarta.xml.bind</groupId>
41 <artifactId>jakarta.xml.bind-api</artifactId>
42 </dependency>
43
```

TestprojectApplication.java application.yml testproject/pom.xml

```
1 package com.demo.entites;
2
3 import jakarta.xml.bind.annotation.XmlRootElement;
4
5 @XmlRootElement
6 public class Student {
7
8     private int id;
9     private String name;
10    private long rank;
11    public int getId() {
12        return id;
13    }
14    public void setId(int id) {
15        this.id = id;
16    }
17    public String getName() {
18        return name;
19    }
20    public void setName(String name) {
21        this.name = name;
22    }
23    public long getRank() {
24        return rank;
25    }
26    public void setRank(long rank) {
27        this.rank = rank;
28    }
29
30
31
32
33 }
34
```

TestprojectApplication.java × application.yml testproject/pom.xml Student.java student.xml

```

1 package com.demo;
2
3 import java.io.File;
4
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7 import org.springframework.context.ConfigurableApplicationContext;
8
9 import com.demo.entites.Student;
10
11 import jakarta.xml.bind.JAXBContext;
12 import jakarta.xml.bind.JAXBException;
13 import jakarta.xml.bind.Marshaller;
14
15 @SpringBootApplication
16 public class TestprojectApplication {
17
18     public static void main(String[] args) throws JAXBException {
19         ConfigurableApplicationContext context = SpringApplication.run(TestprojectApplication.class, args);
20
21         Student sobj = new Student();
22         sobj.setId(101);
23         sobj.setName("hari");
24         sobj.setRank(144);
25
26         JAXBContext ins = JAXBContext.newInstance(Student.class);
27         Marshaller mar = ins.createMarshaller();
28         mar.marshal(sobj, new File("student.xml"));
29
30     }
31 }
32
33

```

TestprojectApplication.java application.yml testproject/pom.x

Bind to grammar/schema...

```

1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <student>
3   <id>101</id>
4   <name>hari</name>
5   <rank>144</rank>
6 </student>

```

In spring 2.x

for 2.x version

```

<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>2.3.4</version>
</dependency>

```

```

<dependency>

  <groupId>javax.xml.bind</groupId>

  <artifactId>jaxb-api</artifactId>

  <version>2.3.1</version>

</dependency>

```

un-marshalling (xml to java object)

```
1 package com.demo;
2
3 import java.io.File;
4
5
6
7
8
9
10
11
12
13
14
15
16 @SpringBootApplication
17 public class TestprojectApplication {
18
19     public static void main(String[] args) throws JAXBException {
20         ConfigurableApplicationContext context = SpringApplication.run(TestprojectApplication.class, args);
21
22         // Student sobj = new Student();
23         // sobj.setId(101);
24         // sobj.setName("hari");
25         // sobj.setRank(144);
26         //
27         // JAXBContext ins = JAXBContext.newInstance(Student.class);
28         // Marshaller mar = ins.createMarshaller();
29         // mar.marshal(sobj, new File("student.xml"));
30
31         JAXBContext ins=JAXBContext.newInstance(Student.class);
32         Unmarshaller unmar = ins.createUnmarshaller();
33         Student stu =(Student) unmar.unmarshal(new File("student.xml"));
34
35         System.out.println(stu.getId()+" "+stu.getName()+" "+stu.getRank());
36
37     }
38
39 }
40 }
```

Here we using unmarshal() methods. Which again converts xml into object.

JSON

- >JSON stands for javascript object Notation.
- >JSON format is universal format to exchange data over a network.
- >JSON is intereoperable (platform independent & language independent).
- >as compared to XML ,JSON is light weight (it occupies less memory).
- >JSON represent data into key-value format.
- >JSON object always starts with { and ends with }.
- >with in { and } , JSON object contains multiple elements where each elements represent properties.
- >elements are present in key:value structure.

->each key:value pair is separated by comma(,).

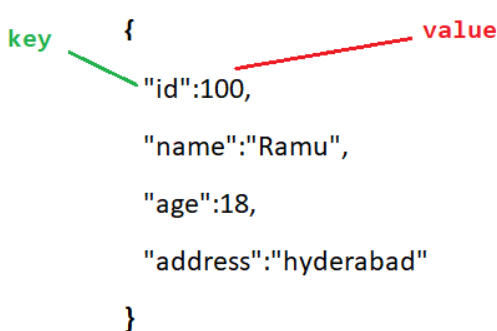
->The values corresponding to JSON keys can be String , Integer , boolean , JSONObject array.

->JSON array can contains array or list of String,numbers,json object etc.

->JSON array starts with [and] .

json object example

```
{  
  "id":100,  
  "name":"Ramu",  
  "age":18,  
  "address":"hyderabad"  
}
```



-> In Java we don't have direct support to work with JSON data

-> We have third party apis to work with JSON data in java applications

- **Jackson Api**

- **Gson Api**

-> Using above third party apis **we can convert Java Object to Json and Json to Java Object.**

-> Converting **Java Object into JSON data** is called as **Serialiation.**

-> Converting **JSON data to Java Object** is called as **De-Serialization.**

-> Normal POJO classes we can use to perform Serializaion and De-Serialization with JACKSON api

Working with Jackson API

Java object to json

Add maven dependency

```
<dependency>
<groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-databind</artifactId>
</dependency>
```

Create pojo class

```
public class Teacher {

    private int tid;
    private String tname;
    private String tschool;
    private String tquali;
    public int getTid() {
        return tid;
    }
    public void setTid(int tid) {
        this.tid = tid;
    }
    public String getTname() {
        return tname;
    }
    public void setTname(String tname) {
        this.tname = tname;
    }
    public String getTschool() {
        return tschool;
    }
    public void setTschool(String tschool) {
        this.tschool = tschool;
    }
    public String getTquali() {
        return tquali;
    }
    public void setTquali(String tquali) {
        this.tquali = tquali;
    }
}
```

Create main logic

```
@SpringBootApplication
public class JacksondemoApplication {

    public static void main(String[] args) throws JsonProcessingException {
        SpringApplication.run(JacksondemoApplication.class, args);

        Teacher teacher=new Teacher();
        teacher.setTid(101);
        teacher.setTname("him");
        teacher.setTquali("dev");
        teacher.setTschool("techno");

        ObjectMapper mapper=new ObjectMapper();
        String teacherjson = mapper.writeValueAsString(teacher);

        System.out.println(teacherjson);

    }
}
```

If we want json file then use

```
@SpringBootApplication
public class JacksondemoApplication {

    public static void main(String[] args) throws IOException {
        SpringApplication.run(JacksondemoApplication.class, args);

        Teacher teacher=new Teacher();
        teacher.setTid(101);
        teacher.setTname("him");
        teacher.setTquali("dev");
        teacher.setTschool("techno");

        ObjectMapper mapper=new ObjectMapper();

        mapper.writeValue(new File("teacher.json"), teacher);

        this mapper writeValue() method is used to create json string and
        write to teacher.json file
    }
}
```

Now JSON to JAVA

->we use readValue(jsonfile,object)

json to object

```
@SpringBootApplication
public class JacksondemoApplication {

    public static void main(String[] args) throws IOException {
        SpringApplication.run(JacksondemoApplication.class, args);

        ObjectMapper mapper=new ObjectMapper();

        Teacher teach = mapper.readValue(new File("teacher.json"), Teacher.class);

        System.out.println(teach.getTid()+" "+teach.getTname()+" "+teach.getTquali()+" "+teach.getTschool());
    }
}
```

->there are lots of method but this is most imp concept.

Working with GSON API

->In gson api we have a predefined class called 'gson'.

->Gson class contains below methods

- **toJson ()** ==> To convert java obj to json format
- **fromJson ()** ==> To convert json to java obj

Java object to JSON string

java object to json using Gson

```
@SpringBootApplication
public class GsondemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(GsondemoApplication.class, args);

        Teacher tech=new Teacher();
        tech.setTid(101);
        tech.setTname("codemines");
        tech.setTquali("webdev");
        tech.setTaddress("jbp");

        Gson gson=new Gson();

        String jsonstring = gson.toJson(tech);

        System.out.println(jsonstring);

    }
}
```

However, the `toJson` method in `Gson` doesn't have an overload that directly takes a `File` as a parameter.

You should first convert the object to a JSON string using `toJson`, and then write that string to the file.

```

@SpringBootApplication
public class GsendemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(GsendemoApplication.class, args);

        Teacher tech=new Teacher();
        tech.setTid(101);
        tech.setTname("codemines");
        tech.setTquali("webdev");
        tech.setTaddress("jbp");

        Gson gson=new Gson();
        String jsonstring = gson.toJson(tech);

        try (FileWriter writer = new FileWriter("tech.json")) {

            writer.write(jsonstring);

        } catch (IOException e) {
            e.printStackTrace();
        }

    }

}

```

Now JSON to java Object

```

@SpringBootApplication
public class GsendemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(GsendemoApplication.class, args);

        Gson gson = new Gson();

        Teacher t = gson.fromJson("{\"tid\":\"101\",\"tname\":\"codemines\",\"taddress\":\"jbp\",\"tquali\":\"webdev\"}", Teacher.class);

        System.out.println(t.getTid() + " " + t.getTaddress() + " " + t.getTname() + " " + t.getTquali());

    }

}

```

json string
java object type

↓
↓

->above code we have pass json string directly. We can also pass FileReader as an parameter see below

```

@SpringBootApplication
public class GsondemoApplication {

    public static void main(String[] args) throws JsonSyntaxException, JsonIOException, FileNotFoundException {
        SpringApplication.run(GsondemoApplication.class, args);

        Gson gson = new Gson();
        Teacher t = gson.fromJson(new FileReader("tech.json"), Teacher.class);

        System.out.println(t.getId() + " " + t.getAddress() + " " + t.getName() + " " + t.getTquali());
    }
}

```

HTTP Protocol

- >HTTP stands for **hyper text transfer protocol**.
- >Http is a kind of medium or we can say mediator protocol between client & server.
- >in client server-architecture . client will send HTTP request to server and server will process that request and send HTTP response back to client.
- >Http is stateless protocol.
- >stateless means it can't remember the conversation happened between client & server.

HTTP Methods

- >there are several method which help us to deal with http request/response
 - GET:-it is used to get response from the server.
 - POST:-it is used to send data to server(used in creation purpose)
 - PUT:-it is used to update record at server.
 - DELETE:-it is used to delete the record at server.

->When we deal with REST API and all then we need to bind Our controller end methods with these HTTP protocol methods.

Ex:-

GET -----> @GetMapping

POST -----> @PostMapping

PUT-----> @PutMapping

DELETE-----> @DeleteMapping.

Request/response contains two major parts i.e

- Header (it contains metadata)
- Body(it contains payload (business data))

Some HTTP Status Code

-> Response Header will contains HTTP Status Code. When we send response.

->actually this status code give information that how our request get processed .every thing fine or not.

1xx - INFORMATIONAL

2xx - OK (Success)

3xx - Redirectional (www.sunmicrosystem.com -> www.oracle.com)

4xx - Client Error ex:- 404

5xx - Server Error

Note: Http Status Codes will play very important in distributed applications development

HTTP Status Code

level 2xx	level 4xx	level 5xx
200:ok 201:created 202:Accepted 203:Non-Authoritative 204:No Contentetc	400:Bad Request 401:Unauthorized 403:Forbidden 404:Not Found 409:Conflict ...etc	500:Internal Server Error 501:Not implemented 502:Bad Gateway 503:Service unavailable 504:Gateway Timeout 599:network timeout ...etc.

Note: POST, PUT and DELETE Methods having request body. GET method don't have request body.

Building First REST API using Spring Boot

Let us know some more before building first api.

-> **ResponseEntity** is a predefined class available in Spring Web MVC

-> Using **ResponseEntity** class we can construct Response with body and status code.

Syn:- new ResponseEntity<>(body, HttpStatus);

Note: REST API Methods should have Unique URL Patterns otherwise we will get ambiguity problem. We need to make sure our methods should be identified uniquely.

->we will use **@RestController** because **@RestController** method will return response directly.

Note: No concept of returning "view file names" in RestController.

@Controller + @ResponseBody = **@RestController**

-> **@GetMapping** annotation is used to bind RestController method to HTTP **GET** Req.

-> HTTP GET request is used to get data from server

-> HTTP GET request will not have body

-> To send (small piece of data like Id,key or etc) data to server in GET request we will use below 2 options

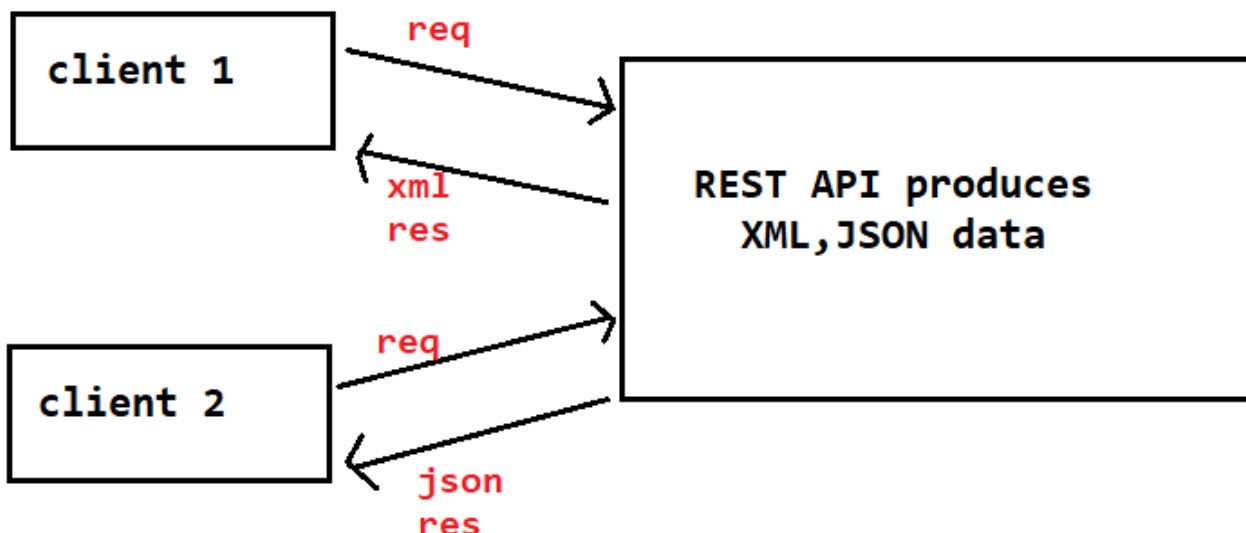
1) QueryParams

2) PathParams

These two we already discussed before.

Note: In Spring Boot, we no need to write JAX-B and Jackson logics directly because Spring Boot will take care of that.

Note: Spring Boot internally uses **HttpMessageConverters** to deal with XML and JSON.



->Now days we uses JSON format to exchange data over applications

First Rest API using SpringBoot.

1) Create Spring Starter Application with below dependencies

- **spring-boot-starter-web**

- **spring-boot-devtools**

2) Create **RestController** class using **@RestController** annotation

3) Write methods in RestController and bind them to HTTP Protocol method

4) Run the application and test it

Note: To test REST API functionality we will use **POSTMAN** software

```
@RestController
public class MyController {

    @GetMapping("/")
    public String getData()
    {
        return "hii welcome to codeminestech";
    }

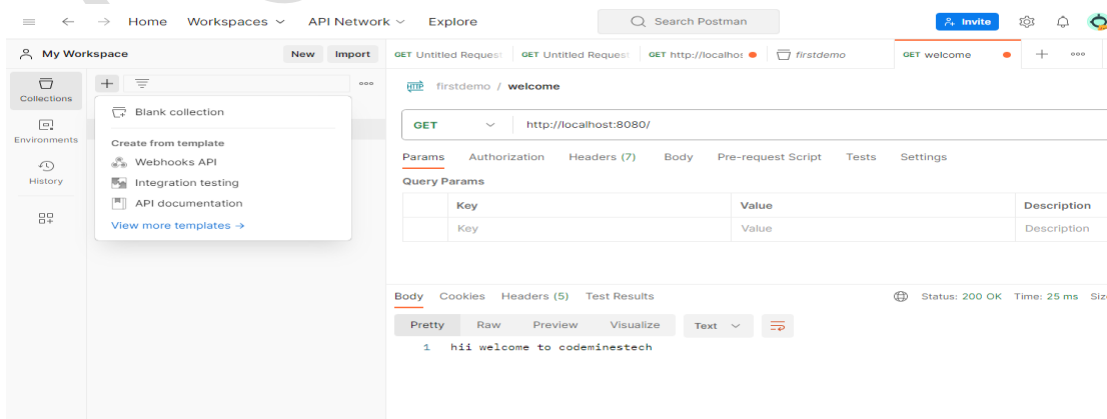
    @GetMapping("/msg")
    public ResponseEntity<String> getMessage()
    {
        String msg="welcome java full stack!!";
        return new ResponseEntity<>(msg,HttpStatus.OK);
    }

}

//
```

↓ ↓

data http code



firstdemo / welcome

Save

Send

GEThttp://localhost:8080/

ParamsAuthorizationHeaders (7)BodyPre-request ScriptTestsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

BodyCookiesHeaders (5)Test Results

Status: 200 OKTime: 25 msSize: 192 BSave as example

PrettyRawPreviewVisualizeText

1 hii welcome to codeminestech

Dealing with XML response.

->first add JAXB API for XML ..because bydefault it gives JSON response no need any configuration for JSON but for XML we need to use **@XmlRootElement**

```
<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
</dependency>

<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
</dependency>
```

```

@XmlRootElement
public class Student {

    private int sid;
    private String sname;
    private String saddress;
    public int getSid() {
        return sid;
    }
    public void setSid(int sid) {
        this.sid = sid;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSaddress() {
        return saddress;
    }
    public void setSaddress(String saddress) {
        this.saddress = saddress;
    }
}

}

```

```

@RestController
public class MyController {

    @GetMapping("/")
    public String getData()
    {
        return "hii welcome to codeminestech";
    }

    @GetMapping(value="/msg",produces= {"application/xml","application/json"})
    public ResponseEntity<Student> getMessage()
    {
        Student stu=new Student();
        stu.setSid(101);
        stu.setSaddress("jbp");
        stu.setSname("rakesh");

        return new ResponseEntity<>(stu,HttpStatus.OK);
    }
}

}

```

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/msg`
- Method:** `GET`
- Headers:**

Key	Value	Description
☒ <code>Accept</code>	<code>application/xml</code>	
- Body:**

```

1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <student>
3   <saddress>jbp</saddress>
4   <sid>101</sid>
5   <sname>rakesh</sname>
6 </student>

```
- Status:** 200 OK, Time: 1388 ms, Size: 296 B

In header we need to pass `accept :application/xml`.

Note-1 : In `@GetMapping` annotation, "**produces**" attribute represents in which formats our REST API method can provide response to clients

Note-2 : Client will send request with "Accept" header.

Ex : `Accept = "application/json"`

Based on "**Accept**" header REST API will send response to client in that format. Internally "**HTTP Msg Converters**" will be used to convert the object data into client understandable format.

produces & Accept

=> "produces" attribute represents in which formats REST API method can provide response

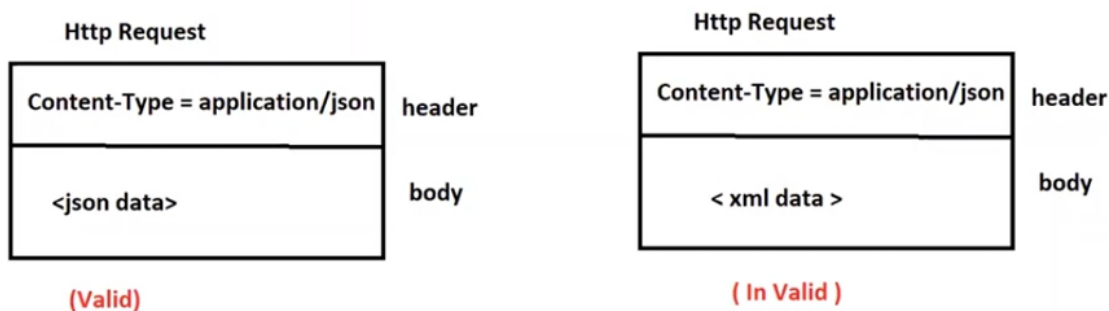
=> "Accept" header represents in which format client expecting response from REST API.

Dealing with XML & JSON data in request body

-> To send sensitive data and huge amount of data clients will use Request Body.

-> "**Content-Type**" header represents in which format client sending data in Request Body.

-> "**consumes**" represents in which formats REST API method can take input data.



-> if we send request with some huge data then we use

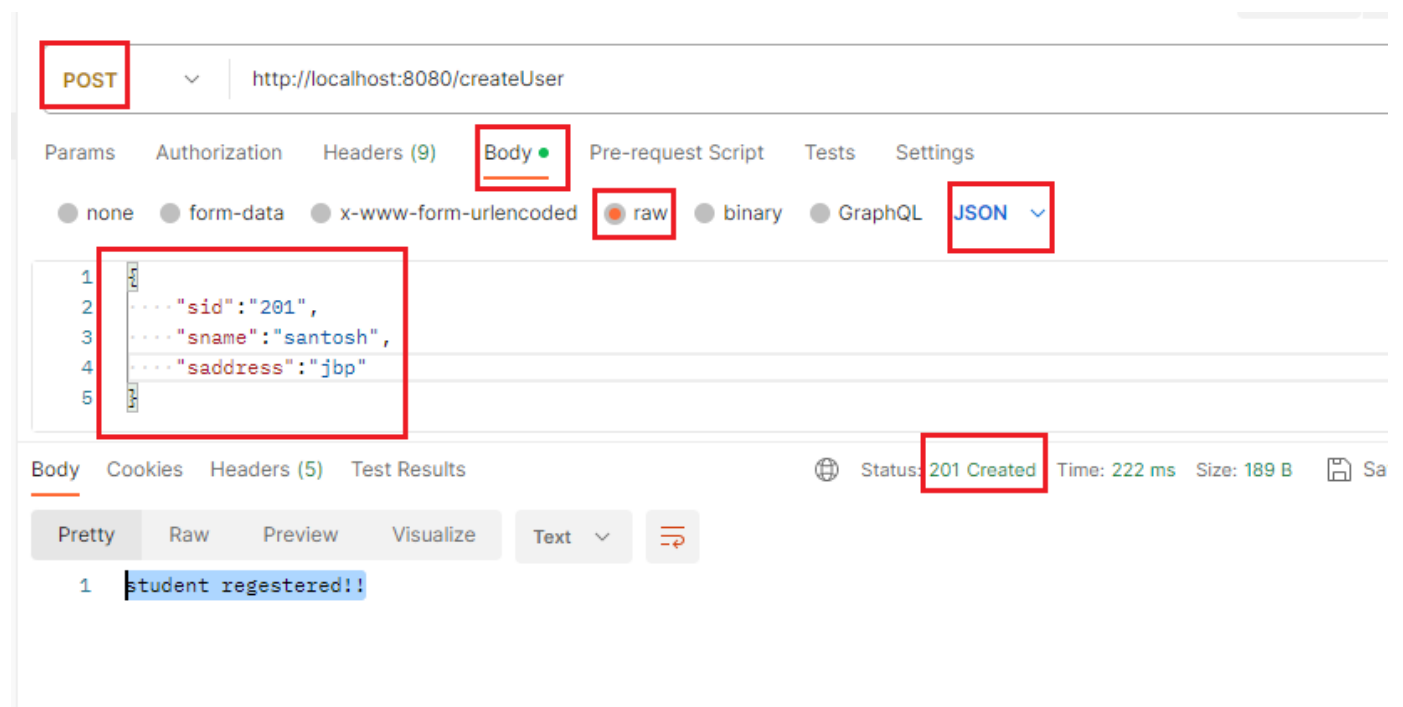
@RequestBody annotation . this enabling automatic deserialization of the inbound HttpRequest body onto a Java object. Spring automatically deserializes the JSON into a Java type,

@RequestBody deserialization of the inbound
HttpRequest body onto a Java object

```
@RestController
public class MyController {

    @PostMapping("/createUser")
    public ResponseEntity<String> createStudent(@RequestBody Student stu) {

        System.out.println(stu.getSid() + " " + stu.getSname() + " " + stu.getSaddress());
        return new ResponseEntity<String>("student regestered!!", HttpStatus.CREATED);
    }
}
```



Task:- develop full Rest API using controller, interface with loosely coupled diagram with mysql database.

In Realtime, Producer Application Development team will provide documentation to Client Side Development in 2 ways

- Swagger
- PDF document

Swagger

->Swagger is third party api

->Swagger is used to generate documentation for Rest API.

->Swagger provides attractive UI to test REST API.

->To use Swagger in our project we need to add some dependencies to our project.

Swagger implementation for spring 2.xx version.

For 2.xx version add these dependencies.

```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.6.1</version>
</dependency>
```

```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.6.1</version>
</dependency>
```

-> After adding dependency, we need to create SwaggerConfig class. In this class we will specify for which classes it has to generate documentation.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.5.2</version>
    <relativePath /> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.demo</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>indianrailway-1</name>
  <description>Demo project for Spring Boot</description>
  <properties>
    <java.version>11</java.version>
  </properties>
  <dependencies>
```

```
    <dependency>
      <groupId>io.springfox</groupId>
      <artifactId>springfox-swagger2</artifactId>
      <version>2.6.1</version>
    </dependency>
    <dependency>
      <groupId>io.springfox</groupId>
      <artifactId>springfox-swagger-ui</artifactId>
      <version>2.6.1</version>
    </dependency>
```

```

@RestController
public class MyController {

    @GetMapping("/")
    public String welcome()
    {
        return "welcome to codeminestech";
    }

    @GetMapping("/allusers")
    public ResponseEntity<Map<String,String>> getUserData()
    {
        Map<String ,String> maps=new HashMap<>();
        maps.put("name","himanshu");
        maps.put("userid","101");
        maps.put("address","jbp");

        return new ResponseEntity<Map<String,String>>(maps,HttpStatus.OK);
    }

    @PostMapping("/add")
    public ResponseEntity<Map<String,String>> addUser(@RequestBody HashMap<String, String> map)
    {
        return new ResponseEntity<Map<String,String>>(map,HttpStatus.OK);
    }

}

}

@Configuration
@EnableSwagger2
public class SwaggerConfig {

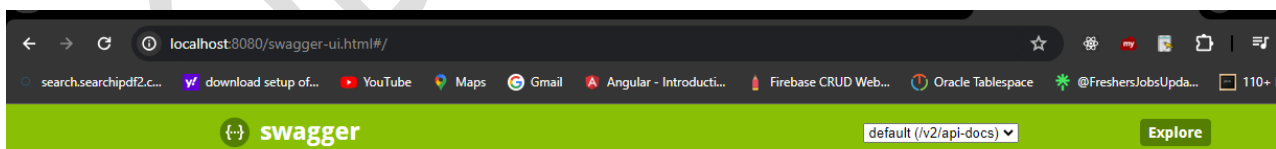
    @Bean
    public Docket apiDoc()
    {

        return new Docket(DocumentationType.SWAGGER_2)
            .select()
            .apis(RequestHandlerSelectors.basePackage("com.irctcdemo.controller"))
            .paths(PathSelectors.any())
            .build();

    }

}

```



Api Documentation

Api Documentation

[Apache 2.0](#)

my-controller : My Controller

Show/Hide | List Operations | Expand Operations

GET	/	welcome
POST	/add	addUser
GET	/allusers	getUserData

[BASE URL: / , API VERSION: 1.0]

GET / welcome

Response Class (Status 200)
string

Response Content Type

Response Messages

HTTP Status Code	Reason	Response Model	Headers
401	Unauthorized		
403	Forbidden		
404	Not Found		

Try it out!

In latest springboot 3.xx swagger

->in this we need to use OPEN-API

```
<dependency>  
<groupId>org.springdoc</groupId>  
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
<version>2.0.3</version>  
</dependency>
```

```
<dependency>  
<groupId>org.springdoc</groupId>  
<artifactId>springdoc-openapi-ui</artifactId>  
<version>1.6.15</version>  
</dependency>
```

Just add this dependency ,here no need to create object class of Docket and all just add this dependency and access URL

URL:- **localhost:8080/swagger-ui.html**

As we know earlier that we can create distributed application for B2B . In which one application which provide services is producer application . and another application which consumes that services is called consumer application.

-> The application which is accessing REST API is called as **REST CLIENT**.

-> In Spring Boot we can develop REST Client in 2 ways

- RestTemplate (Synchronus client) - Outdated
- WebClient (Sync + Async client) – Trending

-> **RestTemplate** is a predefined class, it is part of "spring-boot-starter-web" dependency. It is used to create synchronous client .

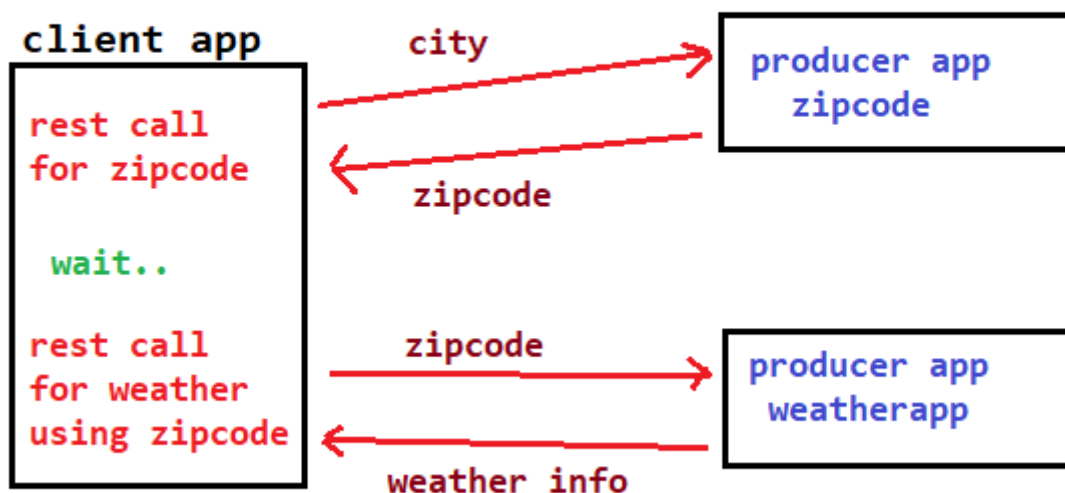
-> **WebClient** is a interface, it is part of "spring-boot-starter-webflux" dependency.

Note:-using webclient we can make synchronous and asynchronous client.

Synchrnonus

->after sending a request it will wait for the response to proceed further.

Synchronus client



Usecase of synchronus client:- see above diagram in this our client app is developed to show weather info on the basis of zip code.

->so we have one another app which provide zipcode on the bases of city name and we have one another app which provide weather info on the basis of zipcode.

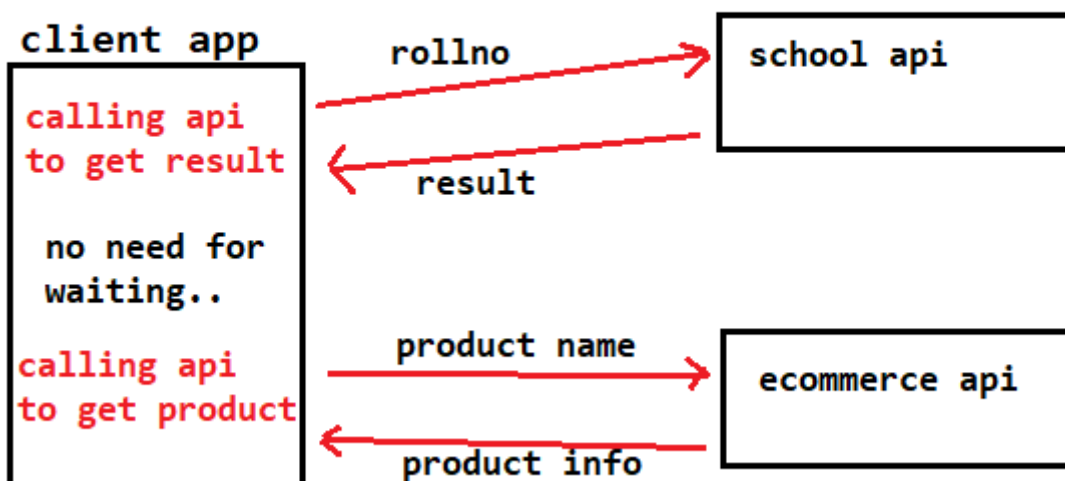
->so we can use here synchronus kind of thing because until we don't have zipcode no use to hit weather info provider api.

->so after sending the request for zipcode wait for response until response not come execution will not move further.

asynchronous

program to handle multiple tasks simultaneously rather than executing them one after the other. Means execution will not wait for response of other task .

ASynchronus client



->here no api are related then why to wait for response of one api . so in this case we can go with asynchronous.

Ex:- RESTTEMPLATE

```
@SpringBootApplication
public class ResttemplatedemoApplication {

    @SuppressWarnings("deprecation")
    public static void main(String[] args) {
        SpringApplication.run(ResttemplatedemoApplication.class, args);

        String endpoint="https://catfact.ninja/fact";

        RestTemplate rt=new RestTemplate();

        ResponseEntity<String> rs = rt.getForEntity(endpoint,String.class);

        int statusCode = rs.getStatusCodeValue();

        if(statuscode==200)
        {
            String body = rs.getBody();
            System.out.println(body);
        }

    }

}
```

Annotations and variables in the code are color-coded: `@SpringBootApplication` is blue, `public class` is red, `ResttemplatedemoApplication` is black, `{` is red, `@SuppressWarnings("deprecation")` is blue, `public static void` is red, `main` is black, `(String[] args)` is black, `{` is red, `SpringApplication.run` is blue, `(ResttemplatedemoApplication.class, args);` is black, `String` is blue, `endpoint` is black, `"https://catfact.ninja/fact";` is black, `RestTemplate` is blue, `rt=new` is red, `RestTemplate()` is black, `ResponseEntity<String>` is blue, `rs =` is red, `rt.getForEntity` is blue, `(endpoint,String.class);` is black, `int` is blue, `statusCode` is black, `=` is red, `rs.getStatusCodeValue();` is black, `if` is red, `(statuscode==200)` is black, `{` is red, `String` is blue, `body` is black, `=` is red, `rs.getBody();` is black, `System.out.println` is blue, `(body);` is black, `}` is red, `}` is red.

Annotations and variables in the code are color-coded: `@SpringBootApplication` is blue, `public class` is red, `ResttemplatedemoApplication` is black, `{` is red, `@SuppressWarnings("deprecation")` is blue, `public static void` is red, `main` is black, `(String[] args)` is black, `{` is red, `SpringApplication.run` is blue, `(ResttemplatedemoApplication.class, args);` is black, `String` is blue, `endpoint` is black, `"https://catfact.ninja/fact";` is black, `RestTemplate` is blue, `rt=new` is red, `RestTemplate()` is black, `ResponseEntity<String>` is blue, `rs =` is red, `rt.getForEntity` is blue, `(endpoint,String.class);` is black, `int` is blue, `statusCode` is black, `=` is red, `rs.getStatusCodeValue();` is black, `if` is red, `(statuscode==200)` is black, `{` is red, `String` is blue, `body` is black, `=` is red, `rs.getBody();` is black, `System.out.println` is blue, `(body);` is black, `}` is red, `}` is red.

Demo application for asynchronus client (webclient).

->to create asynchronus client .we need to add one more dependency call 'spring reactive web'

Spring Boot Version: 3.1.6

Frequently Used:

☐ MySQL Driver	☐ Spring Boot DevTools	☐ Spring Data JPA
☐ Spring Security	☒ Spring Web	☐ Thymeleaf

Available: Selected:

web

▼ Messaging

☐ WebSocket

▼ Template Engines

☐ Thymeleaf

☐ Apache Freemarker

☐ Mustache

▼ Testing

☐ Testcontainers

▼ Web

☒ Spring Web

☒ Spring Reactive Web

☐ Spring Web Services

☐ Jersey

X Spring Web

X Spring Reactive Web

->we will try to communicate with our old app train app.

```
1 package com.webclient.entites;
2
3
4 public class Ticket {
5
6     private Integer pnr;
7     private String sourceStation;
8     private String destStation;
9     private String trainClass;
10
11     private Passanger passanger;
12
13 }
14
15
16 @Service
17 public class TicketService {
18
19     public Flux<Ticket> getTicket()
20     {
21
22         String url="http://localhost:8080/alltickets";
23
24         WebClient client=WebClient.create();
25         Flux<Ticket> tickets = client.get()
26             .uri(url)
27             .retrieve()
28             .bodyToFlux(Ticket.class);
29
30         return tickets;
31     }
32 }
33
34
35
36 @RestController
37 public class MyController {
38
39     @Autowired
40     private TicketService ticketService;
41
42     @GetMapping("/alltickets")
43     public ResponseEntity<Flux<Ticket>> allTickets()
44     {
45         Flux<Ticket> ticket = ticketService.getTicket();
46         return new ResponseEntity<Flux<Ticket>>(ticket,HttpStatus.OK);
47     }
48 }
49
50 }
```

Run this app in different port server.port=9999.

Because our train app is running at 8080.

```

@Service
public class TicketService {

    public void getTicket()
    {

        String url="http://localhost:8080/alltickets";

        WebClient client=WebClient.create();
        Flux<Ticket> tickets = client.get()
            .uri(url)
            .retrieve()
            .bodyToFlux(Ticket.class);

        tickets.subscribe(t->TicketService.ticketCount());

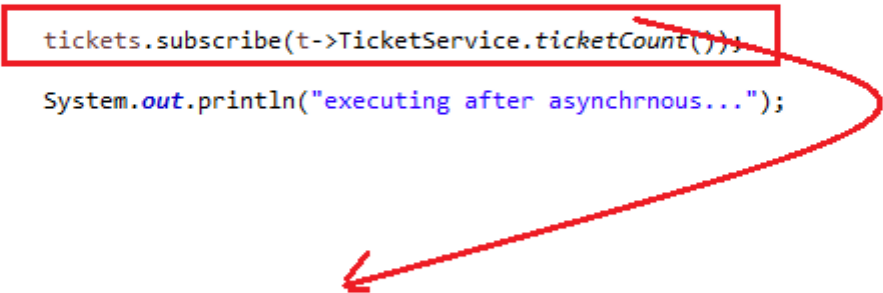
        System.out.println("executing after asynchronous...");

    }

    public static void ticketCount()
    {
        System.out.println("response received");
    }

}

```



we can subscribe it and get response when we receive it from client. asynchronously.

So for handling asynchronous response we create handler method so whenever response comes we execute this method. Our execution will move forward it will not wait ..

Exception Handling

- > Exception means an unexpected and unwanted situation occurring during the program execution is called as Exception as we already discussed in core java.
- > when exception occurs then program will be terminated abnormally (stops in middle).
- > as a developer we need to take care of that our program must terminate normally instead of abnormally.
- > as we already know in java we have 5 keywords to handle exception
 - try

- catch
- throw
- throws
- final

->as we know code which have chance of exception we put inside try block.

->catch is used to handle that exception.

->throw keyword is used to re-throw the exception.

->throws keyword is used to ignore the exception.

->final block is used to write clean up activities (closing files, closing connection..etc).

Exception handling in REST API

->In Rest API we can handle Exception handle in two ways.

- Controller Based Handling.
- Global Exception Handling.

->Controller Based Handling means writing Exception Handler methods in Controller class. That exception handler works only for the controller class exception .

->To handle exception of any class in project level then we go for Global Exception Handling. We use **@RestControllerAdvice** annotation for Global Exception handling.

->we use **@ExceptionHandler** annotation over handler method either in controller based or global based.

Controlle based exception handling ex

```

private TicketService ticketService;

@GetMapping("/allpassanger")
public ResponseEntity<List<Passanger>> allPassanger()
{
    passangerService=null;
    List<Passanger> allPassanger = passangerService.allPassanger();
    return new ResponseEntity<>(allPassanger,HttpStatus.OK);
}

@GetMapping("/alltickets")
public ResponseEntity<List<Ticket>> allTickets()
{
    int a=11/0; // creating exception

    List<Ticket> alltickets = ticketService.alltickets();
    return new ResponseEntity<List<Ticket>>(alltickets,HttpStatus.OK);
}

@ExceptionHandler(value= {ArithmeticException.class,IOException.class,ClassCastException.class})
public ResponseEntity<ExceptionMessage> apiExcept(Exception e)
{
    String expMsg = e.getMessage();
    String expCode="e101";

    ExceptionMessage em=new ExceptionMessage();
    em.setErrorMsg(expMsg);
    em.setErrorCode(expCode);

    return new ResponseEntity<ExceptionMessage>(em,HttpStatus.INTERNAL_SERVER_ERROR);
}

@ExceptionHandler(value= {NullPointerException.class})
public ResponseEntity<ExceptionMessage> nullExcept(Exception e)
{
    String expMsg = e.getMessage();
    String expCode="null101";

    ExceptionMessage em=new ExceptionMessage();
    em.setErrorMsg(expMsg);
}

```

This is controller based exception handling in which we create handler method on every controller separately.

as exception occurs it is handled by that specific handler method.

->Global Exception handling Ex

->in Controller based handling we create handler methods but that handler method is specific upto that handler only means if we have 5 controllers then separately we need to maintain handler method in each controller .code redundancy may occur . suppose for NullPointerException we create separate handler in first controller as well as in second controller separate .Our code will work fine but handler method duplication occur.

TO overcome that limitation we can use GLOBAL EXCEPTION HANDLING.

->in Global handling technique we create separate class which is dedicated for handling Exception occurs in any where in our application. Inside this class only we define different-different handler method based on exception.

->we need to use **@RestControllerAdvice** annotation over this global class.

Global rest controller

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(value = {ArithmeticException.class})
    public ResponseEntity<ExceptionMessage> arthmaticExceptHand(ArithmeticException e)
    {

        String msg = e.getMessage();
        String code="arthi1111";

        ExceptionMessage em=new ExceptionMessage();
        em.setErrorMsg(msg);
        em.setErrorCode(code);

        return new ResponseEntity<ExceptionMessage>(em,HttpStatus.INTERNAL_SERVER_ERROR);

    }

    @ExceptionHandler(value = {NullPointerException.class})
    public ResponseEntity<ExceptionMessage> myNullExcept(NullPointerException e)
    {

        String msg = e.getMessage();
        String code="null222";

        ExceptionMessage em=new ExceptionMessage();
        em.setErrorMsg(msg);
        em.setErrorCode(code);

        return new ResponseEntity<ExceptionMessage>(em,HttpStatus.INTERNAL_SERVER_ERROR);

    }
}

```

```

@RestController
public class TrainController {

    @Autowired
    private PassangerService passangerService;

    @Autowired
    private TicketService ticketService;

    @PostMapping("/create")
    public ResponseEntity<Passanger> createPassanger(@RequestBody Passanger passanger) {
        passanger.getTickets().forEach(d -> {
            d.setPassanger(passanger);
        });

        Passanger pascreated = passangerService.createPass(passanger);

        return new ResponseEntity<Passanger>(pascreated, HttpStatus.CREATED);

    }

    @GetMapping("/allpassanger")
    public ResponseEntity<List<Passanger>> allPassanger() {
        passangerService = null;
        List<Passanger> allPassanger = passangerService.allPassanger();
        return new ResponseEntity<>(allPassanger, HttpStatus.OK);
    }

    @GetMapping("/alltickets")
    public ResponseEntity<List<Ticket>> allTickets() {
        int a = 11 / 0; // creating exception

        List<Ticket> alltickets = ticketService.alltickets();
        return new ResponseEntitv<List<Ticket>>(alltickets, HttpStatus.OK);
    }
}

```

*****MICROSERVICES*****

Before understanding microservices lets understand about monolith.

What is Monolith Architecture?

->The term 'monolith' is used to refer to applications that are designed as a single unit. Which means all the functionality is bundled together .all the logic like UI logic, business logic , DB logic ..etc are bundled as a single unit.

->We package our application as jar/war to deploy into server .

->as monolith application contains all the functionality it will become heavy jar/war .

Advantage of Monolith

->simple to develop

-> Everything available at one place.

->easy to understand.

->less configuration.

->less resources required (ex:- man power,infra)

Dis-Advantage of Monolithic.

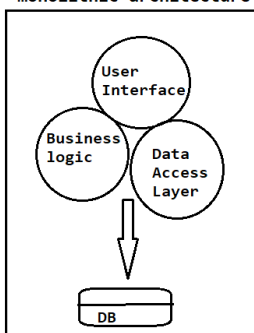
->Difficult to maintain.

->single point of failure:-if any error occur in project then entire project may get failed /down.

->less Scalabiltiy:-because monolithic architecture software is tightly coupled, it is hard to scale.

->dependencies between functionalites because of tightly couple.

monolithic architecture



To overcome the problems of Monolith, **Microservices architecture** came into market.

->Microservice is not a programming language.

->Microservice is not a framework.

->Microservice is an architectural design pattern.

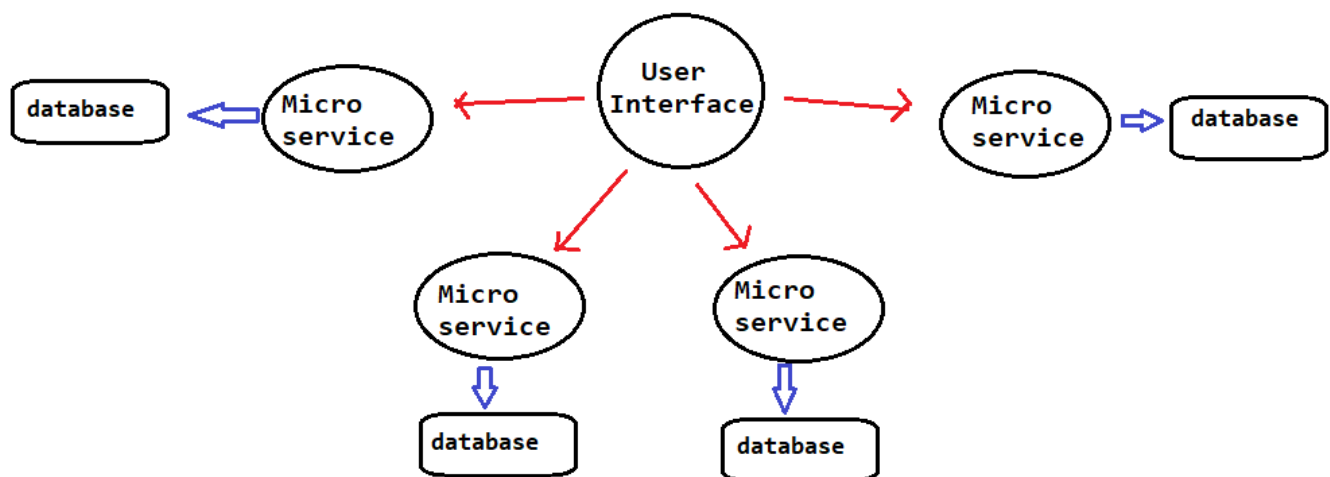
->Microservices design pattern help to develop application functionality with loosely coupling.

->In Microservices architecture we don't develop all the functionalities in single project. We divide project functionalities into several REST API's.

->which means one REST API project is One Microservice.

->for every major functionalities we create one microservice which fulfill that functionality.

->Microservices is not related to Only Java. Microservice pattern can be followed by different project development languages.



Advantages:-

->Easy to maintain because clear and cut separate code base.

->Faster development. Different-different microservices are developed by team members simultanesouly.

->Technology Independent:- different microservice can build on different programming languages and then they can communicate with each other. There is no dependency over languages.

->high Scalability:-each microservice can be scaled independently without downtime.

->No –single point failure:- a failure in one microservice does not affect other microservice. This is also called Resiliency.

->independent component can be tested independently.

Disadvantages of Microservices

->resource consuming and need high man-power and infra.

->complicated deployment:-Once you have developed each microservices, then they need to be integrated and deployed is critical task.

->high budget required .business wise not recommended for small scale projects .

*******Microservices Architecture*******

-> We don't have any fixed architecture for Microservices

-> People are customizing microservices architecture according to their requirement

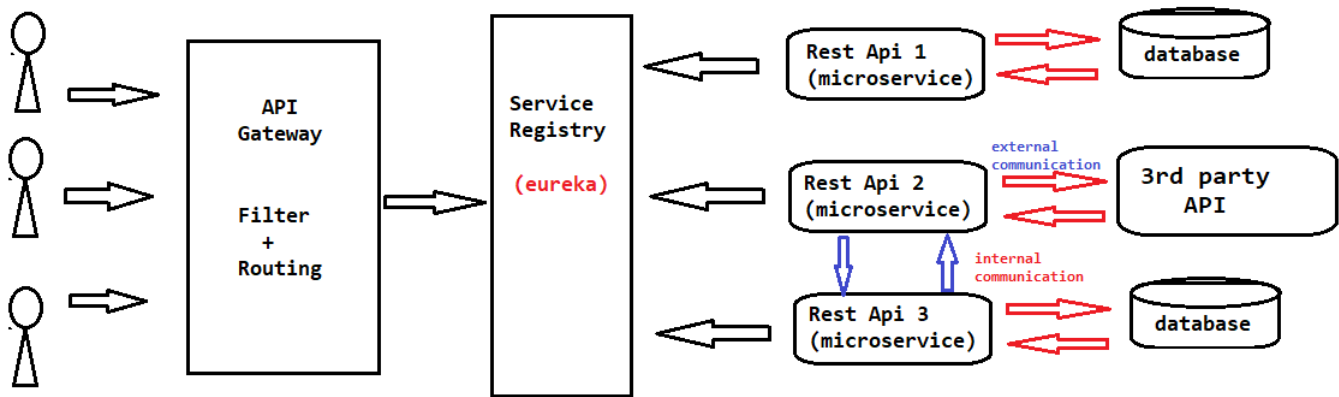
-> Most of the projects will use below components in Microservices Architecture..

1) Service Registry (Eureka Server)

2) Services (REST APIs)

3) Interservice Communication (FeginClient)

4) API Gateway (Zuul Proxy)



Service Registry

- >Service Registry acts as database of services available in the project.
- >it provides the details of all the services (microservices) which are registered with Service registry.
- >we can easily identified how many services available in the project.
- >we can identify how many instances available for each service.
- >we can use “**Eureka Server**” as service registry.
- >Eureka server is provided by “**Spring Cloud Netflix**” library.

Services

- >Services means REST API/Microservice.
- >service contain business logic.
- >In project ,some services will interact with 3rd party rest API.

(if one service communicate with any 3rd party service then we call it external service communication . we perform it using by **RestTemplate** , **WebClient**).

->In the project, some services will interact with another service within the project then we call it inter-service communication. To perform it we use **feign-client**.

->To distribute the load. We can run one service with multiple instances.

Note:-we will register every service with service registry.

API Gateway

->API Gateway is used to manage our backend api's of the project.

->API Gateway acts as mediator between end users and backend api's.

->API Gateway can filter logic to decide request processing.

->API gateway will contain Routing logic (which request should go to which rest api's).

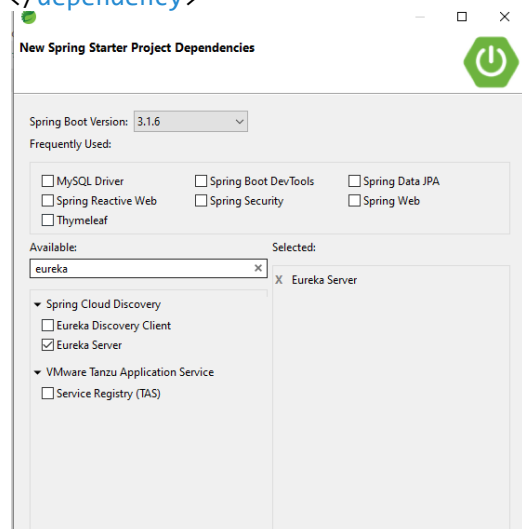
->API gateway also registered with service registry.

Small example:-

Create one service registry app

->just create one small app and add eureka server dependency.

```
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  
</dependency>
```



Configure property file

```
1 server:
2   port: 8761
3 eureka:
4   client:
5     register-with-eureka: false
6
```

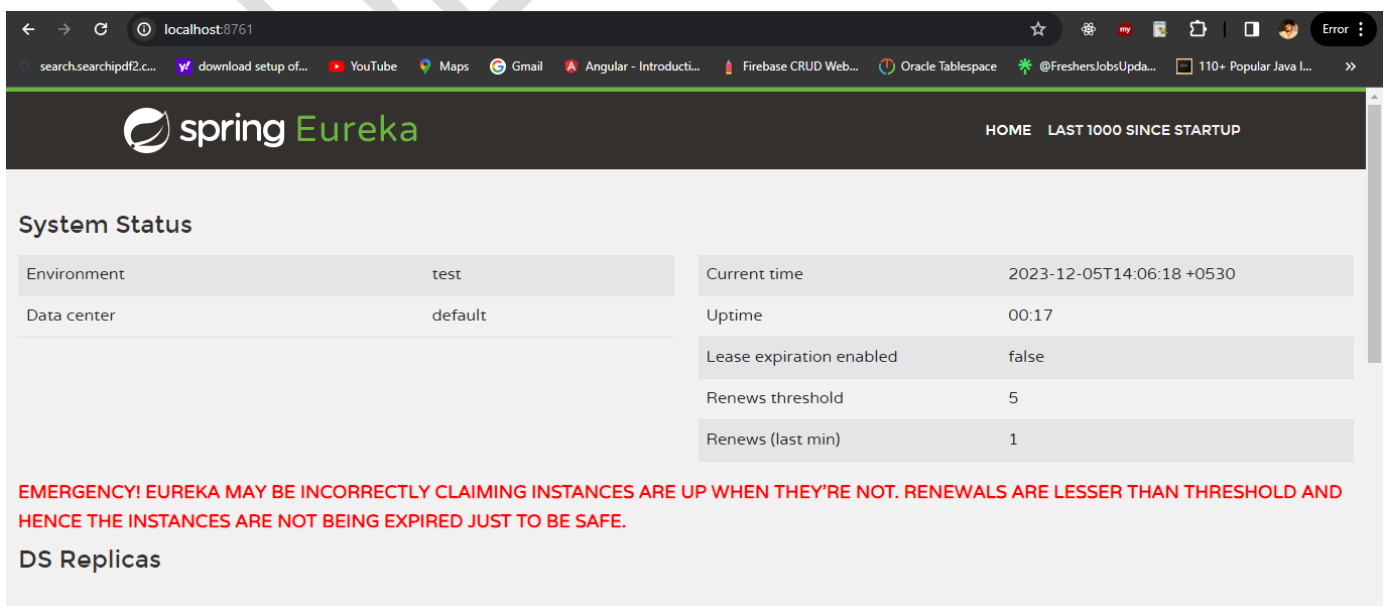
Note:- by default port for eureka is 8761. Which means all the eureka client will automatically detects this service registry and register .

```
1 package com.serv;
2
3 import org.springframework.boot.SpringApplication;
4
5
6
7 @SpringBootApplication
8 @EnableEurekaServer
9 public class ServiceregistryApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(ServiceregistryApplication.class, args);
13     }
14
15 }
16
```

Note:- in starter class add **@EnableEurekaServer** annotation.

Now just access this eureka dashboard. As off now no client service is registered in it we need to configure that services and run them also .

Hit:- <http://localhost:8761>



The screenshot shows the Spring Eureka dashboard. The header includes the Spring Eureka logo and navigation links for HOME and LAST 1000 SINCE STARTUP. The main content area is titled 'System Status' and contains two tables. The left table shows 'Environment' as 'test' and 'Data center' as 'default'. The right table shows 'Current time' as '2023-12-05T14:06:18 +0530', 'Uptime' as '00:17', 'Lease expiration enabled' as 'false', 'Renews threshold' as '5', and 'Renews (last min)' as '1'. Below these tables, a red warning message states: 'EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.' At the bottom, there is a section for 'DS Replicas'.

System Status	
Environment	test
Data center	default
Current time	2023-12-05T14:06:18 +0530
Uptime	00:17
Lease expiration enabled	false
Renews threshold	5
Renews (last min)	1

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

Create services which need to register with eureka service registry.

->we need to add **eureka discovery client** dependency on client.

-> we need to use **@EnableDiscoveryClient** annotation in starter class.

->configure properties.xml file.

NOTE:- in microservice pattern give every service port different because by default port is 8080.

NOTE:- also give different service application name in properties file.

Service 1:

```
1 |
2 | spring:
3 |   datasource:
4 |     username: root
5 |     password: root
6 |     url: jdbc:mysql://localhost:3306/railway
7 |   jpa:
8 |     hibernate:
9 |       ddl-auto: update
10 | application:
11 |   name: irctc1
```

```
1 | package com.irctcdemo;
2 |
3 | import org.springframework.boot.SpringApplication;
4 |
5 | @SpringBootApplication
6 | @EnableDiscoveryClient
7 | public class IndianrailwayApplication {
8 |
9 |     public static void main(String[] args) {
10 |         SpringApplication.run(IndianrailwayApplication.class, args);
11 |     }
12 | }
13 |
14 |
15 |
16 |
```

Service 2

```
1 | spring.application.name=irctc2
2 | server.port=8888
```

```

import org.springframework.boot.SpringApplication;
8
9 @SpringBootApplication
0 @EnableDiscoveryClient
1 public class ResttemplatedemoApplication {
2
3     @SuppressWarnings("deprecation")
4     public static void main(String[] args) {
5         SpringApplication.run(ResttemplatedemoApplication.class, args);
6
7         String endpoint="https://catfact.ninja/fact";
8
9         RestTemplate rt=new RestTemplate();
0         ResponseEntity<String> rs = rt.getForEntity(endpoint,String.class);
1
2         int statusCode = rs.getStatusCodeValue();
3
4         if(statuscode==200)
5         {
6             String body = rs.getBody();
7             System.out.println(body);
8         }
9
0

```

What if we change eurka service registry port other then 8761 ?

->suppose now our eureka is running on port 9090 then. In this case our services will unable to automatically regestired it self then we need to add these config to properties files in every eureka client service.

eureka:

client:

serviceUrl:

defaultZone: http://localhost:9090/eureka

```

1 server:
2   port: 8888
3 spring:
4   application:
5     name: irctc2
6
7 eureka:
8   client:
9     serviceUrl:
10       defaultZone: http://localhost:9090/eureka
11

```

Microservices inter-communication (feign client)

To see its example we need to create

- 1.service registry project (because feign client use it)
- 2.more then one microservice to achieve inter –communication.

Service registry

```
Mycontroller.java  MyLyric.java  SongController.java  ServiceregistryApplication.java >
1 package com.serv;
2
3 import org.springframework.boot.SpringApplication;
4
5
6
7 @SpringBootApplication
8 @EnableEurekaServer
9 public class ServiceregistryApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(ServiceregistryApplication.class, args);
13     }
14 }
15
16
```

Song data app(service 1)

->this is normal service one

```
1 package com.lyrics.demo.controller;
2
3 import java.util.ArrayList;
4
5
6
7 @RestController
8 public class SongController {
9
10     @GetMapping("/lyrics")
11     public ResponseEntity<Lyrics> getLyrics(@RequestParam("singer") String singer) {
12         System.out.println("reached boss "+singer);
13         Lyrics ly = null;
14         if (singer.equals("neha")) {
15             ly = new Lyrics();
16             ly.setLyricId(101);
17             ly.setSingerName("neha kakar");
18             List<String> songs = new ArrayList<>();
19             songs.add("Gaadi Kaali");
20             songs.add("kala chasma");
21             songs.add("o saki saki");
22             songs.add("gali gali");
23             ly.setLyrics(songs);
24         } else if (singer.equals("atif")) {
25             ly = new Lyrics();
26             ly.setLyricId(101);
27             ly.setSingerName("atif");
28             List<String> songs = new ArrayList<>();
29             songs.add("tere sang yara");
30             songs.add("jeene laga hu");
31             songs.add("khai mangda");
32             ly.add("dil diya galyan");
33             ly.setLyrics(songs);
34         }
35
36         return new ResponseEntity<Lyrics>(ly, HttpStatus.OK);
37     }
38 }
39
40
41
42
43
44
45
46
47
48
49
50
```

Service 2 (this service inter-communicate with service one)

```
1 package com.lyrics.demo;
2
3 import org.springframework.boot.SpringApplication;
4
5
6
7 @SpringBootApplication
8 @EnableFeignClients
9 public class SonglyricsApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(SonglyricsApplication.class, args);
13     }
14 }
15
16
```

```

10 </parent>
11 <groupId>com.demo</groupId>
12 <artifactId>demo</artifactId>
13 <version>0.0.1-SNAPSHOT</version>
14 <name>songlyrics</name>
15 <description>Demo project for Spring Boot</description>
16 <properties>
17   <java.version>11</java.version>
18   <spring-cloud.version>2022.0.4</spring-cloud.version>
19 </properties>
20 <dependencies>
21   <dependency>
22     <groupId>org.springframework.boot</groupId>
23     <artifactId>spring-boot-starter-web</artifactId>
24   </dependency>
25   <dependency>
26     <groupId>org.springframework.cloud</groupId>
27     <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
28   </dependency>
29   <dependency>
30     <groupId>org.springframework.cloud</groupId>
31     <artifactId>spring-cloud-starter-openfeign</artifactId>
32   </dependency>
33   <dependency>
34     <groupId>org.springframework.boot</groupId>
35     <artifactId>spring-boot-devtools</artifactId>
36     <scope>runtime</scope>
37     <optional>true</optional>
38   </dependency>
39 </dependencies>

```

```

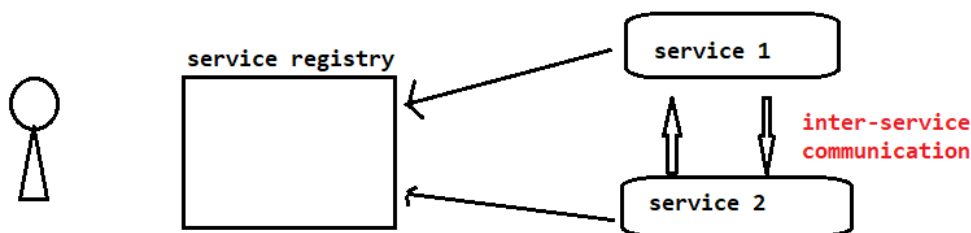
Mycontroller.java  MyLyric.java  SongController.java  ServiceregistryApplicati...  Sor
1 package com.lyrics.demo.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4
5 @RestController
6 public class Mycontroller {
7
8   @Autowired
9   private MyLyric myLyric;
10
11   @GetMapping("/getlyrics")
12   public ResponseEntity<SongLyrics> getSongs(@RequestParam("singer") String name)
13   {
14     System.out.println("hahahah");
15     SongLyrics data = myLyric.getData(name);
16     System.out.println("----end");
17     return new ResponseEntity<SongLyrics>(data, HttpStatus.OK);
18   }
19
20 }

```

```

1 package com.lyrics.demo.service;
2
3 import org.springframework.cloud.openfeign.FeignClient;
4
5 @FeignClient(name = "hungama")
6 public interface MyLyric {
7
8   @GetMapping("/lyrics")
9   public SongLyrics getData(@RequestParam("singer") String sname);
10 }

```



Q)why we use feign-client , if we have RestTemplate and WebClient for communication ?

->in these both we need to defined endpoint in hardcode way. What ever end-point we define it hit that api and communicate. Which is fine . but to achieve load balancing these both will not full fill requirement.

->in load balancing one api is running on multiple instances. So feign-client is designed in such a way that it pick api name from service registry and internally it perform load-balancing if one api is running in multiple instances. It using round-robin approach.

->eureka ,feign-client all these are cloud based concept.

*****API Gateway*****

->as we discussed API Gateway will act as a mediator between client request and backend.

->Api gateway will provide single entry point to access our backend api.

->we need to register this gateway to service registry.

-> In Api Gateway we will write mainley below 2 types of logics

1) Filters

2) Routing

Filters:-Filters are used to execute some logic before request processing and after request processing.

Routing:-routing is used to configure which request should go to which rest-api.

-> In Spring Cloud we have 2 options to create API Gateway

- **Zuul Proxy** (older version not supported by latest spring boot v)
- **Cloud Gateway** (latest)

API –Gateway example using cloud gateway.

1)create one api gateway service.

-> web-stater

-> eureka-client

-> cloud-gateway

-> devtools

```
6 <groupId>org.springframework.boot</groupId>
7 <artifactId>spring-boot-starter-parent</artifactId>
8 <version>3.1.6</version>
9 <relativePath/> <!-- lookup parent from repository -->
10 </parent>
11 <groupId>com.demo</groupId>
12 <artifactId>demo</artifactId>
13 <version>0.0.1-SNAPSHOT</version>
14 <name>apigateway</name>
15 <description>Demo project for Spring Boot</description>
16 <properties>
17 <java.version>11</java.version>
18 <spring-cloud.version>2022.0.4</spring-cloud.version>
19 </properties>
20 <dependencies>
21 <dependency>
22 <groupId>org.springframework.boot</groupId>
23 <artifactId>spring-boot-starter-webflux</artifactId>
24 </dependency>
25 <dependency>
26 <groupId>org.springframework.cloud</groupId>
27 <artifactId>spring-cloud-starter-gateway</artifactId>
28 </dependency>
29 <dependency>
30 <groupId>org.springframework.cloud</groupId>
31 <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
32 </dependency>
33 <dependency>
34 <groupId>org.springframework.boot</groupId>
35 <artifactId>spring-boot-devtools</artifactId>
36 <scope>runtime</scope>
37 <optional>true</optional>
38 </dependency>
```

```
1 package com.gateway;
2
3 import org.springframework.boot.SpringApplication;
4
5
6
7 @SpringBootApplication
8 @EnableDiscoveryClient
9 public class ApigatewayApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(ApigatewayApplication.class, args);
13     }
14
15 }
16
```

```
server:
  port: 9090
spring:
  application:
    name: gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
      routes:
        - id: irctcl
          uri: http://localhost:8080
          predicates:
            - Path=/allpassanger/**
        - id: irctcl
          uri: http://localhost:8080
          predicates:
            - Path=/create/**
```

id: You can give any id as of now.
uri: Here you have to provide the port in which your microservice is running
predicates (- Path): Here we have provided the path "/create/**", which means any request starting with path "/create/**" rout it to irctcl.

```

server:
  port: 9090
spring:
  application:
    name: gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
      routes:
        - id: irctc1
          uri: http://localhost:8080
          predicates:
            - Path=/allpassanger/**
        - id: irctc2
          uri: lb://irctc1 ➡ from service
          predicates:          registry it takes
            - Path=/create/** service name and hit

```

```

@Component
public class MyPreFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {

        //accessing http request information.
        ServerHttpRequest request = exchange.getRequest();
        HttpHeaders headers = request.getHeaders();
        Set<String> keySet = headers.keySet();

        keySet.forEach(k->{
            List<String> list = headers.get(k);
            System.out.println(k+" "+list);
        });
        return chain.filter(exchange);
    }
}

```

from exchange we can get every information regarding request

chain.filter(exchange):-we can pass exchange to another filters..as per requirement project can have multiple filters.

-> We can validate client given token in the request using Filter for security purpose

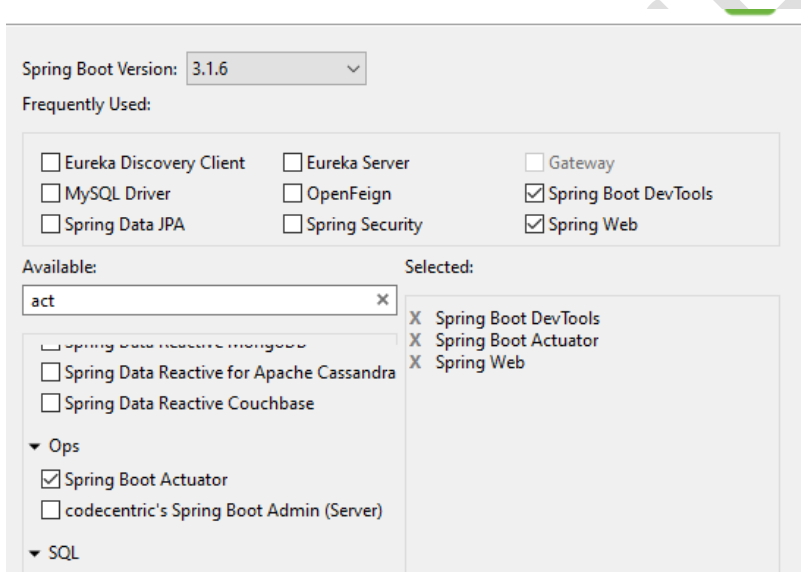
-> We can write request and response tracking logic in Filter

-> Filters are used to manipulate request & response of our application

-> Any cross-cutting logics like security, logging, monitoring can be implemented using Filters.

Actuator

- >Actuators will provide various information relation to our web api like
- >it provide health of our application .
- >how many beans loaded by our application.
- >what config props loaded by our application.
- >what is heap info of our application.
- >how many threads runnings in our application.
- >how many URL mappings available in our application.
- >To Configure actuators in our applications spring boot provided below starter
- >**Note:-** by default actuator exposes only health endpoint . to expose all endpoint we need to configure one property in properties file.



- >config these property in properties file.

management.endpoints.web.exposure.include=*

start '*' expose all endpoints but if you want to exclude some endpoint then use

management.endpoints.web.exposure.exclude=bean,mappings

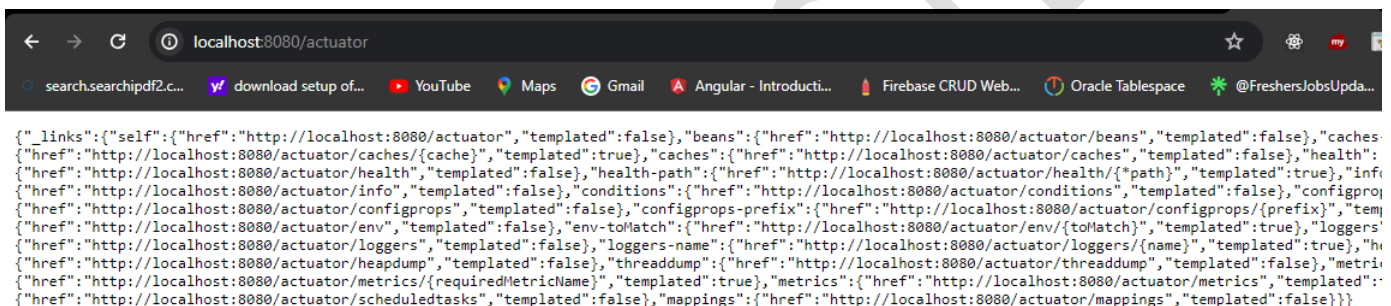
- >make controller (I have create just simple controller to demonstrate actuators).

```

MyController.java x application.properties
1 package com.actuator.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class MyController {
8
9     @GetMapping("/greet")
10    public String welcomeMsg()
11    {
12        return "welcome to codeminestech";
13    }
14
15 }
16
17

```

->run project and visit:- <http://localhost:8080/actuator>



```

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8080/actuator/beans",
      "templated": false
    },
    "caches": {
      "href": "http://localhost:8080/actuator/caches/{cache}",
      "templated": true
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    },
    "info": {
      "href": "http://localhost:8080/actuator/info",
      "templated": false
    },
    "conditions": {
      "href": "http://localhost:8080/actuator/conditions",
      "templated": false
    },
    "configprops": {
      "href": "http://localhost:8080/actuator/configprops",
      "templated": false
    },
    "configprops-prefix": {
      "href": "http://localhost:8080/actuator/configprops/{prefix}",
      "templated": true
    },
    "env-toMatch": {
      "href": "http://localhost:8080/actuator/env/{toMatch}",
      "templated": true
    },
    "loggers": {
      "href": "http://localhost:8080/actuator/loggers",
      "templated": false
    },
    "loggers-name": {
      "href": "http://localhost:8080/actuator/loggers/{name}",
      "templated": true
    },
    "heapdump": {
      "href": "http://localhost:8080/actuator/heapdump",
      "templated": false
    },
    "threaddump": {
      "href": "http://localhost:8080/actuator/threaddump",
      "templated": false
    },
    "metrics-requiredMetricName": {
      "href": "http://localhost:8080/actuator/metrics/{requiredMetricName}",
      "templated": true
    },
    "metrics": {
      "href": "http://localhost:8080/actuator/metrics",
      "templated": false
    },
    "scheduledtasks": {
      "href": "http://localhost:8080/actuator/scheduledtasks",
      "templated": false
    },
    "mappings": {
      "href": "http://localhost:8080/actuator/mappings",
      "templated": false
    }
  }
}

```

->you will get all end points now from here you can see any info regarding app.

->In Actuator we have a special endpoint called shutdown.

->This /shutdown endpoint is binded to HTTP POST request.

->using this endpoint we can stop our application.

->to use this we need to enable configuration in properties file.

```

management.endpoints.web.exposure.include=*
management.endpoints.web.exposure.exclude=beans,mappings

```

```

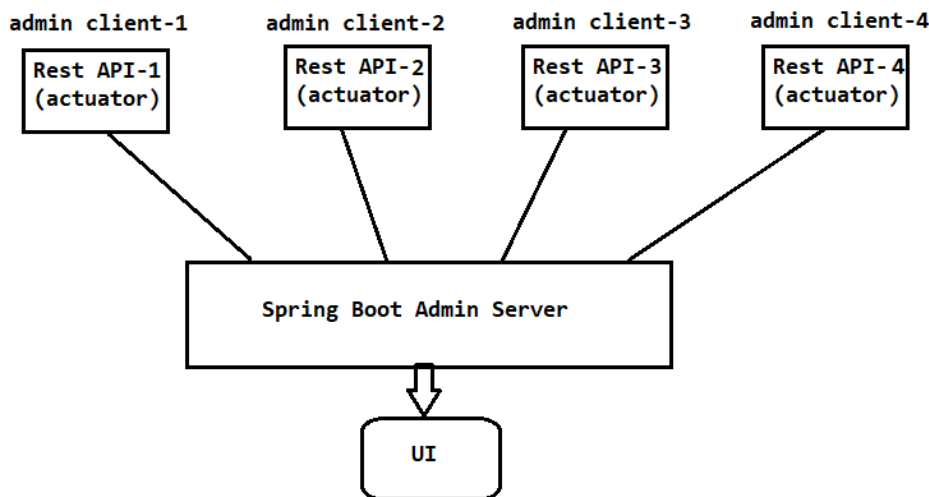
management.endpoint.shutdown.enabled=true

```

Note:- to fire <http://localhost:8080/actuator/shutdown> you need to make POST request not get because actuator all endpoints are get but this one is post.

Spring Boot Admin Server and Admin Client

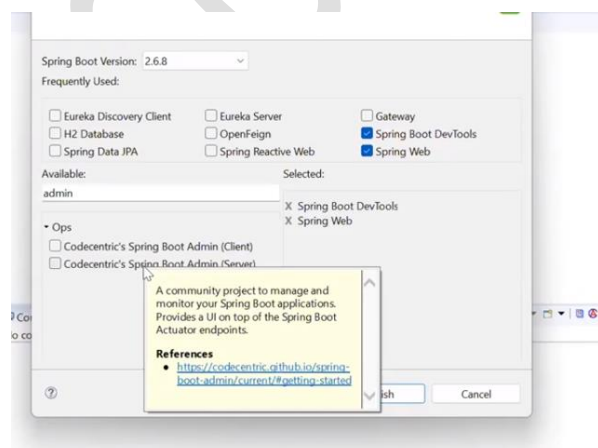
->spring boot admin is a web application. Used for managing and monitoring spring boot application. Here each application is considered as a client and register to admin server. Behind the scene magic is done by actuator.



Create project for admin service

Working with Admin-Server

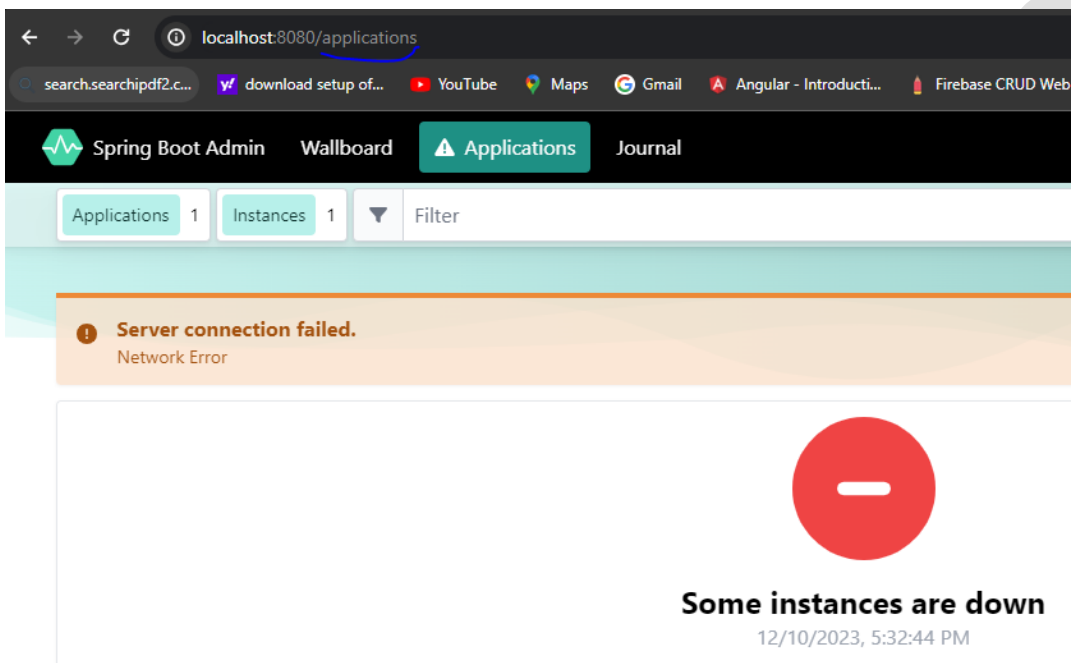
- 1) Create Boot application with admin-server dependency (select it while creating the project)
- 2) Configure **@EnableAdminServer** annotation at start class
- 3) Run the boot application
- 4) Access application URL in browser (We can see Admin Server UI)



```

AdminclientApplication.java  application.yml  MyController.java  Adminserverde
1 package com.adminserver;
2
3 import org.springframework.boot.SpringApplication;
4
5
6
7
8 @SpringBootApplication
9 @EnableAdminServer
10 public class AdminserverdemoApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(AdminserverdemoApplication.class, args);
14     }
15
16 }
17

```



Working with Admin-Client

1) Create Spring Boot application with below dependencies

- a) starter-web
- b) starter-actuator
- c) admin-starter-client
- d) devtools

2) Configure below properties in application.yml file

- a) server-port
- b) application-name
- c) enable-actuator-endpoints
- d) configure admin serve URL to reiger

```
1 server:
2   port: 1111
3 spring:
4   application:
5     name: client1
6 boot:
7   admin:
8     client:
9       url: http://localhost:8080/
10 management:
11   endpoints:
12     web:
13       exposure:
14         include: '*'
15
16
```

3) Create Rest Controller with required methods

4) Run the application (It will register in Admin Server)

Circuit Breaker

-> Circuit Breaker is a design pattern in Microservices

-> Circuit Breaker is used to implement fault-tolerance systems

-> Fault-tolerance systems are also called as resilience systems

-> Fault-tolerance system means when main logic is failed to execute then we should execute fallback logic to process client request.

Usecase example

get data from redis

if redis logic is failing then we should get data from database

Note: If redis logic is failing for 3 requests continuously then execute db logic for 30 mins. After 30 mins re-try for redis logic execution if it is working then execute redis logic only. If 3 re-try executions failed with redis then execute db logic for next 30 mins.

POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
  4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.6</version> //or use 2.5.4
    <relativePath /> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.demo</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>circuitapp</name>
  <description>Demo project for Spring Boot</description>
  <properties>
    <java.version>11</java.version>

    <spring.cloud.version>2020.0.3</spring.cloud.version>
    <spring-cloud-starter-netflix-hystrix.version>2.2.8.RELEASE</spring-cloud-starter-netflix-
    hystrix.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
      <version>${spring-cloud-starter-netflix-hystrix.version}</version>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <image>
            <builder>paketobuildpacks/builder-jammy-base:latest</builder>
          </image>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```

</image>
</configuration>
</plugin>
</plugins>
</build>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring.cloud-version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

</project>

```

Starter class

```

@SpringBootApplication
@EnableHystrix
public class CircuitappApplication {

    public static void main(String[] args) {
        SpringApplication.run(CircuitappApplication.class, args);
    }

}

```

Controller

```

@RestController
public class MyController {

    @GetMapping("/data")
    @HystrixCommand(fallbackMethod = "getdb", commandProperties = {
        @HystrixProperty(name = "circuitBreaker.requestVolumeThreshold", value = "3"),
        @HystrixProperty(name = "circuitBreaker.sleepWindowInMilliseconds", value = "10000"),
        @HystrixProperty(name = "circuitBreaker.enabled", value = "true") })
    public String getData() {
        System.out.println("getting data from redis.....");
        int nextInt = new Random().nextInt(10);
        if (nextInt <= 10) {
            System.out.println("----- " + nextInt);
            throw new RuntimeException("redis down..");
        }

        return "data from redis " + nextInt;
    }

    public String getdb() {
        System.out.println("getting data from database.....");
        return "data from db";
    }
}

```

Sleuth & Zipkin

-> Zipkin is a very efficient tool for **distributed tracing** in the microservices ecosystem. Distributed tracing, in general, is the **latency measurement of each component in a distributed transaction** where multiple microservices are invoked to serve a single business usecase.

Sleuth is another tool from the Spring cloud family. It is **used to generate the *trace id*, *span id* and add this information to the service calls in the headers and MDC**, so that It can be used by tools like Zipkin and ELK etc. to store, index and process log files.

-> If we add Sleuth dependency in REST API then it will add **span-id** and **trace-id** for log messages

-> **For every request once span-id will be generated by Sleuth**

-> **If one request is processing multiple REST API then Sleuth will use same span-id for REST APIs to generate log message.**

-> **Trace-id is specific to one REST API.**

-> By using span-id and trace-id we can understand which REST api has taken more time process request.

-> **To monitor span-id and trace-id details we will use ZipKin server.**

-> Zipkin server is providing user interface (UI) to monitor all the details.

Note: The REST APIs which are having sleuth dependency should register with Zipkin server.

Note: By using Sleuth and Zipkin we achieve Distributed Log Tracing.

Ex: In Spring 2.xx we use sleuth

Controller

```
@RestController
public class MyController {

    Logger logger = LoggerFactory.getLogger(MyController.class);

    @GetMapping(value = "/all")
    public String getAll() {
        logger.info(" **** getAll execution started ****");
        String msg = "Welcome to codeminentech...!!";
        logger.info(" **** getAll execution ended ****");
        return msg;
    }
}
```

Pom

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
  4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.3.3.RELEASE</version>
    <relativePath /> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.demo</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>zipkidemo</name>
  <description>Demo project for Spring Boot</description>
  <properties>
    <java.version>11</java.version>
    <spring-cloud.version>Hoxton.SR8</spring-cloud.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-sleuth</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-zipkin</artifactId>
    </dependency>

  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <image>
            <builder>paketobuildpacks/builder-jammy-base:latest</builder>
          </image>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```

</configuration>
</plugin>
</plugins>
</build>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

</project>

```

Spring boot 3.xx (we not use sleuth we use zipkin with following config)

Controller

```

@RestController
public class MyController {

    Logger logger = LoggerFactory.getLogger(MyController.class);

    @GetMapping(value = "/wel")
    public String Wel() {
        logger.info(" **** wel() execution started ****");
        String msg = "Welcome to codemintech...!!";
        RestTemplate restTemplate = new RestTemplate();
        String result = restTemplate.getForObject("http://localhost:8080/msg", String.class);

        logger.info(" **** wel() execution ended ****");
        return result;
    }

}

```

properties

```

server.port=9090

# Zipkin configuration
spring.zipkin.base-url=http://127.0.0.1:9411/
spring.sleuth.sampler.probability=10
spring.application.name=welcome

```

poM

->when you select zipkin dependency in springboot 3.xx then these 3 dependencies will come.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>io.micrometer</groupId>
<artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>

<dependency>
<groupId>io.zipkin.reporter2</groupId>
<artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

->just run zipkin server from cmd and run our project.

=====

This PDF is meant for quick reference. Since frameworks like Spring Boot evolve frequently & version may change time to time , some syntax, features, or configurations may vary based on the version you are using . Use these notes as a learning reference, not as the only source.